เอกสารพกเข้าห้องสอบ · แบบเปิดตำรา (Open Book)

📘 เอกสารพกเข้าห้องสอบ Final — Operating Systems (ระบบปฏิบัติการ)

05506007 · เรียบเรียงจาก Presentation ของอาจารย์ (ภาพต้นฉบับ) + Handout + วิดีโอ + Quizizz ในห้องเรียน

เน้นละเอียดบทที่ 7–13 · พิมพ์เป็น A4 ได้ (Ctrl+P) · ภาพสไลด์ทุกภาพอ้างอิงเลขสไลด์จริงจาก presentation ของอาจารย์

🖨️ วิธีใช้ + พิมพ์

• เปิดหน้านี้ทิ้งไว้ กด `Ctrl+P` → เลือก Save as PDF → เปิด "Background graphics" เพื่อให้สีสไลด์/ตารางออกครบ

• กล่อง **แดง = จุดหลอกข้อสอบ** · **เขียว = เทคนิค/สูตรทำเร็ว** · **ม่วง = อุปมาช่วยจำ** · ภาพสไลด์ = ข้อมูลต้นฉบับของอาจารย์ (ตัวเลขในโจทย์ตัวอย่างตรงกับสไลด์)

สารบัญ

A. Process (กระบวนการ) & fork (แตกสำเนากระบวนการ)/exec (เปลี่ยนโปรแกรมที่กำลังรัน)/wait (รอ + หยิบสิทธิ์) 🔥

B. Thread (เธรด (สายงานย่อย)) & Multi-Threading

C. CPU Scheduling (การจัดคิว CPU) — Gantt/TAT/WT 🔥

D. Race Condition (สภาพแข่งขันของข้อมูล) & Critical Section (ช่วงวิกฤต (โค้ดแตะของแชร์))

E. Semaphore (ตัวนับควบคุมสิทธิ์เข้าถึง) · Monitor (กล่องควบคุมระดับสูง) · Java (ภาษาจาวา) sync 🔥

F. Deadlock (การติดตาย (รอกันตลอดกาล)) + Banker's Algorithm (อัลกอริทึมนายธนาคาร) 🔥

G. Real Memory (หน่วยความจำ) Management

H. Paging (การแบ่งหน่วยความจำเป็นหน้า) & Address (ที่อยู่หน่วยความจำ) Translation 🔥🔥

I. Virtual Memory & Page Replacement (การเลือกเพจออกเมื่อ RAM เต็ม) 🔥🔥

J. File System (ระบบไฟล์)

K. ความรู้จาก Quizizz ทั้ง 4 ชุด

L. คลังข้อสอบซ้อม Socrative (16 ชุด) 🔥

M. แผ่นสูตรเร็วท้ายเล่ม (1 หน้า)

A. Process & การสร้าง Process ด้วย fork / exec / wait (OSNo03)

📖 รู้จักส่วนนี้ก่อนอ่าน

ความหมาย: Process = โปรแกรมที่ "มีชีวิต" กำลังทำงานอยู่ใน RAM มีสถานะและทรัพยากรของตัวเอง · fork = คำสั่งแตกสำเนา process (กระบวนการ), exec = เปลี่ยนโปรแกรมที่ process กำลังรัน, wait = ให้พ่อรอลูกจบ

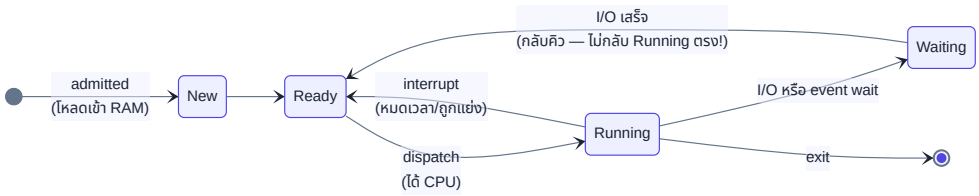
ความสำคัญกับข้อสอบ: ข้อสอบชอบให้ "ไล่โค้ด fork" — นับว่ามี process เกิดกี่ตัว เดาว่า output พิมพ์อะไร รุ่นพี่เตือนว่ามีจุดหลอกเยอะ และย้ำว่าโค้ดทุกข้อ compile ผ่านหมด ไม่มีคำตอบว่า Error

ใช้จริงในคอมพิวเตอร์: ทุกครั้งที่พิมพ์คำสั่งใน terminal หรือดับเบิลคลิกเปิดแอป คือ shell (โปรแกรมรับคำสั่งผู้ใช้) ทำ fork → exec → wait ให้เราทั้งหมด — เช่น พิมพ์ ls, เปิด Chrome, compile ไฟล์

A1. Program vs Process · สถานะของ Process

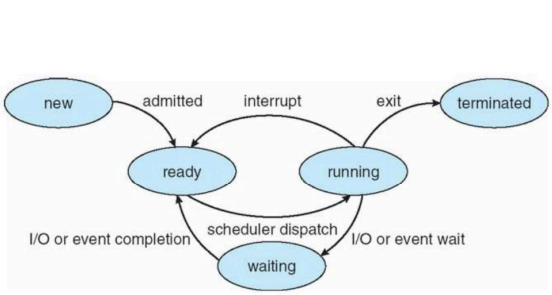
	Program	Process
คือ	ไฟล์คำสั่งนิ่ง ๆ บนดิสก์ — **passive entity**	**program in execution** — active entity ต้องใช้ทรัพยากร
มีอะไร	แค่โค้ด	Program counter (PC), stack, data (ข้อมูล) section, heap, registers
ชื่อเรียกอื่น	—	Task (time-sharing) · Job (batch system)

สถานะของ process (state diagram):

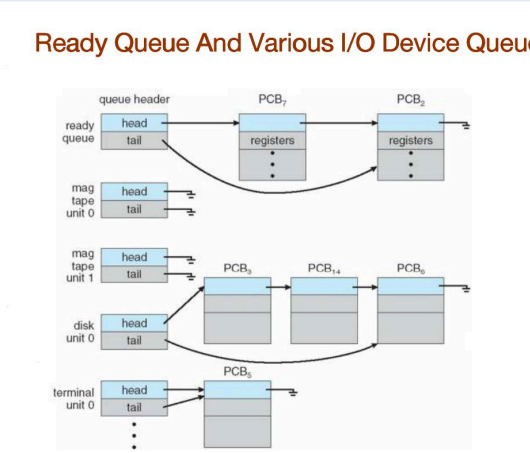


🎯 จุดหลอก

Waiting เสร็จแล้วไป **Ready ก่อนเสมอ** ไม่ใช่กลับ Running ทันที (ต้องรอ dispatcher เรียก) · Batch system เรียก process ว่า Job, time-sharing เรียก Task



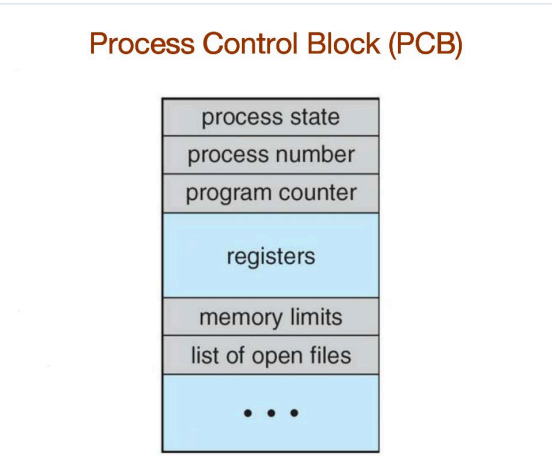
สไลด์ OSNo03 p.4 — Diagram of Process State (ต้นฉบับอาจารย์)



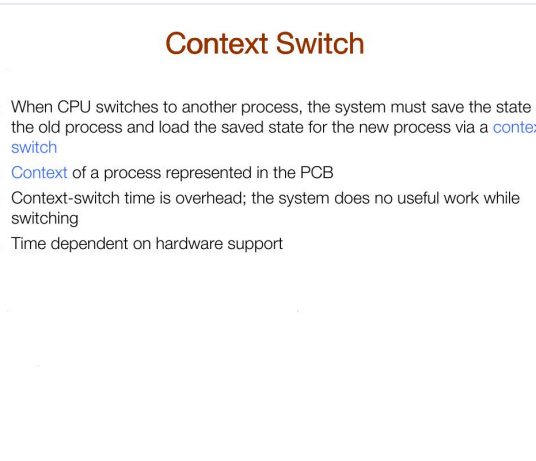
สไลด์ OSNo03 p.7 — Ready Queue (คิวงานที่พร้อมรัน) และ I/O Device Queues: process วิ่งวน Ready → CPU → I/O คิว

A2. PCB — Process Control Block (บล็อก (หน่วยจัดเก็บ))

PCB = "รหัสประชาชน" ของ process เก็บทุกอย่างที่ OS ต้องรู้: **process state, program counter, CPU registers, scheduling (การจัดคิว) info (priority (ลำดับสิทธิ์)/queue), memory (หน่วยความจำ) info (base/limit), accounting info, I/O status**



สไลด์ OSNo03 p.8 — Process Control Block



สไลด์ OSNo03 p.9 — Context Switch (การสลับงานบน CPU): เซฟ state ของ P0 ลง PCB0 → โหลด PCB1 → รัน P1 (ช่วงนี้ = pure overhead)

Dispatcher (ผู้ส่งงานเข้า CPU) ทำ 3 อย่าง

(1) switching context (2) switch to user mode (3) jump ไปตำแหน่งเดิมใน user program · **Dispatch latency** = เวลาที่ dispatcher ใช้หยุดตัวหนึ่ง เริ่มอีกตัวหนึ่ง

A3. fork() — สร้าง process แบบโคลนตัวเอง

`pid_t fork(void)` สร้าง child โดย**ก๊อปปี้ parent** (address (ที่อยู่หน่วยความจำ) space, ตัวแปร, fd ทุกอย่าง) แล้วทั้งคู่รันต่อจากบรรทัดเดียวกัน (หลัง fork)

ใครเรียก	ค่าที่ fork() คืน	เงื่อนไข if
ใน Parent	**PID ของลูก** (> 0)	`if (pid > 0)`
ใน Child	**0**	`else if (pid == 0)`
สร้างไม่สำเร็จ	**ค่าลบ**	`if (pid < 0)` → error

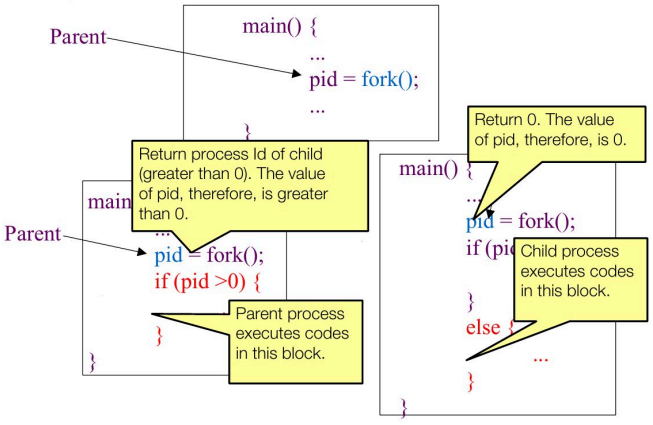
> ✨ **เข้าใจง่าย**
> fork() เรียก**ครั้งเดียวแต่คืนค่า 2 ครั้ง** — ครั้งนึงใน "โลกพ่อ" ครั้งนึงใน "โลกลูก" ทั้งคู่ถือ copy ของตัวแปร pid ของตัวเอง แก้ฝั่งไหนไม่กระทบอีกฝั่ง

```c
/* fork_example.c — ตัวอย่างจากสไลด์/วิดีโอ: ผลลัพธ์พิมพ์ 2 บรรทัด */
#include <stdio.h>
#include <sys/types.h>   // pid_t
#include <unistd.h>      // fork()

int main(void) {
    pid_t pid;                  // เก็บค่า return ของ fork
    int i = 0;                  // ตัวแปรธรรมดา — จะถูก "ก๊อป" ไปให้ลูกด้วย

    pid = fork();               // ← จุดแตกหน่อ: หลังบรรทัดนี้มี 2 process รันโค้ดชุดเดียวกัน
    if (pid > 0) {              // โลกพ่อ: ได้ PID ของลูก (>0) เสมอ
        i = 1;                  // แก้ i ใน address space ของพ่อ — ลูกไม่เห็น
        printf("I am parent i = %d\n", i);
    }
    else {                      // โลกลูก: fork คืน 0
        i = 2;
        printf("I am child i = %d\n", i);
    }
    return 0;
}
/* Output: "I am parent i = 1" และ "I am child i = 2" (2 บรรทัดเสมอ)
   แต่ลำดับก่อน-หลัง non-deterministic — ขึ้นกับ scheduler ห้ามสรุปใครพิมพ์ก่อน */
```

Diagram for fork() example



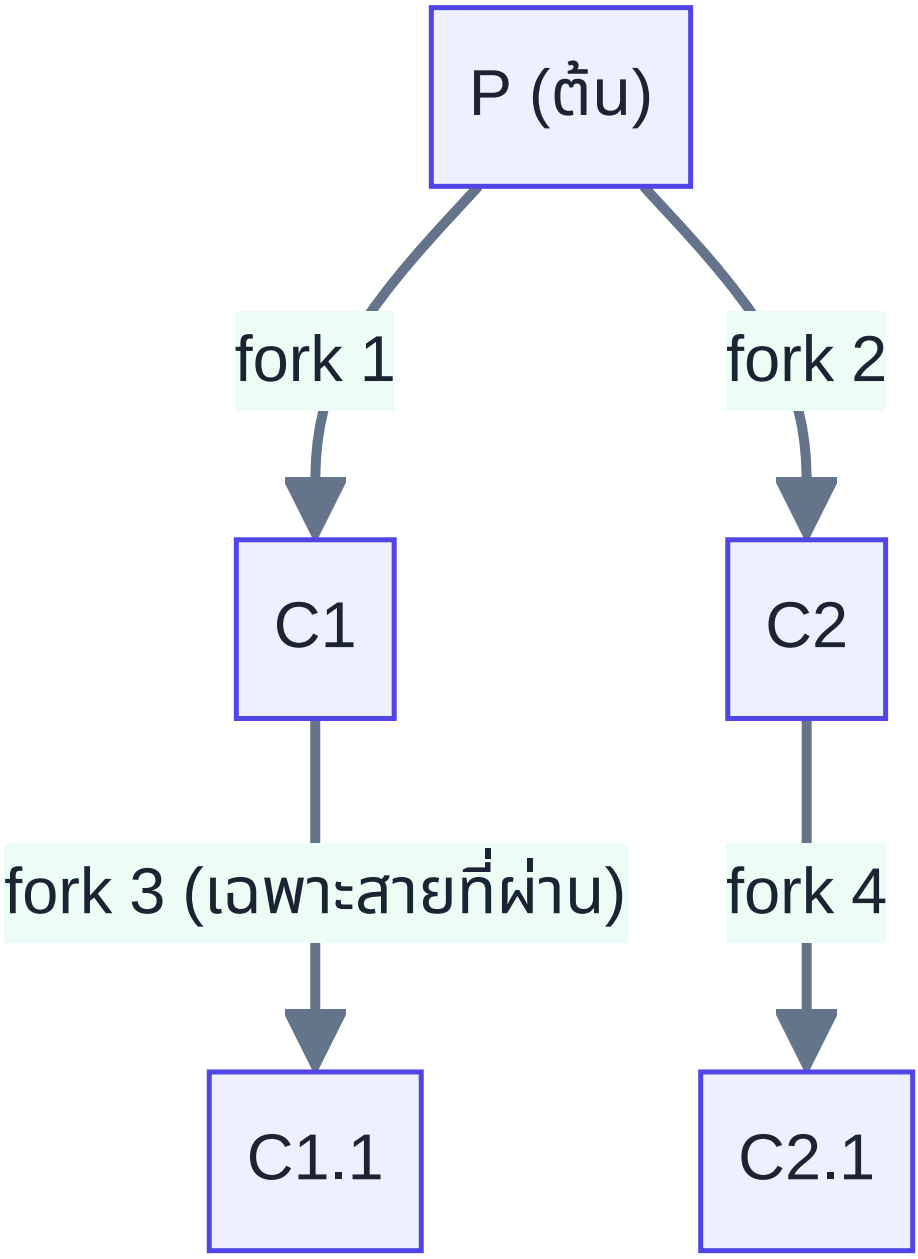
สไลด์ OSNo03 p.19 — Diagram for fork() example: พ่อ/ลูกรันต่อจากจุดเดียวกัน แยกคนละสาย

Process IDs (pids)

- pid = fork();
- In the child: pid == 0;
  In the parent: pid == the process ID of the child.
- If the value of pid is negative, it indicates that the creation of a child process was unsuccessful.
- The application can utilize this difference in the value of pid to execute distinct actions in the parent and child processes.

สไลด์ OSNo03 p.20 — Process IDs: ลูกได้ 0 / พ่อได้ PID ลูก / ติดลบ = fail

A3.1 นับจำนวน process — วาด Process Tree (แผนภาพต้นไม้ความสัมพันธ์พ่อ-ลูก)



`fork()` ติดกัน `n` ครั้ง ไม่มีเงื่อนไข → process ทั้งหมด = 2^n (ลูกใหม่ = $2^n - 1$)

fork 3 ครั้ง = 8 process · ถ้ามี if/exit ให้วาด tree แล้วนับ "สายที่ผ่าน fork จุดนั้น" ทีละจุด

A3.2 fork + loop จากวิดีโอ (จุดหลอก: ลูกต้อง exit)

```
for (i = 0; i < 5; i++) {          // พ่อวน 5 รอบ
    pid = fork();
    if (pid == 0) {                 // ลูกเท่านั้นเข้า if
        if (i % 2 != 0) printf("I am happy\n");  // ลูกที่เกิดตอน i คี่
        else            printf("I am sad\n");    // ลูกที่เกิดตอน i คู่
        exit(0);                    // ← ถ้าลืม! ลูกจะหลุด if ไปวน for ต่อ
    }                               //   แล้ว fork ลูก/หลานของตัวเองเพิ่ม
}
while (wait(NULL) != -1);           // พ่อรอลูกทุกตัว (wait คืน -1 เมื่อไม่มีลูกเหลือ)
printf("I am parent goodbye\n");
```

🎯 จุดหลอก (อาจารย์สาธิตในวิดีโอ)

มี exit() → sad ×3 (i=0,2,4), happy ×2 (i=1,3), goodbye **1 ครั้งสุดท้าย** · ถ้าลูกเลขคี่**ลืม exit()** → ลูก C1, C3 กลายเป็น "พ่อ" สร้างหลาน → goodbye พิมพ์ **4 ครั้ง** (พ่อหลัก + C1 + C3 + หลาน $C1_{-3}$)

A4. exec() — เปลี่ยนร่าง ไม่สร้างคนใหม่

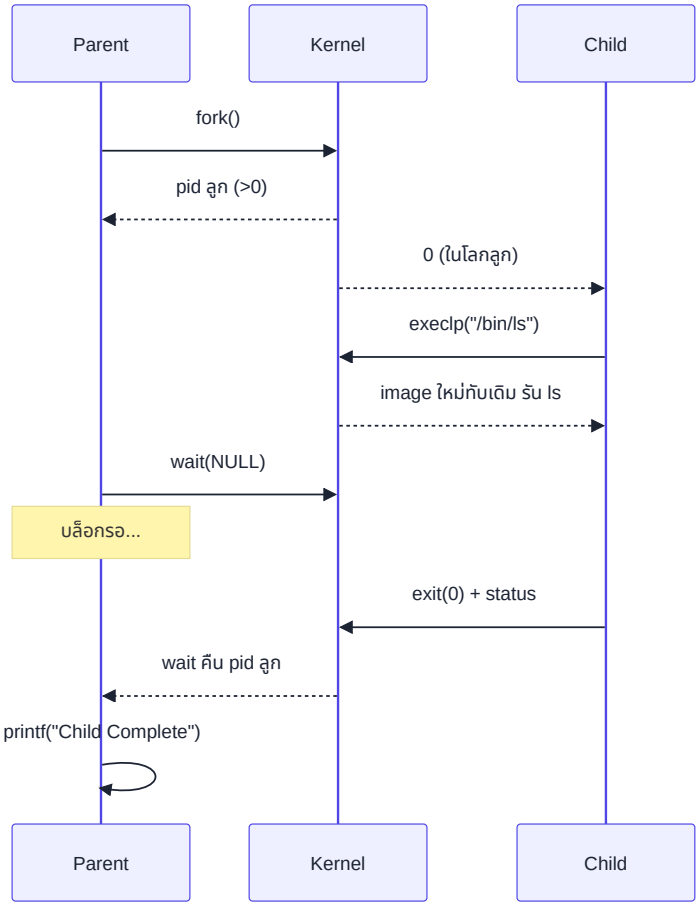
- ตระกูล exec แทนที่ **process image ทั้งหมด** (โค้ด+data) ด้วยโปรแกรมใหม่ — PID คงเดิม
- `execlp(file, arg0, arg1, ..., NULL)` — **l** = list (ส่งทีละตัว) · **p** = หาโปรแกรมตาม PATH · **arg0** = ธรรมเนียมใส่ชื่อโปรแกรม (exec ข้าม แต่ต้องใส่) · **NULL ปิดท้ายบังคับ**
- execv: ส่งเป็น array `char *argv[]`

🎯 จุดหลอก exec

exec **สำเร็จ** → โค้ดหลังบรรทัด exec **ไม่มีวันรัน** (image โดนทับหมด) · exec **ล้มเหลว** (ชื่อผิด เช่น "ly") → รันบรรทัดถัดไปต่อ · `exec("cd")` fail เสมอ เพราะ **cd เป็น shell built-in** ไม่ใช่ไฟล์ (echo ได้เพราะมี /usr/bin/echo)

A5. fork + exec + wait — รูปแบบมาตรฐานของ shell

```c
int main() {
    pid_t pid = fork();
    if (pid < 0) {                          // สร้างลูกไม่สำเร็จ
        fprintf(stderr, "Fork Failed");
        exit(-1);
    }
    else if (pid == 0) {                    // ── ลูก: เปลี่ยนร่างเป็น ls ──
        execlp("/bin/ls", "ls", NULL);      // สำเร็จ = โค้ดลูกถูกแทนที่ทั้งหมด
    }
    else {                                  // ── พ่อ: รอลูก ──
        wait(NULL);                          // บล็อกรอลูกตัวใดตัวหนึ่งจบ (ได้ pid ลูกกลับ)
        printf("Child Complete");           // พิมพ์หลังลูกจบเสมอ
        exit(0);
    }
}
/* Output: ผล ls ก่อน ตามด้วย "Child Complete" — ลำดับแน่นอนเพราะ wait() */
```



A6. exit / wait / waitpid (รอลูกที่ระบุตัวเจาะจง) + สถานะพิเศษ

Call	ทำอะไร	จุดสำคัญ
`exit(status)`	ขอ OS ทำลายตัวเอง ส่ง status ให้พ่อ	ส่งได้แค่ **8 บิต (0–255)** — exit(300) พ่ออ่านได้ 300 mod 256 = 44
`wait(&status)`	รอ*ลูกใดก็ได้*จบ	คืน pid ลูกที่จบ · ไม่มีลูก → คืน **-1** · ไม่สน status ใช้ wait(NULL)
`waitpid(pid, &status, 0)`	รอ*ลูกตัวเจาะจง*	พ่อรายงานผลเรียงลำดับได้ (ต่างจาก wait ที่ได้ใครมาก่อน)
`WIFEXITED(status)`	เช็คลูกจบแบบปกติ?	พิมพ์ status ตรง ๆ ได้เลขแปลก (32768 ฯลฯ) เพราะ OS แปะข้อมูลอื่นปน — **ต้อง decode ด้วยแมโคร**
`WEXITSTATUS(status)`	ดึงค่า exit จริง	

🧟 Zombie (กระบวนการที่ตายแล้วแต่รายการยังค้างในระบบ) process

ลูก**ตายแล้ว**แต่พ่อไม่ wait() → ทรัพยากรถูกคืนแต่รายการใน process table (ตาราง) ค้าง สถานะ **Z / defunct** — ทางแก้: พ่อต้อง wait

👶 Orphan (กระบวนการที่พ่อตายก่อน) process

พ่อ**ตายก่อนลูก** → OS ให้ init/systemd "รับเลี้ยง" (ps -l เห็น PPID เปลี่ยน) · **Cascading termination**: พ่อโดน terminate แล้ว OS บางระบบลบลูกหลานตามทั้งตระกูล

A6.1 ประสานพ่อ-ลูก: wait vs waitpid (จากวิดีโอ parent.c/child.c)

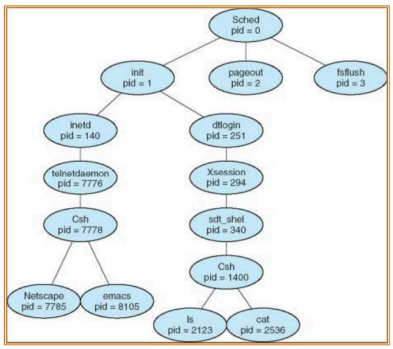
```c
/* child.c: รับ pid ตัวเอง สุ่ม sleep 0-4 วิ แล้ว exit(pid % 256)
   parent.c: fork 3 ลูก (pid[i]) แล้วเปรียบเทียบสองแบบ */
// แบบ 1: wait(&status) → ลูกตัวไหนจบก่อน พ่อรายงานตัวนั้นก่อน (ลำดับไม่แน่)
while ((pid = wait(&status)) > 0) printf("child %d done\n", pid);
// แบบ 2: waitpid(pid[i], &status, 0) → พ่อรอ "ตามลำดับที่กำหนด" เสมอ
for (i = 0; i < 3; i++) {
    waitpid(pid[i], &status, 0);   // นิ่งรอลูกตัวที่ 1 ก่อน แม้ตัว 3 จบก่อน
    printf("child %d done\n", pid[i]);
}
```

Fork and exec()

```c
int main()
{
    pid_t  pid;
    /* fork another process */
    pid = fork();
    if (pid < 0) { /* error occurred */
        fprintf(stderr, "Fork Failed");
        exit(-1);
    }
    else if (pid == 0) { /* child process */
        execlp("/bin/ls", "ls", NULL);
    }
    else { /* parent process */
        /* parent will wait for the child to complete */
        wait (NULL);
        printf ("Child Complete");
        exit(0);
    }
}
```

สไลด์ OSNo03 p.23 — Fork and exec(): โครงโค้ดมาตรฐานจากสไลด์อาจารย์

A tree of processes on a typical Solaris



สไลด์ OSNo03 p.15 — Process tree (login → sh → คำสั่งต่าง ๆ)

B. Thread & การเขียนโปรแกรม Multi-Threaded (OSNo04)

📖 รู้จักส่วนนี้ก่อนอ่าน

ความหมาย: Thread (เธรด/สายงานย่อย) = หน่วยทำงานที่เล็กกว่า process หลาย thread (เธรด (สายงานย่อย)) รันขนานกันภายใน process เดียวกันได้ โดยแชร์หน่วยความจำร่วมกัน

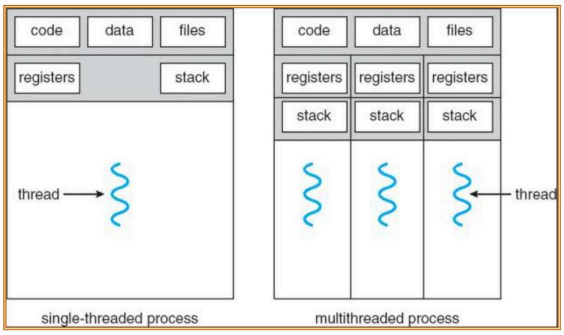
ความสำคัญกับข้อสอบ: ถามโมเดลการ map user thread (thread ฝั่งผู้ใช้) → kernel thread (Many-to-One / One-to-One / Many-to-Many) พร้อมตัวอย่าง OS, อ่านโค้ด Pthread/Java ว่าอะไรแชร์กัน และเชื่อมไปบท Synchronization (การประสานงาน)

ใช้จริงในคอมพิวเตอร์: เบราว์เซอร์ 1 แท็บใช้หลาย thread (โหลดหน้าเว็บ + เล่นวิดีโอ + รับการคลิก พร้อมกัน) · เกมออนไลน์ (เครือข่าย/กราฟิก/ฟิสิกส์แยกกัน) · Thread Pool (บ่อรวมเธรดเตรียมใช้งาน) ใช้ในเว็บเซิร์ฟเวอร์รับคำขอนับพันพร้อมกัน

B1. Thread คืออะไร

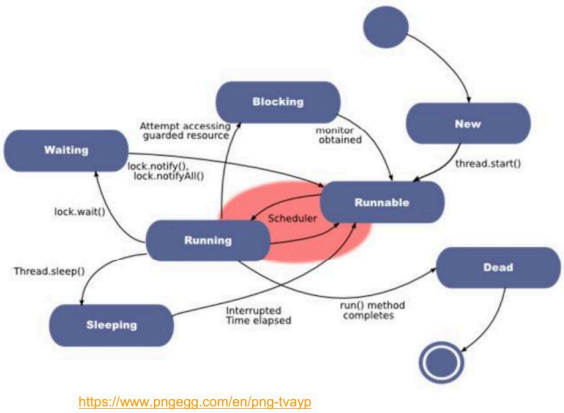
Thread = ลำดับคำสั่งที่รันได้อิสระภายใน process เดียวกัน (lightweight process) · สิ่งที่แต่ละ thread มี**ส่วนตัว**: **program counter, registers, stack** · สิ่งที่**แชร์กัน**: code section, data section, heap, ไฟล์ที่เปิด — thread คือ "หน่วยที่ OS จัดคิวเข้า CPU" ในระบบสมัยใหม่ และ**สลับ thread ถูกกว่าสลับ process**

Single and Multithreaded Processes



สไลด์ OSNo04 p.4 — Single vs Multithreaded: ขวาคือแชร์ code/data/files ร่วมกัน แยกแค่ registers+stack ของแต่ละ thread

Java Thread States



https://www.pngegg.com/en/png-tvayp

สไลด์ OSNo04 p.25 — Java Thread States (New/Runnable/Running/Blocked/Terminated)

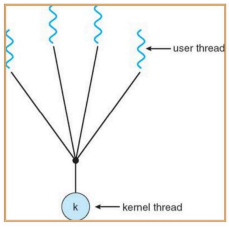
OS แบบไหน	ตัวอย่าง
Single-process single-threaded	**MS-DOS**
Single-process multi-threaded	**JVM**
Multiprocessing single-threaded	Traditional UNIX (ระบบยูนิกซ์)
Multiprocessing multi-threaded	**Modern OS: Windows (ระบบวินโดวส์), Linux (ระบบลินุกซ์), macOS**

B2. User Thread vs Kernel (แกนหลักของ OS) Thread + Multithreading Models

- **User-Level Thread (ULT)**: จัดการโดย thread library ฝั่งผู้ใช้ kernel ไม่รู้จัก — เร็วมาก แต่ถ้าตัวใด block (บล็อก (หน่วยจัดเก็บ)) ทั้ง process หยุด
- **Kernel-Level Thread (KLT)**: kernel จัดการตรง — block ตัวเดียวตัวอื่นยังรัน ขนานจริงได้

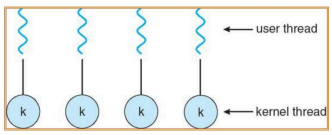
Model	Map	จุดเด่น/จุดพัง	ตัวอย่าง
Many-to-One	ULT หลาย → KLT 1	เร็ว (user space) แต่ block ตัวเดียว = หยุดทั้ง process, ขนานบนหลาย CPU ไม่ได้	Early Unix (ระบบยูนิกซ์)
One-to-One	ULT 1 : 1 KLT	ขนานจริง ตัวใด block ตัวอื่นไม่พัง · สร้าง kernel thread เยอะ = overhead	**Windows NT+, Linux, Solaris 9+**
Many-to-Many	ULT หลาย → KLT หลาย	OS สร้าง KLT ให้พอดีจำนวน	Solaris < 9, Tru64 UNIX

Many-to-One Model



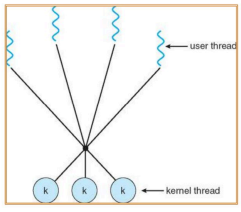
สไลด์ p.14 Many-to-One Model — ULT หลายเส้นรวมเป็น KLT เส้นเดียว

One-to-one Model



สไลด์ p.16 One-to-One Model — ULT 1 เส้น = KLT 1 เส้น

Many-to-Many Model



สไลด์ p.18 Many-to-Many Model — ULT หลายเส้น map ไป KLT หลายเส้น

B3. Pthreads (C) — โค้ดตัวอย่างพร้อมคอมเมนต์

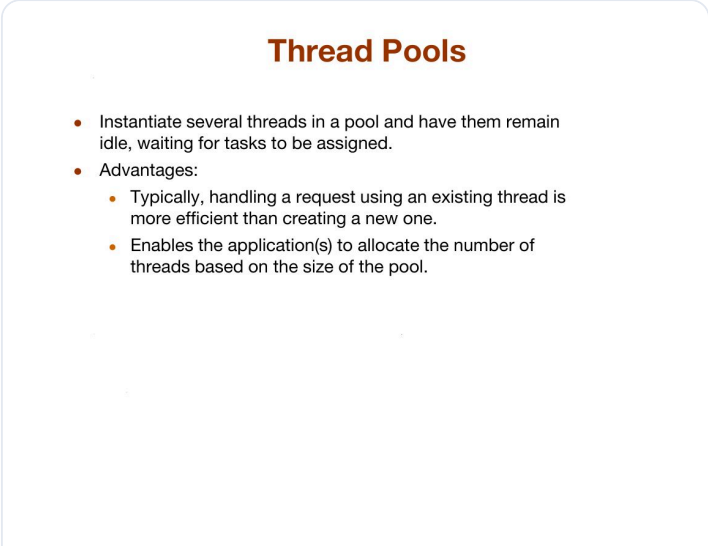
```c
int sum;  // ตัวแปร global — "แชร์" ระหว่าง thread (main กับ runner เห็นตัวเดียวกัน!)

/* ฟังก์ชันที่ thread จะรัน — signature ต้องเป็น void *f(void *) */
void *runner(void *param) {
    int upper = atoi(param);      // รับค่าจาก argv[1] ที่ส่งมาตอน create
    sum = 0;
    for (int i = 1; i <= upper; i++) sum += i;   // หาผลรวม 1..upper
    pthread_exit(NULL);           // จบ thread นี้ (≈ exit ของ process)
}

int main(int argc, char *argv[]) {
    pthread_t tid;                // เก็บ id ของ thread ที่สร้าง
    pthread_attr_t attr;
    pthread_attr_init(&attr);     // ตั้งค่า attribute = default
    pthread_create(&tid, &attr, runner, argv[1]);
    //  ↑ สร้าง thread ให้รันฟังก์ชัน runner โดยส่ง argv[1] เป็น param
    pthread_join(tid, NULL);      // หลักของ main: รอ thread จบก่อน (≈ wait ของ process)
    printf("sum = %d\n", sum);    // เห็นค่า sum ที่ runner แก้ — เพราะแชร์กันจริง
}
/* จับคู่: pthread_create ≈ fork · pthread_join ≈ wait · pthread_exit ≈ exit */
```



สไลด์ OSNo04 p.19 — Pthreads (โค้ดต้นฉบับ)



สไลด์ OSNo04 p.29 — Thread Pools: สร้าง thread เตรียมไว้ใน "บ่อ" รับงานเรื่อย ๆ

Thread Pool (Java 3 แบบ)

Single thread executor (บ่อขนาด 1) · **Fixed thread executor** (ขนาดคงที่) · **Cached thread pool** (ขนาดไม่จำกัด สร้างเพิ่มเอง) — ประโยชน์: ใช้ thread เดิมเร็วกว่าสร้างใหม่ + คุมจำนวน thread ได้

B4. Java Thread — 2 วิธีสร้าง + cancellation

วิธี	โค้ด	หมายเหตุ
extends Thread	`class W extends Thread { public void run() {...} }` → `new W().start()`	เผา slot extends
implements Runnable	`class W implements Runnable { public void run() {...} }` → `new Thread(new W()).start()`	นิยมกว่า

- **Cancellation 2 แบบ: Asynchronous (ยกเลิกทันที)** = ยิงปิดทันที (เสี่ยงข้อมูลครึ่ง ๆ กลาง ๆ) · **Deferred (ยกเลิกแบบให้ตัวมันเช็คเองเป็นระยะ)** = บอกเป้าหมายให้เช็คเองเป็นระยะ (`Thread.currentThread().isInterrupted()`) แล้ว cleanup ก่อนจบ — ปลอดภัยกว่า
- `task.join()` = main รอ task จบก่อนเสมอ (เทียบ wait)

> **🎯 ประโยคทอง Java scheduling**
>
> Java: priority **1–10, default (ค่าเริ่มต้น) 5, มาก = สูง** (Unix ตรงข้าม: น้อย = สูง) · การตั้ง priority **ไม่การันตีลำดับ 100% — ห้ามฝากความถูกต้องของ logic ไว้กับ scheduler (ตัวจัดคิว CPU)** ต้องใช้ Synchronization เท่านั้น

C. CPU Scheduling — Gantt · TAT · WT (OSNo04 สไลด์ท้าย + วิดีโอ 6.x)

> **📖 รู้จักส่วนนี้ก่อนอ่าน**
>
> **ความหมาย:** CPU Scheduling = การตัดสินใจว่า process ไหนจะได้ใช้ CPU "ตัวต่อไป" เมื่อหลายงานแย่งกัน — มีหลายอัลกอริทึม เช่น FCFS (มาก่อนได้ก่อน), SJF (งานสั้นก่อน), RR (วนแบ่งเวลา)
>
> **ความสำคัญกับข้อสอบ:** โจทย์วาด Gantt (แผนภูมิช่วงเวลา) หา TAT/WT เป็นข้อคำนวณหลักประจำทุกปี — ฝึกให้ทำข้อละไม่เกิน 2 นาที ใช้เทคนิค "หา TAT ก่อนแล้วลบ BT เสมอ"
>
> **ใช้จริงในคอมพิวเตอร์:** Windows/Linux/macOS ใช้แนวคิด priority + time slice (โควตาเวลาแบบ RR) จริง ๆ ในตัวจัดคิวของเคอร์เนล — เปิด Task Manager เห็น process แย่ง CPU ด้วยกลไกพวกนี้ทั้งหมด

C1. พื้นฐาน + สูตรทุกตัวที่ใช้คิดเลข

- **CPU Scheduler (ตัวจัดคิว CPU)** เลือก process จาก ready queue ให้ใช้ CPU · **Non-preemptive** = รันจนจบ/ขอออกเอง · **Preemptive (ขัดจังหวะ/แย่งได้)** = ขัดกลางคันได้ (หมดเวลา/มีตัวสำคัญกว่ามา)
- **CPU burst** = ช่วงที่ process ใช้ CPU ติดกัน · I/O-bound = burst สั้นถี่ / CPU-bound = burst ยาว

Turnaround Time (เวลารวมในระบบ): TAT = Completion − Arrival

เวลารวมตั้งแต่งานมาถึง → เสร็จ

Waiting Time (เวลารอคิว): WT = TAT − Burst Time (เวลาที่ต้องใช้ CPU)

🟢 เทคนิคทำเร็ว: หา TAT ก่อนแล้วลบ BT เสมอ — โดยเฉพาะ preemptive ที่งานโดนแทรกหลายรอบ

Response Time (เวลาตอบสนอง) = เวลาที่งานมาถึง → เริ่มได้ response ครั้งแรก

เกณฑ์วัดอื่น: CPU utilization ↑, throughput ↑, TAT ↓, WT ↓, response ↓

C2. FCFS — First-Come, First-Served (non-preemptive)

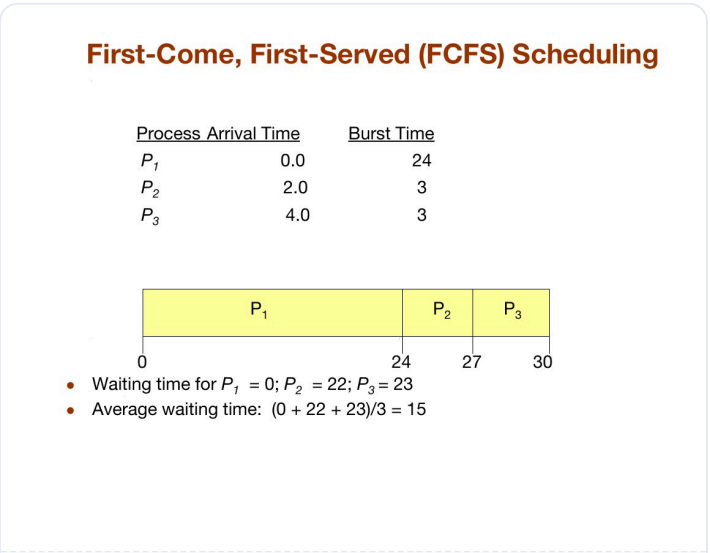
โจทย์ (ตรงจากสไลด์ p.34): P1 (Arrival 0, Burst 24) · P2 (2, 3) · P3 (4, 3)



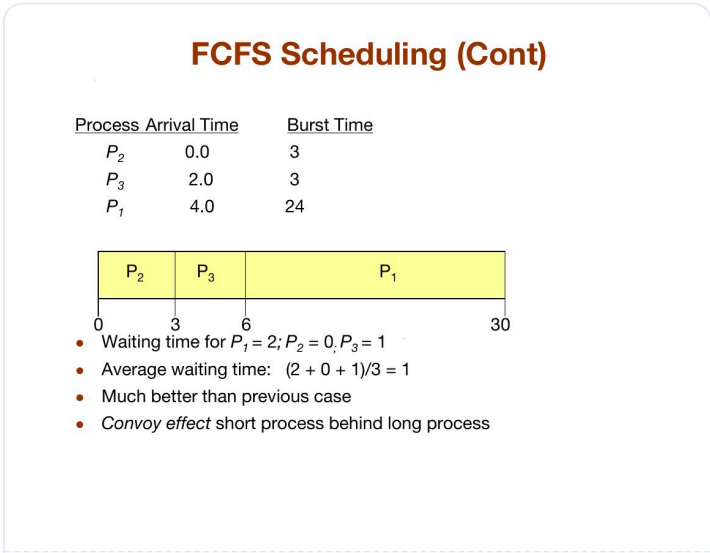
Process	TAT = C − A	WT = TAT − BT
P1	24 − 0 = 24	24 − 24 = **0**
P2	27 − 2 = 25	25 − 3 = **22**
P3	30 − 4 = 26	26 − 3 = **23**
เฉลี่ย	**25**	**(0+22+23)/3 = 15**

🎯 Convoy Effect (อาการคอยตามงานใหญ่)

งานใหญ่ตัวเดียวบล็อกหน้า งานเล็กทั้งบ้านต่อแถวรอ — สไลด์ p.35 เปรียบเทียบ: ถ้างานสั้นมาก่อน (P2 มา 0, P3 มา 2, P1 มา 4) avg WT เหลือ **1** เท่านั้น!



สไลด์ p.34 FCFS

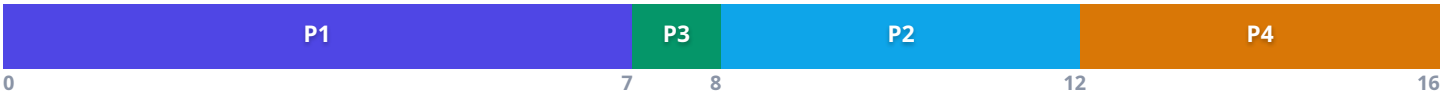


สไลด์ p.35 FCFS — สลับงานสั้นมาก่อน WT=1

C3. SJF — Shortest Job First (non-preemptive) — OPTIMAL

โจทย์ (p.36): P1 (0, 7) · P2 (2, 4) · P3 (4, 1) · P4 (5, 4) — เมื่อ CPU ว่าง เลือก BT สั้นสุดในคิว ณ ตอนนั้น (เท่ากันเลือกมาก่อน)

1. เวลา 0: มีแค่ P1 → รันจนจบที่ 7 (non-preemptive ใครมาทีหลังรอก่อน)
2. เวลา 7: คิว P2(4), P3(1), P4(4) → สั้นสุด P3 → 7→8
3. เวลา 8: P2(4), P4(4) เท่ากัน → มาก่อนชนะ: P2 → 8→12
4. เวลา 12: P4 → 12→16



Process	Completion	TAT	WT = TAT − BT
P1	7	7	7−7 = **0**
P2	12	10	10−4 = **6**
P3	8	4	4−1 = **3**
P4	16	11	11−4 = **7**
เฉลี่ย WT	**—**		**(0+6+3+7)/4 = 4**

📌 SJF คือ optimal

ให้ avg WT ต่ำสุดจากชุดงานหนึ่ง ๆ แต่ใช้จริงยากเพราะ OS ต้อง**ทำนาย**ความยาว CPU burst ถัดไป

C4. SRTF (งานที่เหลือสั้นสุดทำก่อน) — Shortest Remaining Time First (Preemptive SJF) 🔥

กติกา: **งานใหม่มาถึง → เทียบ BT ของงานใหม่ กับ Remaining Time (เวลาที่เหลือ) ของงานที่รันอยู่** — งานใหม่สั้นกว่า = ขัดทันที

1. 0–2: P1 รันไป 2 (เหลือ 5) — P2 มาถึง (BT=4) → เทียบ 4 < 5 → **ขัด P1**
2. 2–4: P2 รันไป 2 (เหลือ 2) — P3 มาถึง (BT=1) → 1 < 2 → **ขัด P2**

3 4–5: P3 จบที่ 5 พอดี + P4 มาถึง (BT=4) → คิว: P1(5), P2(2), P4(4) → รัน P2 จบที่ 7

4 7: เหลือ P1(5), P4(4) → P4 จบที่ 11 → P1 เศษ 5 จบที่ 16

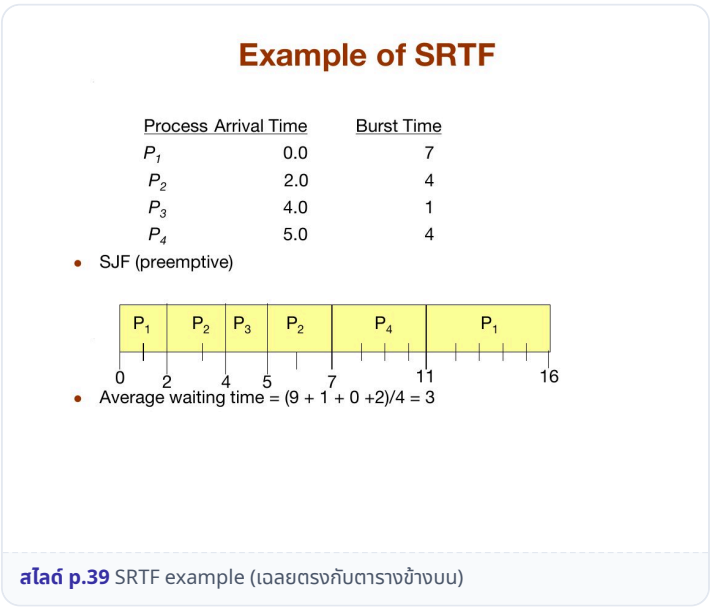


Process	Completion	TAT = C − A	WT = TAT − BT
P1	16	16	16−7 = **9**
P2	7	5	5−4 = **1**
P3	5	1	1−1 = **0**
P4	11	6	6−4 = **2**
เฉลี่ย WT	**—**		**(9+1+0+2)/4 = 3**

> 🎯 **จุดหลอก SRTF**
> เทียบด้วย **Remaining** เสมอ ไม่ใช่ BT ดั้งเดิม! (ตอนเวลา 4 ต้องเทียบ "P2 เหลือ 2" ไม่ใช่ "P2 = 4") · SRTF overhead มากกว่า SJF เพราะ context switch บ่อย



สไลด์ p.38 SRTF นิยาม



สไลด์ p.39 SRTF example (เฉลยตรงกับตารางข้างบน)

C5. Priority (ลำดับสิทธิ์) Scheduling (การจัดคิว)

- CPU ให้งาน priority สูงสุดก่อน (มีทั้ง preemptive/non-preemptive) · SJF = priority ที่ใช้ "predicted next CPU burst" เป็น priority
- **ปัญหา Starvation (การอดตาย (ไม่ได้รันเลย))** (งานสิทธิต่ำไม่ได้รัน) → **แก้ด้วย Aging**: ยิ่งรอนานยิ่งเพิ่ม priority
- **ทิศทางตัวเลข:** Unix-based → **เลขน้อย = สิทธิสูง** / Java Thread → **เลขมาก = สูง** (10 สูงสุด, default 5)

C6. Round Robin (RR) — preemptive, ตอบสนองไว

แต่ละงานรันไม่เกิน **Time Quantum (q)** หมดเวลายังไม่เสร็จ → ถูกตัดไปต่อท้ายคิว · คิวมี n งาน → ไม่มีงานรอเกิน **(n−1)q**

โจทย์ (p.41–42): ข้อมูล FCFS เดิม (P1: 0,24 · P2: 2,3 · P3: 4,3) q = 4



Process	TAT = C − A	WT = TAT − BT
P1	30 − 0 = 30	30 − 24 = **6**
P2	7 − 2 = 5	5 − 3 = **2**
P3	10 − 4 = 6	6 − 3 = **3**
เฉลี่ย WT	**—**	**11/3 ≈ 3.67**

⚖️ ขนาด quantum

q ใหญ่มาก → RR กลายเป็น **FCFS** · q เล็กมาก → **context switch ถี่เกิน overhead โต** (ช่วง switch = pure overhead)

ไทม์ไลน์ RR ทีละจังหวะ

0–4 P1(เหลือ 20) → 4–7 P2 จบ → 7–10 P3 จบ → 10–30 P1 ตัวเดียว รันรอบละ 4 จนครบ 24

Round Robin (RR)

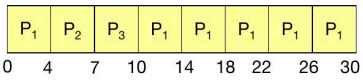
- Every process is allocated a small amount of CPU time, typically ranging from 10 to 100 milliseconds, known as a time quantum.
- Once this time has passed, the process is interrupted and placed at the end of the ready queue
- If the ready queue contains n processes and the time quantum is q, each process is allocated no more than q time units at a time. No process waits more than $(n-1)q$ time units.
- Performance
  - q large ⇒ FIFO
  - q small ⇒ q must be large with respect to context switch, otherwise overhead is too high

สไลด์ p.41 Round Robin

Example of RR with Time Quantum = 4

Process	Arrival Time	Burst Time
P_1	0.0	24
P_2	2.0	3
P_3	4.0	3

- The Gantt chart is:



- Typically, higher average turnaround than SJF, but better *response*

สไลด์ p.42 Example RR q=4 (เฉลยตรงกับตาราง)

C7. Multilevel Queue (คิวหลายชั้น) & Multilevel Feedback Queue (MLFQ)

- **Multilevel Queue**: แยก ready queue หลายคิว (เช่น foreground=RR / background=FCFS) — **งานอยู่คิวไหนตลอดชีวิต ย้ายไม่ได้** · ระหว่างคิว: fixed priority (คิวหน้าจบก่อน — เสี่ยง starvation) หรือแบ่งเวลา (80/20)
- **MLFQ**: **งานย้ายคิวได้** → ทำ aging ได้ จัดกลุ่มตามพฤติกรรมได้

โมเดลตัวอย่าง (p.46–47): Q0 = RR q8ms → ไม่จบ ลง Q1 = RR q16ms → ยังไม่จบ ลง Q2 = FCFS จนจบ · คิวล่างได้รันเมื่อคิวบนว่างหมด

Multilevel Queue Scheduling

- Multilevel Queue Scheduling is a scheduling algorithm that partitions the ready queue into several separate queues.
- Each queue can have its own scheduling algorithm. For Examples
  - foreground – RR
  - background – FCFS
- This method is used when processes can be easily categorized into different classes with different scheduling needs.
- No Movement Between Queues: Processes do not move between queues. Once assigned to a queue, a process stays there.

สไลด์ p.44 Multilevel Queue

Example of Multilevel Feedback Queue

- Three queues:
  - Q_0 – RR with time quantum 8 milliseconds
  - Q_1 – RR time quantum 16 milliseconds
  - Q_2 – FCFS
- Scheduling
  - A new job enters queue Q_0 which is served FCFS. When it gains CPU, job receives 8 milliseconds. If it does not finish in 8 milliseconds, job is moved to queue Q_1.
  - At Q_1 job is again served FCFS and receives 16 additional milliseconds. If it still does not complete, it is preempted and moved to queue Q_2.

สไลด์ p.47 Example MLFQ 3 คิว

🎯 สรุป 5 จุดพลาดของบทนี้

1) ลบ Arrival ไม่ครบ (เอาเวลาเริ่มรันตอบเป็น WT) 2) WT ใน preemptive ไล่นับมือผิด → ใช้ WT=TAT−BT 3) ตี priority ผิดฝั่ง 4) SRTF ใช้ BT ดิบแทน remaining 5) เลือก quantum ผิด

D. Race Condition & Critical Section (OSNo06 ต้น ๆ)

📖 รู้จักส่วนนี้ก่อนอ่าน

ความหมาย: Race Condition = ปัญหาที่ 2 process แก้ข้อมูลแชร์พร้อมกันจนผลลัพธ์เพี้ยนตามใครแก้ก่อน-หลัง · Critical Section (ช่วงวิกฤต) = ช่วงโค้ดที่แตะข้อมูลแชร์ ห้ามใครเข้าพร้อมกัน

ความสำคัญกับข้อสอบ: ไล่ timeline การแย่งตัวแปร count (S0–S5), ท่องเงื่อนไข 3 ข้อ (Mutual Exclusion (เข้าถึงได้คนเดียว) / Progress / Bounded Waiting (รอได้จำกัดจำนวนครั้ง)), อธิบาย Peterson's Solution (โซลูชันสำหรับ 2 กระบวนการ) ว่าครบ 3 ข้อยังไง — ออกทั้งถูก/ผิดและไล่ค่า

ใช้จริงในคอมพิวเตอร์: ระบบธนาคารที่โอนเงินพร้อมกัน 2 คน, ตั๋วหนังที่นั่งสุดท้ายถูกซื้อพร้อมกัน, ตัวนับยอดวิว — ถ้าไม่ล็อก ข้อมูลเสียแบบใน timeline เลย

D1. จุดเริ่ม: Bounded Buffer (บัฟเฟอร์ขนาดจำกัด) ยังไม่มี sync (Producer (ฝั่งผู้ผลิตข้อมูล)-Consumer (ฝั่งผู้บริโภคข้อมูล))

Producer-Consumer แชร์ buffer วงแหวนขนาด BUFFER_SIZE ช่อง + ตัวนับ `count` — producer เขียนแล้ว count++ / consumer อ่านแล้ว count−−

```
/* Producer (สไลด์ p.3)                    Consumer (สไลด์ p.4) */
while (true) {                            while (true) {
    while (count == BUFFER_SIZE);             while (count == 0);   // busy wait
    buffer[in] = nextProduced;                nextConsumed = buffer[out];
    in = (in + 1) % BUFFER_SIZE;              out = (out + 1) % BUFFER_SIZE;
    count++;                                  count--;
}                                         }
/* ปัญหา: ข้อมูลหาย (producer เขียนทับที่ยังไม่อ่าน) / อ่านซ้ำ — และ count ไม่ atomic */
```

D2. Race Condition — ไล่ S0–S5 ให้เห็นตา 🔥

`count++` ในระดับเครื่องแตกเป็น 3 คำสั่ง (ไม่ atomic!):

```
/* count++: */            /* count--: */
register1 = count;         register2 = count;
register1 = register1 + 1; register2 = register2 - 1;
count = register1;         count = register2;
```

ไทม์ไลน์จากสไลด์ p.5 (เริ่ม count = 5):

จังหวะ	ใครทำอะไร	ค่าที่เกี่ยวข้อง
S0	producer: register1 = count	r1 = 5
S1	producer: r1 = r1 + 1	r1 = 6
S2	consumer: register2 = count	r2 = **5** ⚠️ (ยังเก่า!)
S3	consumer: r2 = r2 − 1	r2 = 4
S4	producer: count = r1	count = 6
S5	consumer: count = r2	count = **4** ❌ (ควรได้ 5)

> 💳 **อุปมา**
> เงินในบัญชี 5,000 · คุณโอนเข้า +1,000 แฟนถอน −1,000 พร้อมกัน แอปให้ทั้งคู่เห็นยอด 5,000 พร้อมกันแล้วบันทึกทับ → เหลือ 4,000 เงินหายในอากาศ!

> 📖 **นิยามที่ต้องพิมพ์ได้**
> **Race condition**: หลาย process เข้าถึง/จัดการข้อมูลร่วมกันพร้อมกัน แล้ว**ผลลัพธ์ขึ้นกับลำดับการเข้าถึง** · โค้ดช่วงที่แตะข้อมูลแชร์ = **critical section (critical region)**

D3. Critical Section — เงื่อนไข 3 ข้อที่โซลูชันต้องผ่าน 🔥

เงื่อนไข	ความหมาย (พิมพ์ได้ต้องได้)
1. Mutual Exclusion	ถ้า P_i อยู่ใน CS (ช่วงวิกฤต) → **ห้ามมี process อื่นอยู่ใน CS ของมันพร้อมกัน**
2. Progress	ถ้า*ไม่มีใคร*อยู่ใน CS และมีคนอยากเข้า → ต้องเลือกจาก*คนที่อยากเข้าเท่านั้น* และตัดสินไม่ล่าช้าไม่สิ้นสุด (กันคนไม่สนมาขวาง / กันไม่มีใครเข้าเลย)
3. Bounded Waiting	มีขอบเขตว่า process หนึ่งถูก "ตัดหน้า" ได้**จำกัดกี่ครั้ง**หลังยื่นขอ (กัน starvation ของคิว CS)

โครงโค้ด: ENTRY SECTION → CRITICAL SECTION → EXIT SECTION → REMAINDER SECTION (วน while(true)) · สมมติ: แต่ละ process รันด้วยความเร็วไม่เป็นศูนย์ แต่**ไม่มีข้อสมมติเรื่องความเร็วสัมพัทธ์**

D4. Peterson's Solution (2 process)

แชร์ 2 ตัวแปร: `int turn` (ตาใคร) + `boolean flag[2]` (ใครอยากเข้า) — สมมติ LOAD/STORE **atomic**

```
/* โค้ดของ Pᵢ (อีกฝั่งคือ Pⱼ) — สไลด์ p.9–10 */
while (true) {
    flag[i] = TRUE;              // 1. ประกาศว่า "ฉันอยากเข้า!"
    turn = j;                    // 2. ยกสิทธิ์ให้เพื่อนก่อน (ความสุภาพ = หัวใจ)
    while (flag[j] && turn == j); // 3. รอเฉพาะเมื่อ: เพื่อนอยากเข้า "และ" ถึงตาเพื่อน

    CRITICAL SECTION

    flag[i] = FALSE;             // 4. ออกแล้ว — ปล่อยให้คนอื่นเข้าได้
    REMAINDER SECTION
}
```

เงื่อนไข	Peterson ทำไม่ผ่านทำไมยังไง
Mutual Exclusion ✓	ทั้งคู่ตั้ง turn พร้อมกันไม่ได้ — คนที่ตั้ง turn เป็นฝ่ายอื่น**เป็นคนสุดท้าย** ต้องรอก่อน
Progress ✓	เงื่อนไขรอต้องเจอ flag[j] = true จริง — คนที่ไม่สนใจเข้าไม่ขวาง
Bounded Waiting ✓	หลังออกจาก CS อีกฝั่งได้เข้าภายใน 1 รอบ (turn สลับเสมอ)

> 📌 **ในโลกจริง**
> Peterson เวิร์คในทฤษฎี แต่ CPU สมัยใหม่ (compiler optimization, out-of-order) อาจพัง — ใช้จริงด้วยฮาร์ดแวร์/ไลบรารี แต่**ออกสอบเยอะ**เพราะสอนหลักคิด

D5. Bakery Algorithm (n process) — จ่ายคิวร้านเบเกอรี่

- ก่อนเข้า CS จ่ายเลขคิว = max(number[0..n−1]) + 1 → คนถือเลข**น้อยสุด**เข้าก่อน · เลขซ้ำเทียบ **PID** มาก่อนชนะ

- เทียบแบบ lexicographical: `(a,b) < (c,d)` ถ้า a < c หรือ (a = c และ b < d) — (เลขคิว, pid)
- แชร์: `boolean choosing[n]`, `int number[n]` (เริ่ม false/0)

```
do {
    choosing[i] = true;                          // กำลังจ่ายเลข (ยังไม่นับคิวจริง)
    number[i] = max(number[0..n-1]) + 1;          // จ่ายเลขใหญ่กว่าใคร ๆ +1 (อาจซ้ำ)
    choosing[i] = false;
    for (j = 0; j < n; j++) {
        while (choosing[j]);                      // รอคนที่กำลังจ่ายเลขจบก่อน
        while ((number[j] != 0) && ((number[j],j) < (number[i],i)));
        // รอถ้า: เพื่อนถือคิวอยู่ และ (เลขน้อยกว่า หรือ เลขเท่าแต่ pid น้อยกว่า)
    }
    CRITICAL SECTION
    number[i] = 0;                               // ออก → ยกเลิกคิวตัวเอง
    REMAINDER SECTION
} while (true);
```

เลขคิวอาจกระโดด (1,2,3,3,3,4) — ไม่เป็นไร lexicographical จัดการเคสซ้ำ · รับประกันครบ 3 เงื่อนไขสำหรับ n process

E. Semaphore · ปัญหาคลาสสิก · Monitor · Java Synchronization (OSNo06 ท้าย ๆ) 🔥

📖 รู้จักส่วนนี้ก่อนอ่าน

ความหมาย: Semaphore (ตัวนับควบคุมคิว) = เครื่องมือที่ OS ให้มาจัดคิวเข้าถึงของแชร์อย่างปลอดภัย มีคำสั่ง wait (หยิบสิทธิ์) / signal (คืนสิทธิ์) · Monitor = กล่องควบคุมระดับสูงที่ล็อกให้อัตโนมัติ

ความสำคัญกับข้อสอบ: โค้ด Bounded Buffer / Readers-Writers (ปัญหาคนอ่าน-คนเขียน) / Dining Philosophers (ปัญหานักปราชญ์กินข้าว) ต้องจำค่าเริ่มต้นให้แม่น (mutex=1, empty=n, full=0) + จุดหลอก "สลับลำดับ wait → deadlock (การติดตาย (รอกันตลอดกาล))" — ออกถี่มาก

ใช้จริงในคอมพิวเตอร์: คิวงานเครื่องปริ้นต์, connection pool ของฐานข้อมูล, ปุ่ม like ที่คนกดพร้อมกันหลายพันคน — ระบบจริงทั้งหมดใช้ semaphore (ตัวนับควบคุมสิทธิ์เข้าถึง)/mutex คุมจังหวะ

E1. Semaphore — wait / signal (อุปมาโหลหิน 🪨)

```
wait(S) {                 // P — "หยิบหินจากโหล"
    while (S <= 0);       // โหลว่าง → ยืนรอ (busy wait)
    S--;
}
signal(S) {               // V — "คืนหินใส่โหล" แล้วปลุกคนรอ
    S++;
}
```

- **Binary semaphore** = ค่า 0/1 เท่านั้น อีกชื่อคือ **mutex lock (การล็อก)** → ใช้กับ critical section (เข้าได้ทีละคน)
- **Counting semaphore** = ค่า 0..k → ใช้กับ resource (ทรัพยากร) หลายหน่วย (k = จำนวน resource)

🎯 กับดักจาก Quizizz ชุด 3

(1) CS ใช้ **binary** ไม่ใช่ counting (2) Semaphore การันตีได้แค่ **Mutual Exclusion** — progress/bounded waiting ขึ้นกับ implementation

E2. Implement แบบไม่มี busy waiting (waiting queue + block/wakeup)

```
/* แต่ละ semaphore มี waiting queue (แต่ละรายการเก็บ value + pointer รายการถัดไป) */
wait(S) {
    S.value--;                    // หยิบหินก่อนเลย
    if (S.value < 0) {            // ติดลบ = หินไม่พอ มีคนรอแล้ว
        เพิ่มตัวเองเข้า S.list;       // ต่อคิว
        block();                  // หลับ — ไม่เผา CPU
    }
}
signal(S) {
    S.value++;                    // คืนหิน
    if (S.value <= 0) {           // ยังมีคนรออยู่
        หยิบ P จาก S.list;
        wakeup(P);                // ปลุก 1 คนกลับ ready queue
    }
}
/* 💡 ค่า value ติดลบ = มี process รออยู่ |value| ตัว */
```

E3. ปัญหาคลาสสิก 1: Bounded Buffer 🔥 (semaphore 3 ตัว)

Semaphore	เริ่มต้น	ชนิด	หน้าที่
`mutex`	**1**	binary	ล็อก buffer เข้าแตะทีละคน
`empty`	**n**	counting	นับช่องว่าง
`full`	**0**	counting	นับช่องที่มีข้อมูล

```
/* Producer (สไลด์ p.20–21) */
while (true) {
    // produce an item
    wait(empty);   // ขอช่องว่าง
    wait(mutex);   // ล็อก buffer
    // เขียนลง buffer
    signal(mutex); // ปลดล็อก
    signal(full);  // บอกว่ามีของแล้ว
}
```

```
/* Consumer (สไลด์ p.21) */
while (true) {
    wait(full);    // มีของไหม?
    wait(mutex);
    // ดึงของออกจาก buffer
    signal(mutex);
    signal(empty); // คืนช่องว่าง
    // consume item
}
```

🎯 จุดหลอก: สลับ wait → DEADLOCK

ถ้า producer เขียน `wait(mutex); wait(empty);` — buffer เต็ม → producer ครอง mutex ค้างรอ empty แต่ consumer เข้าอ่านไม่ได้เพราะ mutex ถูกครอง → รอกันตาย · **กฎเหล็ก: เช็ค resource (empty/full) ก่อนล็อก (mutex) เสมอ** · จำค่าเริ่มต้นให้แม่น: mutex=1, empty=n, full=0

E4. ปัญหาคลาสสิก 2: Readers-Writers

กติกา: **Reader หลายตัวอ่านพร้อมกันได้** · Writer ทำงานเมื่อไรห้ามใครยุ่งเด็ดขาด · แชร์: `wrt=1, mutex=1, readcount=0 (int ธรรมดา)`

```
/* Writer — ง่ายสุด */
wait(wrt);
// เขียนข้อมูล
signal(wrt);
```

```
/* Reader — "ตัวแรกล็อก ตัวสุดท้ายปลด" */
wait(mutex);              // แตะ readcount ทีละคน
readcount++;
if (readcount == 1)       // reader แรกล็อก writer
    wait(wrt);
signal(mutex);
// อ่านข้อมูล (พร้อมกันได้หลายคน)
wait(mutex);
readcount--;
if (readcount == 0)       // reader สุดท้ายปลดล็อก
    signal(wrt);
signal(mutex);
```

> **🎯 จาก Quizizz 3**
> semaphore ที่ควบคุมการเพิ่ม/ลดจำนวน reader คือ **mutex** (ครอบ readcount) — **readcount เป็น int ธรรมดา** ไม่ใช่ semaphore · ที่คุม writer คือ wrt

E5. ปัญหาคลาสสิก 3: Dining Philosophers (นักปราชญ์ 5 คน)

ตะเกียบ 5 อัน = **resource** ใช้ semaphore 1 ตัวต่อ 1 ตะเกียบ เริ่มทุกตัว = 1 · กินได้ต้องมีครบ 2 ข้าง (ซ้าย i, ขวา (i+1)%5)

```
/* โค้ดนักปราชญ์ i (สไลด์ p.25–26) */
while (true) {
    wait(chopstick[i]);              // หยิบซ้าย
    wait(chopstick[(i + 1) % 5]);    // หยิบขวา
    // กินข้าว 🍚
    signal(chopstick[i]);            // วางซ้าย
    signal(chopstick[(i + 1) % 5]); // วางขวา
    // คิดปรัชญา
}
```

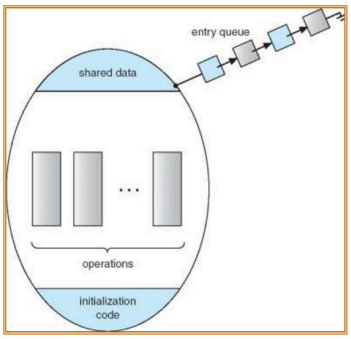
> **🎯 โซลูชัน naive นี้พัง**
> ทุกคน**หิวพร้อมกัน หยิบซ้ายพร้อมกัน** → ไม่เหลือตะเกียบว่าง → ทุกคนรอขวาเป็นวงกลม = **deadlock** (+ ไม่มี bounded waiting → starvation ได้) · แก้: จำกัดจำนวนคนนั่ง ≤ 4 / หยิบพร้อมกันเท่านั้น / ใช้ Monitor (ล่าง)

E6. Monitor — เกราะระดับสูง + Condition (เงื่อนไข) Variables

- Monitor = shared data + ฟังก์ชันรวมในก้อนเดียว (คล้าย class) — **รับประกัน mutual exclusion อัตโนมัติ: 1 process ใน monitor ต่อ 1 เวลา** (ลืมล็อกไม่ได้!)
- **Condition variable x:** `x.wait()` = สลัดล็อก หลับไปรอในคิวของ x · `x.signal()` = ปลุก 1 ตัวในคิว x

> **🎯 ต่างจาก semaphore ยังไง**
> **x.signal() ถ้าไม่มีใครรอ** → **สัญญาณหายไปเฉย ๆ** (ไม่มี counter เก็บยอด) ต่างจาก semaphore ที่ signal สะสมค่า

Schematic view of a Monitor



สไลด์ p.29 Schematic view of a Monitor: คิว entry → ใน monitor มีได้ 1 คน

Condition Variables

- condition x, y;
- Two operations on a condition variable:
  - x.wait () – a process that invokes the operation is suspended.
  - x.signal () – resumes one of processes (if any) that invoked x.wait ()

สไลด์ p.30 Condition Variables x.wait() / x.signal()

E6.1 แก้ Dining Philosophers ด้วย Monitor (กัน deadlock 100%)

```
monitor DP {
    enum {THINKING, HUNGRY, EATING} state[5];   // สถานะของแต่ละคน
    condition self[5];                          // คิวรอของแต่ละคน

    void pickup(int i) {
        state[i] = HUNGRY;
        test(i);                                // ลองดูว่ากินได้ไหม
        if (state[i] != EATING) self[i].wait;   // ยังไม่ได้ → หลับรอ
    }
    void putdown(int i) {
        state[i] = THINKING;
        test((i + 4) % 5);                      // เช็คเพื่อนบ้านซ้าย
        test((i + 1) % 5);                      // เช็คเพื่อนบ้านขวา
    }
    void test(int i) {
        // กินได้เมื่อ: เพื่อนซ้ายไม่กิน และ ฉันหิว และ เพื่อนขวาไม่กิน
        if ( (state[(i + 4) % 5] != EATING) &&
             (state[i] == HUNGRY) &&
             (state[(i + 1) % 5] != EATING) ) {
            state[i] = EATING;
            self[i].signal();
        }
    }
}
/* ใช้: dp.pickup(i) → EAT → dp.putdown(i)
   ไม่มีใคร "ถือตะเกียบเดี่ยวค้าง" → ไม่มี deadlock แน่นอน (แต่ starvation ยังเป็นไปได้) */
```

E7. Java Synchronization — synchronized (ล็อกอัตโนมัติ (คีย์เวิร์ด Java)) · wait · notify (ปลุก 1 thread ที่รออยู่)

- ทุก object มี **lock** — เรียก `synchronized` method ต้องได้ lock ก่อน · ตกคิวไปรอใน **Entry Set (คิวรอเข้าล็อก)** · ออกจาก method = คืน lock อัตโนมัติ
- `wait()` ทำ 3 อย่าง: **(1) คืน object lock (2) สถานะ blocked (3) ย้ายเข้า Wait Set (คิวหลับรอเงื่อนไข)**
- `notify()` = **สุ่ม** 1 thread จาก Wait Set → ย้ายไป Entry Set (รอแย่ง lock ใหม่ — **ไม่ได้รันทันที!** และเลือกตัวเองไม่ได้) · `notifyAll()` = ปลุก**ทุกตัว** (ปลอดภัยกว่าเมื่อ wait set มีหลายตัว)
- Java 5+ มี `Semaphore` class จริง: `acquire()` ≈ wait, `release()` ≈ signal

```java
/* insert() ด้วย wait/notify (สไลด์ p.41) */
public synchronized void insert(Object item) {
    while (count == BUFFER_SIZE) {
        try { wait(); }                    // เต็ม → คืนล็อก หลับรอ
        catch (InterruptedException e) { }
    }
    ++count;
    buffer[in] = item;
    in = (in + 1) % BUFFER_SIZE;
    notify();                              // ปลุกคนที่รออยู่ 1 ตัว
}
```

```java
/* remove() (สไลด์ p.42) */
public synchronized Object remove() {
    Object item;
    while (count == 0) {
        try { wait(); }                    // ว่าง → หลับรอ
        catch (InterruptedException e) { }
    }
    --count;
    item = buffer[out];
    out = (out + 1) % BUFFER_SIZE;
    notify();
    return item;
}
```

🎯 ตารางแยกพวก "wait" ที่หลอกบ่อยสุด

wait ของใคร	ทำอะไร
`wait(S)` ของ Semaphore	หยิบหิน — ลดค่า S
`wait()` ของ Java/Monitor	สลัด lock + หลับใน Wait Set
`wait()` ของ Process (บท A)	พ่อรอลูกจบ เก็บ exit status

แถม: `Thread.yield()` แค่ยอมออกจาก CPU — **ยังถือ lock ค้างอยู่!** (เหมือนล็อกห้องแล้วปีนหน้าต่างหนี)

F. Deadlock + Banker's Algorithm (OSNo07) 🔥

📖 รู้จักส่วนนี้ก่อนอ่าน

ความหมาย: Deadlock (การติดตาย) = กลุ่ม process รอทรัพยากรกันเป็นวงกลม ไม่มีใครได้ทำงานต่ออีกเลย · Banker's Algorithm = วิธี "เช็กก่อนให้" ว่าจัดสรรแล้วระบบยังปลอดภัยไหม

ความสำคัญกับข้อสอบ: เงื่อนไข 4 ข้อ (ต้องครบพร้อมกัน), อ่าน Resource-Allocation Graph (กราฟแสดงการจัดสรรทรัพยากร), และไล่ Safety Algorithm (อัลกอริทึม (ขั้นตอนวิธี)) หา Safe Sequence (ลำดับการรันที่ปลอดภัย) — เป็นแนวคำนวณหลักของ Final

ใช้จริงในคอมพิวเตอร์: ฐานข้อมูลล็อกแถวข้อมูล 2 ธุรกรรมรอกัน, เปิดไฟล์ Excel 2 โปรแกรมล็อกกัน, รถติด 4 แยกไม่มีใครยอมถอย — ระบบจริงเลือก "เมิน + ให้คนรีสตาร์ทเอง" เหมือนที่ OS ทำ

F1. Deadlock คืออะไร + ตัวอย่างรากฐาน

Deadlock = process ≥ 2 ตัว**รอกันไม่สิ้นสุด** เพราะเหตุการณ์ที่จะปลุกนั้นอยู่ในมือกลุ่มผู้รอเอง

```
/* ตัวอย่างโค้ดจากสไลด์ p.4 — S, Q เป็น binary semaphore เริ่ม 1 ทั้งคู่ */
/* P0 */                    /* P1 */
wait(S);   ← S=0 ยึด S      wait(Q);   ← Q=0 ยึด Q
wait(Q);   รอ Q (P1 ถือ)    wait(S);   รอ S (P0 ถือ) → ตายล็อกทั้งคู่
...
signal(S); signal(Q);       signal(Q); signal(S);
```

- **Starvation** = indefinite blocking — process อาจไม่เคยถูกหยิบออกจากคิว semaphore เลย (ต่างจาก deadlock ที่รอกันเป็นวง)
- สไลด์ p.3 สะพานแคบ: รถ = process, ช่วงถนน = resource, แก้โดยให้คันหนึ่ง**ถอย (preempt + rollback)**
- **OS ส่วนใหญ่ (รวม UNIX) ไม่ป้องกัน/ไม่จัดการ deadlock เลย** — ปล่อยให้โปรแกรมเมอร์รับผิดชอบ

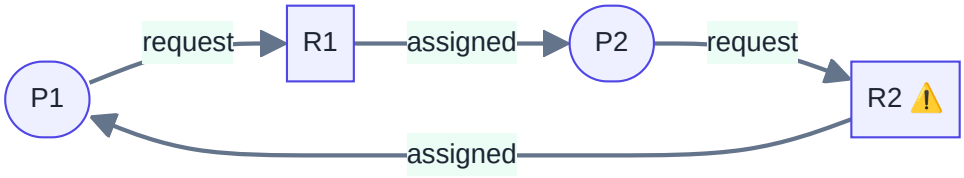
F2. เงื่อนไข 4 ข้อ — ต้องเกิด "พร้อมกัน" จึงเกิด deadlock 🔥

เงื่อนไข	ความหมาย	Prevention ยังไง
1. Mutual Exclusion	ทรัพยากร 1 ชิ้นมีได้ 1 เจ้าของ/เวลา	ทำให้แชร์ได้ — ทำไม่ได้กับทุกชนิด
2. Hold and Wait (ถือของเก่า รอของใหม่)	ถืออย่างน้อย 1 ชิ้น + รอชิ้นอื่นที่คนอื่นถือ	ขอให้**ครบทุกชิ้นก่อนเริ่มรัน** หรือขอเมื่อไม่ถืออะไรเลย → utilization ต่ำ + starvation ได้
3. No Preemption (ยึดคืนกลางคันไม่ได้)	ยึดคืนกลางคันไม่ได้ ปล่อยเมื่อเจ้าของใจดีเท่านั้น	ขอเพิ่มไม่ได้ = ถูกยึดของที่ถือทั้งหมด ไปให้คนรอ แล้วค่อยเริ่มใหม่ → ยุ่งยาก
4. Circular Wait (รอกันเป็นวงกลม)	P0 รอ P1, P1 รอ P2, ... วนกลับมา P0	ให้ทุกทรัพยากรมี**เลขประจำตัว** ขอเรียง**จากน้อยไปมากเท่านั้น** เช่น Disk=4, Printer=11 → ต้องได้ 4 ก่อน 11 ห้ามถือ 11 แล้วขอ 4

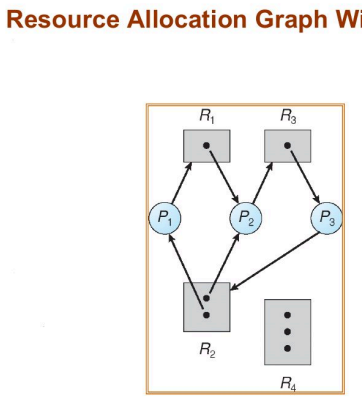
> 🎯 **จำแน่น**
> ต้องครบ **4 ข้อพร้อมกัน** → ตัดข้อเดียวก็ไม่เกิด · Prevention ทั้งหมด "ทฤษฎีดี แต่ปฏิบัติเสียประสิทธิภาพ" อาจารย์สรุปว่าไม่คุ้มใช้

F3. Resource-Allocation Graph (RAG)

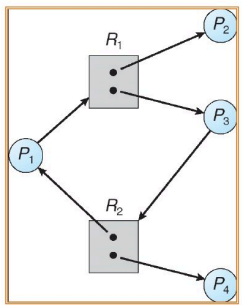
- **วงกลม** = Process P_i · **สี่เหลี่ยม** = Resource (ทรัพยากร) type R_j (จุดในสี่เหลี่ยม = จำนวน instance)
- **Request edge** $P_i \rightarrow R_j$: "ขอ" · **Assignment edge** $R_j \rightarrow P_i$: "ได้ถืออยู่"



ไม่มี cycle (วงจร (วนกลับมาที่เดิม)) → ไม่เกิด deadlock แน่ (100%)
มี cycle → single instance/type = deadlock แน่ · multiple instances = "อาจเกิด" เท่านั้น
slide p.11: มี cycle แต่ R2 มีหลายชิ้น ชิ้นที่เหลือ (ถือโดย P4 นอกวง) มาช่วยได้ → ไม่ deadlock



สไลด์ p.10 RAG มี deadlock



สไลด์ p.11 มี cycle แต่ไม่ deadlock (multiple instance)

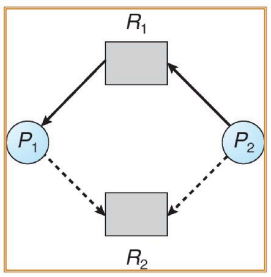
F4. กลยุทธ์จัดการ Deadlock 4 แนวทาง

1. **Prevention** — ออกแบบให้เงื่อนไขข้อใดข้อหนึ่งเกิดไม่ได้เลย (ตาราง F2)

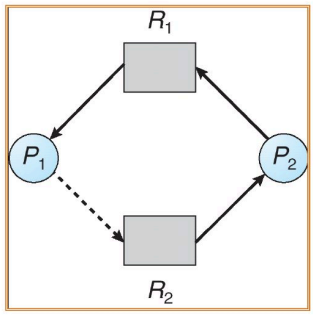
2. **Avoidance** — process แจ้ง **Max demand ล่วงหน้า** OS เช็คทุกครั้งก่อนจัดสรรว่ายัง safe → Banker (อัลกอริทึมนายธนาคาร)'s / claim edge
3. **Detection + Recovery** — ปล่อยเกิด สแกนหา แล้วแก้
4. **เมิน (Ostrich)** — ที่ OS จริงใช้กัน

F4.1 Avoidance แบบ single instance — Claim Edge (เส้นประ)

- **Claim edge** $P_i \dashrightarrow R_j$ (เส้นประ) = "อาจขอในอนาคต" · ขอจริง → กลายเป็น request edge · ได้รับ → assignment edge · ปล่อย → กลับเป็น claim edge
- **ให้ทรัพยากรได้เมื่อ:** แปลง request เป็น assignment แล้ว**เกิด cycle ไม่ได้** (มี cycle = unsafe → ไม่ให้ ให้รอ)
- ข้อเสีย: สแกนหา cycle ทุกครั้งที่ขอ = $O(n^2)$ แพง



สไลด์ p.20 Claim edges (เส้นประ)



สไลด์ p.21 Unsafe state: ถ้าให้ P2 ใช้ R2 จะเกิดวง → ไม่ให้

F5. Banker's Algorithm (multiple instances) 🔥🔥

ข้อมูลที่ OS ถือ (n = process, m = resource type): `Available[m]` · `Max[n×m]` · `Allocation[n×m]` · **Need = Max − Allocation**

$$Need[i,j] = Max[i,j] - Allocation[i,j]$$

หัวใจของทุกโจทย์ Banker's — คิด Need ให้ครบทุกช่องก่อนเสมอ

F5.1 Safety Algorithm (ตรวจว่า state ปลอดภัยไหม)

1. Work = Available · Finish[i] = false ทุกตัว
2. หา i ที่ **Finish[i] = false และ $Need_i \le Work$** (เทียบทีละช่องทุกชนิด) — ไม่มีใครผ่าน → ไปขั้น 4
3. สมมติ i รันจนจบแล้วคืนของ: **Work = Work + $Allocation_i$**, Finish[i] = true → กลับขั้น 2
4. ถ้า Finish = true **ทุกตัว** → **SAFE** (มี safe sequence) · ติดตรงไหน = UNSAFE

F5.2 โจทย์เต็มจากสไลด์ p.26–27 (5 process, A=10 B=5 C=7, Available = 3,3,2)

Process	Allocation			Max			Need = Max − Alloc		
	A	B	C	A	B	C	A	B	C
P0	0	1	0	7	5	3	7	4	3
P1	2	0	0	3	2	2	1	2	2
P2	3	0	2	9	0	2	6	0	0
P3	2	1	1	2	2	2	0	1	1
P4	0	0	2	4	3	3	4	3	1

ไล่ Safety (Work เริ่ม = 3,3,2):

ขั้น	เลือกใคร (เหตุผล)	Work หลังคืนของ
1	P1 — Need (1,2,2) ≤ (3,3,2) ✓	(3,3,2)+(2,0,0) = **(5,3,2)**
2	P3 — Need (0,1,1) ≤ (5,3,2) ✓	(5,3,2)+(2,1,1) = **(7,4,3)**
3	P4 — Need (4,3,1) ≤ (7,4,3) ✓	(7,4,3)+(0,0,2) = **(7,4,5)**
4	P2 — Need (6,0,0) ≤ (7,4,5) ✓	(7,4,5)+(3,0,2) = **(10,4,7)**
5	P0 — Need (7,4,3) ≤ (10,4,7) ✓	(10,4,7)+(0,1,0) = (10,5,7) ครบทุกตัว ✅

`Safe sequence = <P1, P3, P4, P2, P0> → SAFE STATE`

มีได้หลายคำตอบ — ตอบลำดับไหนที่ไล่ได้จริงก็ถูก (สไลด์ให้ <P1,P3,P4,P2,P0>)

F5.3 Resource-Request Algorithm — P1 ขอ (1,0,2) ให้ได้ไหม?

1. เช็ค Request ≤ Need: (1,0,2) ≤ (1,2,2) ✓ (ไม่ขอเกินที่ประกาศ)
2. เช็ค Request ≤ Available: (1,0,2) ≤ (3,3,2) ✓ (ของพอ)
3. **สมมติจัดให้ (pretend):** Available = (2,3,0) · P1 Alloc = (3,0,2) · P1 Need = (0,2,0)
4. รัน Safety ใหม่ → ได้ sequence <P1, P3, P4, P0, P2> → ยัง SAFE ✅ → **ให้ไปได้**

คำขออื่นในสไลด์ p.28	ผล	เหตุผล
P4 ขอ (3,3,0)	**ไม่ให้**	Request > Available (3,3,0) > (2,3,0) — ของไม่พอ P4 ต้อง wait
P0 ขอ (0,2,0)	**ไม่ให้**	ผ่าน 2 เช็คแรก แต่ไล่ Safety แล้ว **UNSAFE** (มี deadlock เป็นไปได้) → ให้รอ คืนสถานะเดิม

> **🟢 วิธีทำในห้องสอบ (เร็ว)**
>
> 1) เติมตาราง Need = Max − Alloc ทุกช่อง 2) เขียน Work = Available กก.ลงข้าง ๆ 3) วนหา "Need ≤ Work" ใครผ่านให้ไปก่อน แล้ว **บวก Allocation แถวนั้นเข้า Work** 4) จนครบทุกตัว = safe · โจทย์ถาม "ขอได้ไหม" → เช็ค 2 ข้อแล้ว pretend แล้วไล่ safety ใหม่เสมอ

> **📌 ปัญหาของ Banker's (เคยออก)**
>
> ต้องประกาศ Max ล่วงหน้า / จำนวนทรัพยากรคงที่ / จำนวนงานคงที่ / overhead สูง / ใช้ทรัพยากรไม่คุ้มเพราะคิดแย่สุดเสมอ

F6. Detection (ตรวจจับ) & Recovery (กู้)

- **Single instance/type:** ยุบ RAG เป็น **Wait-For Graph (กราฟแสดงว่าใครรอใคร)** — ตัดโหนดทรัพยากรทิ้ง เหลือ $P_i \rightarrow P_j$ ("P_i รอ P_j") มี cycle = deadlock (สแกนทุกระยะ, แพง $O(n^2)$)

- **Multiple instances:** Detection Algorithm ทำงานเหมือน Safety แต่เทียบด้วย **Request** (ของที่ร้องขอค้างจริง) แทน Need · Finish เริ่มต้น: คนที่ Allocation ไม่เป็นศูนย์ = false

ตัวอย่าง Detection จากสไลด์ p.36–37 (A=7, B=2, C=6, Available = 0,0,0):

Process	Allocation			Request		
	A	B	C	A	B	C
P0	0	1	0	0	0	0
P1	2	0	0	2	0	2
P2	3	0	3	0	0	0
P3	2	1	1	1	0	0
P4	0	0	2	0	0	2

ไล่: Work=(0,0,0) → P0 (Request 000 ≤ 000) จบ → Work=(0,1,0) → P2 (000 ≤ 010) จบ → Work=(3,1,3) → P3 จบ → (5,2,4) → P1 จบ → (7,2,4) → P4 จบ → **sequence <P0,P2,P3,P1,P4> ครบ → ไม่มี deadlock**

แต่ถ้า P2 ขอเพิ่ม C หนึ่งชิ้น (Request P2 = 0,0,1): Work=(0,0,0) → P0 จบ → (0,1,0) → ใครผ่าน? P1(2,0,2)✗ P2(0,0,1)✗ P3(1,0,0)✗ P4(0,0,2)✗ → ตัน → **deadlock: P1, P2, P3, P4**

Recovery 2 แนวทาง

- **Process Termination:** จบทุกตัวที่ติด (เสียงานหมด) หรือจบทีละตัวจนวงแตก — เลือกจาก: priority, รันมานานแค่ไหน, ใช้ทรัพยากรอะไรไปแล้ว, ต้องการอีกเท่าไร, ต้องจบกี่ตัว, interactive/batch?
- **Resource Preemption (การยึดทรัพยากรกลางคัน):** เลือกเหยื่อ (victim) → **Rollback** ไป safe state → เริ่มใหม่ · ระวัง **Starvation** — ใส่ "จำนวน rollback ที่โดนมาแล้ว" ในตัวแปรต้นทุน

G. Real Memory Management (OSNo08 ต้น)

> 📖 **รู้จักส่วนนี้ก่อนอ่าน**
>
> **ความหมาย:** การจัดสรร RAM ให้ process ตอนโหลด (ยุคก่อนมี virtual memory) — จัดแบบช่องคงที่, ช่องแปรผัน, หรือแบ่งตามส่วน (Segmentation (การแบ่งหน่วยความจำตามส่วน)) พร้อมกลยุทธ์เลือกหลุม First/Best/Worst-fit (หยิบหลุมใหญ่ที่สุด)
>
> **ความสำคัญกับข้อสอบ:** โจทย์ first/best/worst-fit คำนวณเร็วได้คะแนนง่าย + Segmentation (เช็ค offset (ตำแหน่งเลื่อนภายในเพจ) < limit ก่อน แล้วค่อย base + offset) — และการแยก internal/external (ภายนอก (กระจายเป็นหลุม)) fragmentation (การแตกเป็นเศษพื้นที่) ให้ถูกตัว
>
> **ใช้จริงในคอมพิวเตอร์:** เป็นต้นแบบ memory allocator (malloc/free) ในทุกภาษา · ปัญหา fragmentation ยังพบจริงในระบบฝังตัว (embedded) และการจัดการหน่วยความจำ GPU

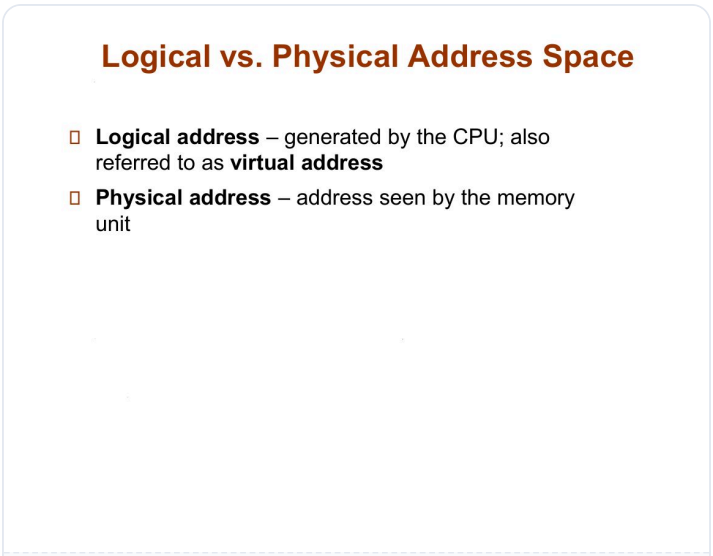
G1. Logical (เชิงตรรกะ (มองจากโปรแกรม)) vs Physical Address (ที่อยู่จริงใน RAM) + MMU

	Logical (Virtual) Address	Physical Address
สร้างโดย	CPU ปล่อยขณะรัน	ตำแหน่งจริงที่ memory unit มองเห็น
จุดอ้างอิง	โปรแกรมเริ่มที่ byte 0 เสมอ	เทียบจุดเริ่ม RAM จริง

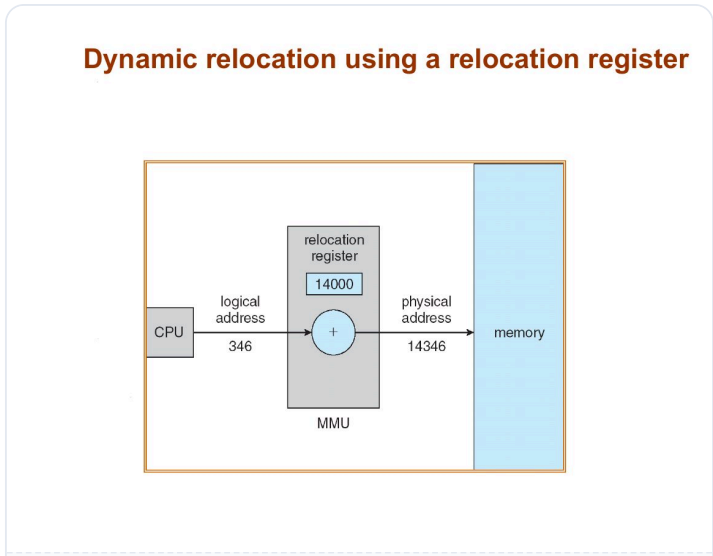
MMU = ฮาร์ดแวร์แปลง logical (เชิงตรรกะ (มองจากโปรแกรม)) → physical (เชิงกายภาพ (ตำแหน่งจริง)) แบบง่ายสุดคือ**บวก Relocation (Base) Register**

`Physical Address = Relocation Register (รีจิสเตอร์เก็บตำแหน่งฐาน) + Logical Address (ที่อยู่เชิงตรรกะ (มองจากโปรแกรม))`

ตัวอย่างวิดีโอ: โปรแกรมโหลดที่ 14,000 · CPU ขอ byte 346 → PA = 14,000 + 346 = 14,346



สไลด์ p.3 Logical vs Physical (เชิงกายภาพ (ตำแหน่งจริง))



สไลด์ p.5 Dynamic relocation ด้วย relocation register

G2. Static vs Dynamic Linking (มีตัวเลขให้คำนวณ)

- **Static linking**: ก๊อปโค้ด library รวมใน executable ตั้งแต่ compile — lib ซ้ำทุกโปรแกรม
- **Dynamic linking**: เลื่อนไป link (ลิงก์ (ทางเชื่อม)) ตอน run — ใส่แค่ **stub** (โค้ดจิ๋วหา library / แทนตัวเองด้วย address routine) → library โหลดครั้งเดียว แชร์กันทั้งระบบ (shared libraries)

```
ตัวอย่างวิดีโอ: A(10MB) + B(20MB) + Lib(20MB)
Static: A=30MB, B=40MB → RAM 30+40 = 70MB (lib ซ้ำ 2 ชุด)
Dynamic: A=10+2, B=20+2, lib 1 ชุด → 12+20+22 = 54MB ประหยัด 16MB
```

G3. Fixed Partition — Internal Fragmentation (เศษภายในช่องที่จองแล้ว)

- แบ่ง RAM ส่วน user เป็นช่องคงที่ล่วงหน้า → **มีกี่ช่องรับ process พร้อมกันได้เท่านั้น** · ป้องกันด้วย limit register (ขนาด) + relocation register (ตำแหน่ง): เช็ค LA < Limit ผ่านแล้วค่อย + Base
- **ตัวอย่างวิดีโอ:** Partitions P1=100K, P2=25K, P3=25K, P4=50K · Job1(30K)→P1 (เหลือ 70K ใช้ไม่ได้ = **internal frag**) · Job2(50K)→P4 พอดี · **Job3(30K) ต้องรอ!** เพราะ P2,P3 มีแค่ 25K แม้ว่างรวม 120K · Job4(25K)→P2 พอดี

🎯 คู่แฝด fragmentation

Fixed = Internal (เศษในช่องจองแล้ว ใครใช้ต่อไม่ได้) · **Variable + Segmentation = External** (เศษกระจายนอก รวมว่างเยอะแต่ช่องเดียวไม่พอ) · **Paging = ไม่มี External แต่มี Internal** ที่เพจสุดท้าย — ออกถูก/ผิดวนไป

G4. Variable Partition — External Fragmentation (เศษกระจายนอก (หลุมเล็ก ๆ)) + วิธีรักษา

จัดสรรตามขนาดจริง ตอนแรกไม่มีของเสีย แต่งานจบทิ้งช่องว่างกระจาย · **ตัวอย่าง:** หลุมว่าง 5+2+1 = 8MB แต่ขอ 7MB โหลดไม่ได้ (ช่องเดียวใหญ่สุด 5MB)

วิธีรักษา	ทำอะไร	ข้อจำกัด
Coalescing (การรวมหลุมว่างที่ติดกัน)	รวมหลุมที่**ติดกัน**เป็นหลุมใหญ่ 1 หลุม	หลุมไม่ติดกันก็ช่วยไม่ได้ — มักไม่พอ
Compaction (การอัดข้อมูลให้ติดกัน)	ลาก process มา**อัดติดกัน** รวมว่างปลายเดียว	overhead แพง และทำได้**เฉพาะ relocation แบบ dynamic (ตอน execution time)**

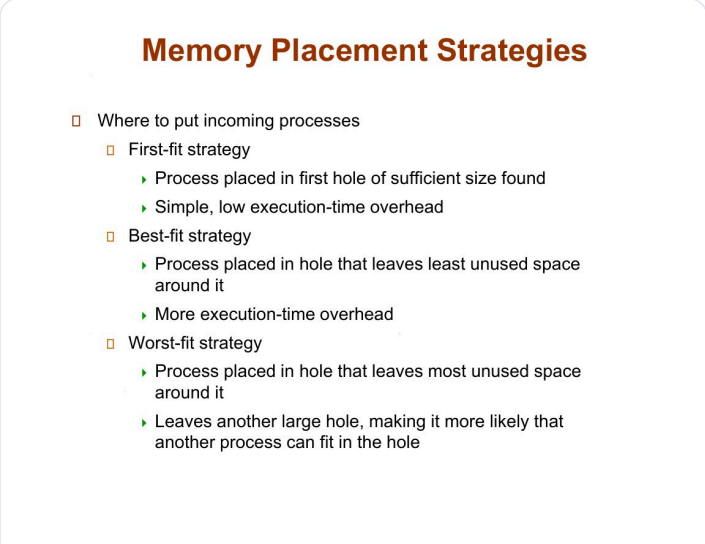
G5. First-fit (หยิบหลุมแรกที่พอ) · Best-fit (หยิบหลุมที่แน่นที่สุด) · Worst-fit 🔥 (ตัวอย่างจากวิดีโอ)

โจทย์ (สไลด์ p.21–24): หลุมเรียงตามลำดับใน RAM: **16, 14, 5, 30 MB** · process ใหม่ต้องการ **13 MB**

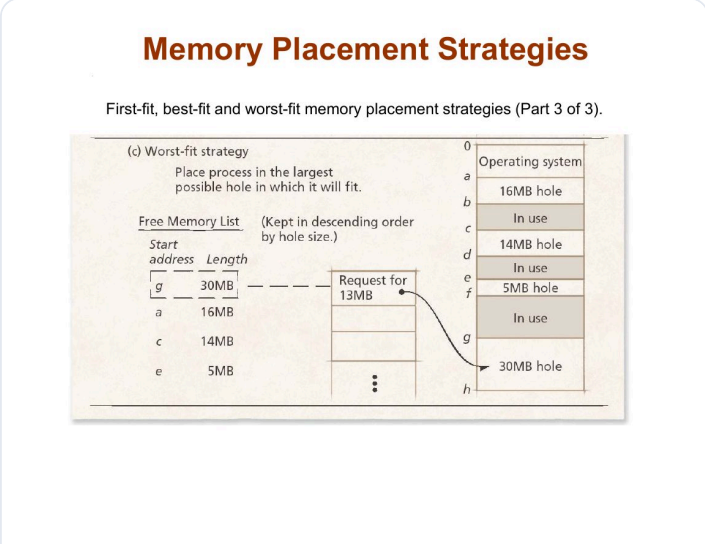
กลยุทธ์	หลักคิด	ลงหลุม	เศษเหลือ	จุดเด่น/ด้อย
First-fit	ไล่จากซ้าย เจอหลุมแรกที่ ≥ ขนาด เอาทันที	**16**	3	เร็วสุด (ไม่สแกนครบ) แต่เปลืองพื้นที่
Best-fit	สแกนครบ เลือกหลุมที่ "ใหญ่พอ + เหลือน้อยสุด"	**14**	1	คุ้มสุด แต่ช้า + ทิ้งเศษจิ๋วใช้ต่อไม่ได้
Worst-fit	สแกนครบ เลือกหลุมใหญ่สุด	**30**	17	ทิ้งชิ้นใหญ่ให้งานต่อไป (โดยรวมมักแพ้ best-fit)

🟢 เทคนิค 30 วินาที

เขียนเลขหลุมตามลำดับที่โจทย์ให้ (ห้ามจัดใหม่!) · First = หยุดหลุมแรกที่พอ · Best = เทียบ (หลุม−ขนาด) เลือกค่าน้อยสุด · Worst = หลุมใหญ่สุด · ถ้าโจทย์ให้จัดหลายงานต่อกัน อย่าลืมหักขนาดหลุมที่ใช้ไปแล้วด้วย



สไลด์ p.21 Memory Placement Strategies



สไลด์ p.24 ผลการจัดทั้ง 3 กลยุทธ์ (ตรงกับตารางข้างบน)

G6. Segmentation 🔥 — แบ่งตามสายตาโปรแกรมเมอร์

- แบ่งเป็น logical unit ขนาดไม่เท่ากัน: main, function, object, stack, global variables · Logical address = คู่ `<segment s, offset d>`
- Segment table แต่ละแถวมี **Base** (จุดเริ่มใน RAM) + **Limit** (ความยาว) · STBR ชี้ตาราง, STLR = จำนวน segment (s ใช้ได้ถ้า s < STLR)

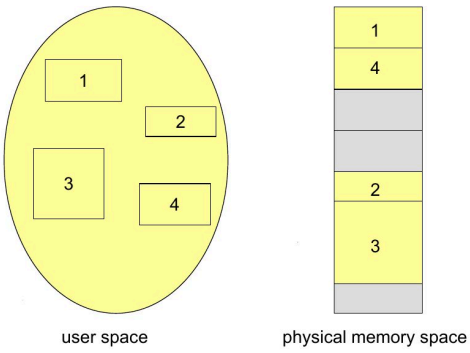
```
ขั้นแรก: เช็ค d < Limit — ถ้า d ≥ Limit → TRAP (error!) ทันที
ขั้นต่อมา: Physical Address = Base + Offset (ตำแหน่งเลื่อนภายในเพจ)
```

ตัวอย่างจากสไลด์ p.50: S1 (base 6300, limit 500) · S2 (base 4300, limit 400)

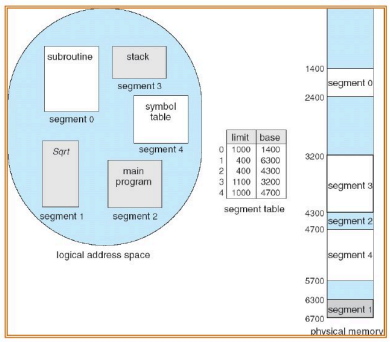
Logical	เช็ค	ผล
<1, 300>	300 < 500 ✓ → PA = 6300 + 300	**6600** ✓
<2, 400>	400 < 400 ✗ (400 ไม่ได้ "น้อยกว่า" 400)	**Trap / Error** ❌

🎯 จุดหลอก

เช็ค **ก่อนเสมอ แล้วค่อยบวก** — เคร่งครัดเรื่อง < ไม่ใช่ ≤ · Segmentation ยังเจ็บกับ **external fragmentation** (แต่ละ segment ต้องวางติดกันเป็นผืน) · สูตรอย่าสลับกับ paging (การแบ่งหน่วยความจำเป็นหน้า): Segment/Base = **Base + offset** / Paging = **frame (เฟรม (ก้อน RAM)) × size (ขนาด) + offset**



สไลด์ p.47 Logical view (user space) vs physical memory



สไลด์ p.50 Example of Segmentation (ตัวเลขตรงกับตาราง)

H. Paging & Address Translation (OSNo08 ท้าย) 🔥🔥

> **📖 รู้จักส่วนนี้ก่อนอ่าน — รีวิวรุ่นพี่: "คนส่วนมากคำนวณ Paging กันไม่ทัน"**
>
> **ความหมาย:** Paging (การแบ่งหน่วยความจำเป็นหน้าเท่า ๆ กัน) = หั่น RAM เป็น frame หั่นโปรแกรมเป็น page (ขนาดเท่ากัน) แล้วใช้ Page Table (ตารางจับคู่เพจ ↔ เฟรม) จับคู่ — "คำนวณ Paging" คือการแปลงที่อยู่ที่โปรแกรมเห็น (Logical) ให้เป็นที่อยู่จริงใน RAM (Physical)
>
> **ความสำคัญกับข้อสอบ:** แปลง address ทีละบิต, TLB (แคชตารางเพจความเร็วสูง) + สูตร EAT (เวลาเข้าถึงเฉลี่ย), Two-level paging — เป็นแนวคำนวณที่ตัดสินว่า "ทันสอบหรือไม่ทัน" ต้องฝึกจนข้อละไม่เกิน 1 นาที
>
> **ใช้จริงในคอมพิวเตอร์:** CPU ทุกตัวสมัยใหม่ (x86, ARM ในมือถือ) + OS ทุกตัว (Windows/Linux/macOS/Android) ใช้ paging จริง 100% — แอปแต่ละตัวเห็น "RAM ของตัวเอง" ก็เพราะกลไกนี้

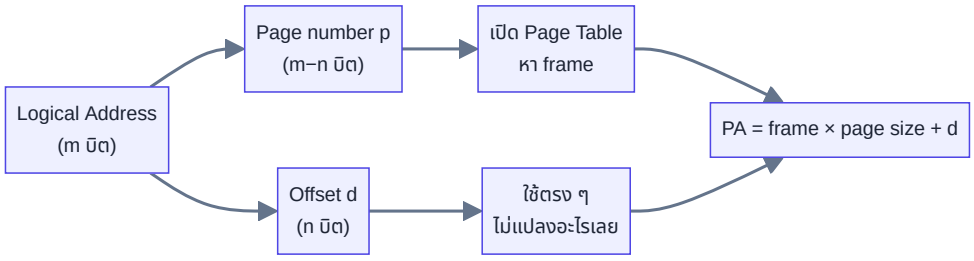
H1. แนวคิด Paging

- **Paging = จัดสรรแบบ non-contiguous** (ไม่ต้องติดกัน) — ตัด external fragmentation ขาด
- **Frame (เฟรม (ก้อน RAM))**: แบ่ง RAM เป็นบล็อกขนาดคงที่ ขนาดเป็นกำลังสอง 2^n bytes (512–8,192)
- **Page (หน้า)**: แบ่งโปรแกรมเป็นบล็อก**ขนาดเท่า frame เป๊ะ** · รันโปรแกรม n เพจ = หา frame ว่าง n อันกระจายวาง
- **Page table**: จับคู่ "เพจ i → frame ไหน" ของแต่ละ process

> **🎯 เช็คความเข้าใจ**
>
> Paging **ไม่มี External frag** แต่มี **Internal frag** — เกิดเฉพาะ**เพจสุดท้าย**ถ้าขนาดโปรแกรมไม่ลงตัว

H2. โครงสร้าง Logical Address — บิต p กับ d 🔥

Address ที่ CPU ปล่อย (ยาว **m บิต**) ซอยเป็น 2 ส่วนโดยฮาร์ดแวร์:



Page size = 2^n bytes → บิต offset = n บิต · บิต page number (หมายเลขเพจ) = m − n บิต → เพจได้สูงสุด 2^(m−n) เพจ

ตัวอย่างวิดีโอ: m = 5 บิต (2^5 = 32 bytes), page = 4 bytes (2^2) → offset 2 บิต, page# 3 บิต → 8 เพจ (0–7 นับจากศูนย์!)

🪑 **อุปมาอาจารย์**

เก้าอี้ 32 ตัว จัด 8 ห้อง ห้องละ 4 — อ้างอิงว่า "เก้าอี้ตัวที่ 3 ในห้องเบอร์ 1" = (Page 1, Offset 3) สองเลขนี้คือทั้งชีวิตของ paging

H3. แปลง Logical → Physical ทีละขั้น 🔥

Physical Address = (Frame Number × Page Size (ขนาดของเพจ)) + Offset

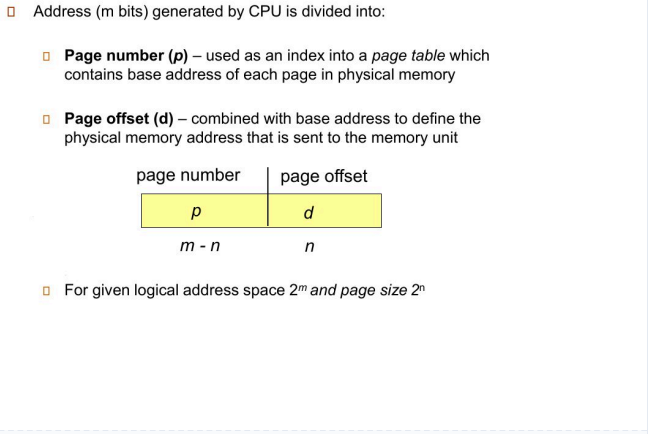
เปิดตารางด้วย p → ได้ frame → คูณขนาด frame = จุดเริ่ม → บวก offset

โจทย์จากสไลด์ p.32 (Paging Example): page size (ขนาดของเพจ) = 4 bytes · Page table: 0→5, 1→4, 2→1, 3→7

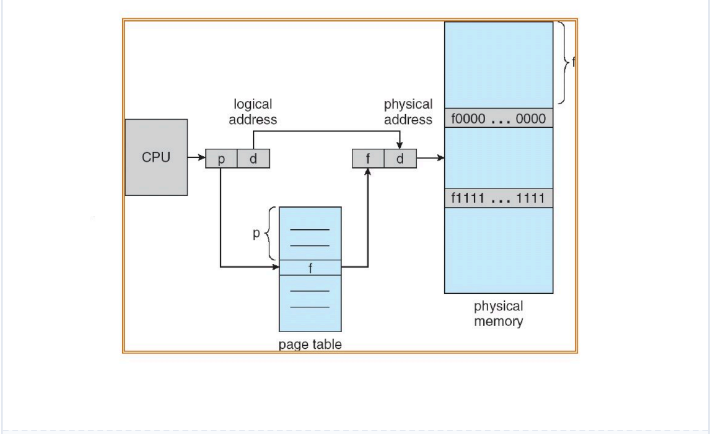
หาอะไร	Logical	ไล่ทีละขั้น	PA
ตัวอักษร 'C'	(p=0, d=2)	เพจ 0 → frame 5 → (5 × 4) + 2 = 20 + 2	**22**
ตัวอักษร 'K'	(p=2, d=2)	เพจ 2 → frame 1 → (1 × 4) + 2 = 4 + 2	**6**

🎯 **3 จุดพลาดคลาสสิก**

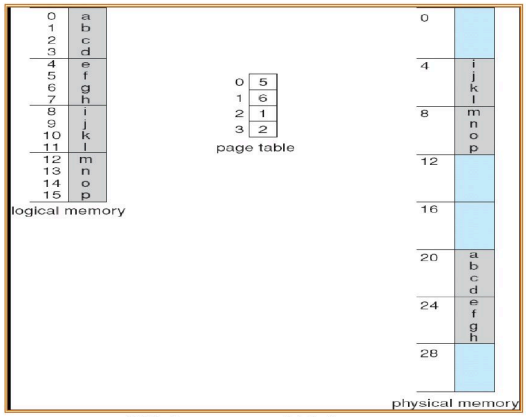
1) **บวก frame+offset ตรง ๆ** (5+2=7 ❌) ต้องคูณขนาดก่อน 2) นับเพจเริ่ม 1 → ต้องเริ่ม **Page 0** 3) ปัดจำนวนเพจลง — 42 bytes / เพจ 4 = 10.5 → **ปัดขึ้น 11 เพจ** (10 เพจเก็บได้ 40 bytes ข้อมูลขาด!) เพจสุดท้ายใช้ 2 bytes → internal frag = 4−2 = **2 bytes**



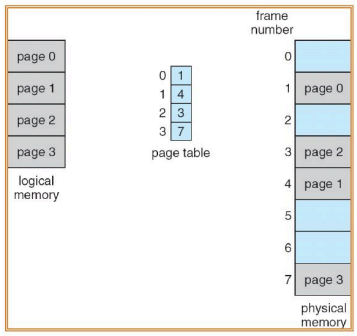
สไลด์ p.29 Address Translation Scheme (p | d)



สไลด์ p.30 Paging Hardware: p ไปตารางได้ frame + d ต่อกันเป็น PA



สไลด์ p.32 Paging Example — หน่วยความจำ 32 bytes, เพจ 4 bytes (โจทย์ 'C' และ 'K' อยู่นี่)



สไลด์ p.31 Paging Model: เพจกระจาย frame ไม่ติดกันก็ได้

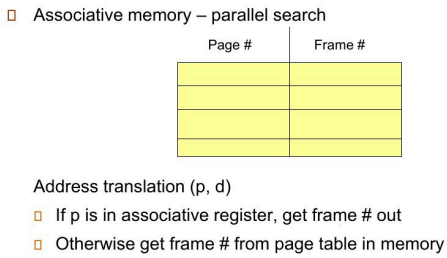
H4. TLB + Effective Access Time 🔥

- Page table อยู่ใน RAM → อ่านข้อมูล 1 ครั้ง = ไป RAM **2 ครั้ง** (1: อ่านตารางหา frame, 2: อ่านข้อมูลจริง)
- **TLB** = แคชฮาร์ดแวร์ความเร็วสูง เก็บคู่ (page # → frame #) ล่าสุด ค้นแบบขนาน (associative memory)

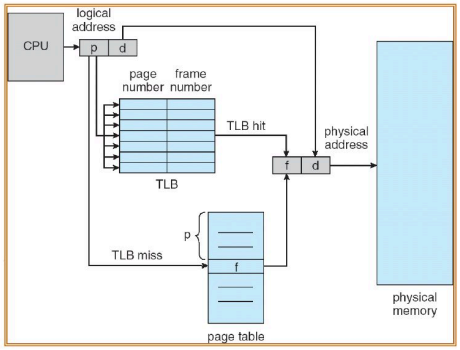
กรณี	ไป RAM	เวลา (memory = 1 μs, TLB search = ε)
TLB Hit (โอกาส α)	1 ครั้ง	ε + 1 = 1 + ε
TLB Miss (โอกาส 1−α)	2 ครั้ง	ε + 1 + 1 = 2 + ε

$$EAT = \alpha(1 + \varepsilon) + (1 - \alpha)(2 + \varepsilon) = 2 + \varepsilon - \alpha$$

โจทย์วิดีโอ: ε = 0.1 μs, α = 80% → (0.8×1.1) + (0.2×2.1) = 0.88 + 0.42 = 1.3 μs (สูตรลัด: 2 + 0.1 − 0.8 = 1.3 ✓)



สไลด์ p.35 Associative Memory (TLB): ค้นขนาน



สไลด์ p.36 Paging Hardware With TLB

H5. Protection — Valid / Invalid bit

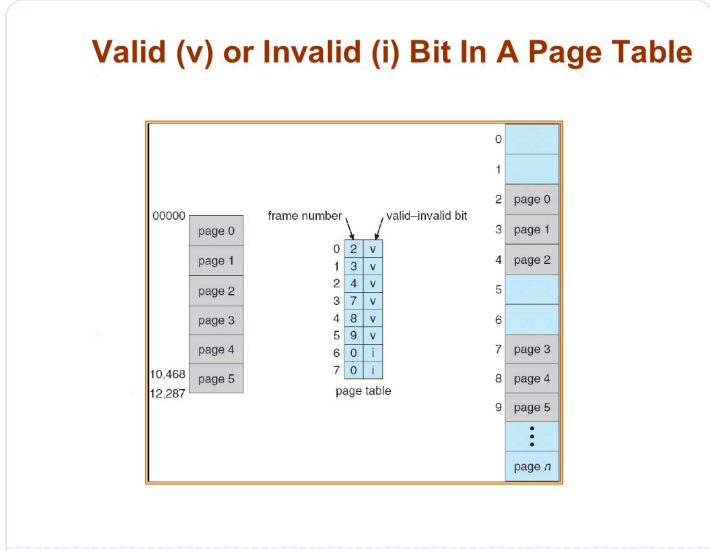
- แต่ละแถว page table (ตารางจับคู่เพจ ↔ เฟรม) มี **valid–invalid bit**: **v** = เพจนี้อยู่ใน address space ของ process (ใช้ได้) · **i** = ห้ามอ้างอิง
- **PTBR** ชี้ตำแหน่ง page table ใน RAM · **PTLR** = ความยาวตาราง (เช็คเบื้องต้นว่าไม่อ้างเพจเกินขอบเขต)

🎯 จุดหลอก: valid ดู "ทั้งเพจ" ไม่ได้ดู "ทีละไบต์"

เพจสุดท้าย valid = v → ระบบยอมให้อ้าง offset *ใด ๆ* ในเพจนั้น (แม้เกินขนาดโปรแกรมจริง) — นี่คือเหตุผลที่ internal fragmentation ยังอยู่รอดใน paging



สไลด์ p.38 Memory Protection

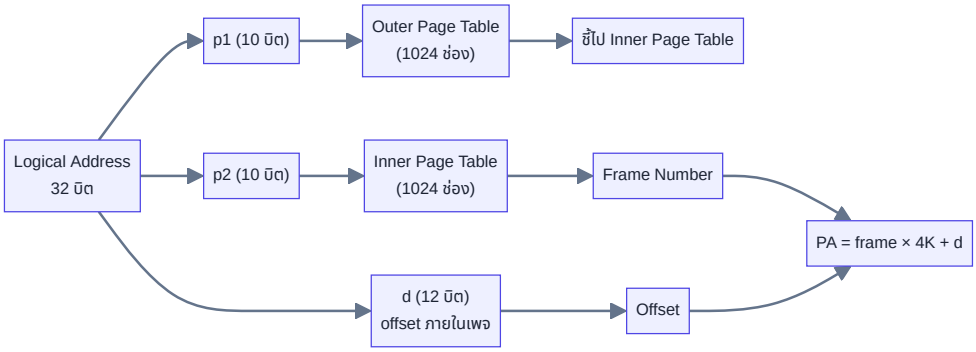


สไลด์ p.39 Valid (v) / Invalid (i) Bit

H6. Two-Level Paging — ทำไม + คำนวณ 🔥

ปัญหา single-level ในระบบ 32-bit: page 4KB (offset 12 บิต) → page# 20 บิต → 2^{20} = 1,048,576 ช่อง × 4B = **4MB ต่อ process ทุกตัว!** แม้โปรแกรมเล็กแค่ 1MB (ตารางต้องอยู่ต่อเนื่องด้วย)

ทางแก้: ซอยบิต 32 ตัวเป็น **p_1 = 10 | p_2 = 10 | d = 12** — p_1 เปิด outer table หา inner table, p_2 เปิด inner table หา frame



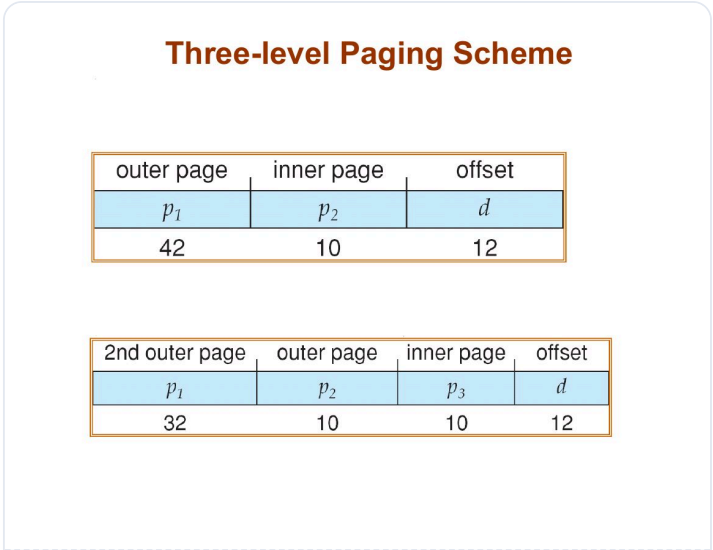
โจทย์ (จากวิดีโอ)	จำนวนเพจ	Inner tables ที่ต้องสร้าง	พื้นที่ page table รวม
โปรแกรม 1 MB	1024KB ÷ 4KB = **256 เพจ**	256 ≤ 1024 → **1 ตาราง**	outer 4KB + inner 4KB = **8 KB** (แทน 4MB!)
โปรแกรม 8 MB	8192KB ÷ 4KB = **2048 เพจ**	2048 ÷ 1024 = **2 ตาราง**	4KB + 2×4KB = **12 KB**

🟢 สูตรรวมทั้งบท

บิต offset = $\log_2$(page size) · บิต page# = บิต address − บิต offset · เพจ = ceil(ขนาดโปรแกรม ÷ page size) · PA = frame × pageSize + offset · EAT = 2 + ε − α · inner tables = ceil(จำนวนเพจ ÷ 1024)



สไลด์ p.42 Hierarchical Page Tables



สไลด์ p.44 Three-level Paging Scheme (x86 มีจริง)

Hashed Page Table — สำหรับ address space ใหญ่มาก

เมื่อ address space กว้างมาก (64-bit) ตารางเพจแบบลำดับชั้นจะลึกเกินไป จึงใช้ **ฟังก์ชัน hash** ของ page number ไปชี้ entry ในตาราง แล้วเดินตาม**ห่วงโซ่ (chain)** ของ element ที่ hash ชนกัน จนเจอแถวที่ page number ตรง แล้วอ่าน frame number — ดีกับเพจที่กระจายตัว ไม่ต้องสร้างตารางครบทุกช่อง

Inverted Page Table — 1 แถวต่อ 1 frame จริง

ปกติตารางเพจมี 1 แถวต่อ 1 *เพจเสมือน* ของ process (สร้างหลายตาราง) แต่ Inverted กลับด้าน: **1 แถวต่อ 1 frame จริงใน RAM** เก็บ (pid, page) ที่ครอบอยู่ — มีตารางเดียวสำหรับทั้งระบบ ประหยัด RAM มาก · ราคาที่จ่าย: ตอนแปลง address ต้อง**ค้นหา** (pid, page) ของตัวเองทั้งตาราง → ช้า จึงต้องมี hash table + TLB ช่วย

หมายเหตุ: ทั้งสองแบบเป็นแนวทางแก้ปัญหา "page table ใหญ่เกิน" ร่วมกับ Two-level ที่เจอข้างบน — ข้อสอบมักถามรู้จักชื่อ + หลักคิดสั้น ๆ แบบนี้พอ

I. Virtual Memory & Page Replacement (OSNo09) 🔥🔥

📖 รู้จักส่วนนี้ก่อนอ่าน

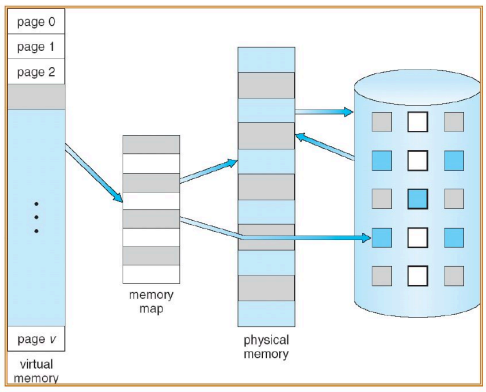
ความหมาย: Virtual Memory (หน่วยความจำเสมือน) = โปรแกรมไม่ต้องอยู่ใน RAM ครบทั้งตัว โหลดเฉพาะเพจที่กำลังใช้ (demand paging) ทำให้ RAM จริงเหมือนใหญ่กว่าจริง · Page Replacement = เมื่อ RAM เต็ม ต้องเลือกเพจ "ตัวปลอด" ออก

ความสำคัญกับข้อสอบ: นับ page fault (เพจยังไม่อยู่ใน RAM) ทั้ง FIFO (เข้าก่อน-ออกก่อน) / Optimal (ดีที่สุดเชิงทฤษฎี) / LRU (รู้ผล 15/9/12 ของ string หลัก), Belady's Anomaly (ปรากฏการณ์เพิ่ม RAM แล้ว fault เพิ่ม), Thrashing (สลับเพจหนักจนระบบแย่) — เป็นแนวคำนวณหลักอีกแนวของ Final

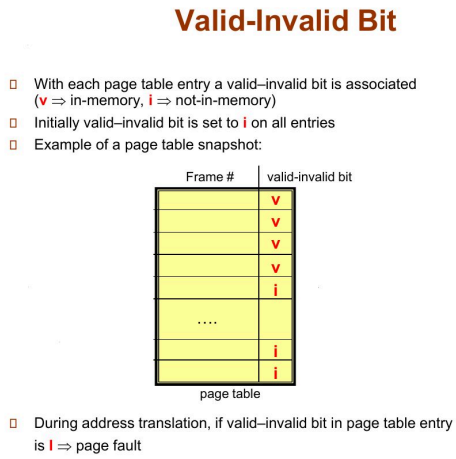
ใช้จริงในคอมพิวเตอร์: เปิดเกมใหญ่กว่า RAM เครื่องได้, สลับแอปเร็วโดยไม่โหลดใหม่, ขนาด "หน่วยความจำเสมือน" ที่เห็นใน Task Manager คือกลไกนี้ — ข้อมูลที่ไม่ได้ใช้ถูกส่งไปพักบน disk

I1. Demand Paging (โหลดเพจเฉพาะเมื่อถูกใช้) — ขอเมื่อไรค่อยเอามา

- **Virtual memory** = แยก logical memory ออกจาก physical → address space ใหญ่กว่า RAM จริงก็ได้ · implement ผ่าน **demand paging** หรือ demand segmentation
- **Demand paging**: โหลดเพจเฉพาะเมื่อถูกเรียกใช้ — หลักขี้เกียจของ **lazy swapper** (ตัวทำงานกับเพจเรียก **pager**) · ผลดี: I/O น้อย, RAM น้อย, ตอบเร็ว, รองรับ user มาก
- เพจที่ยังไม่โหลด → bit = **i (invalid)** → แปลง address เจอ i → **Page Fault (เพจยังไม่อยู่ใน RAM)** — **invalid ที่นี่ส่วนใหญ่ไม่ใช่ error** แค่ "ยังไม่มา"

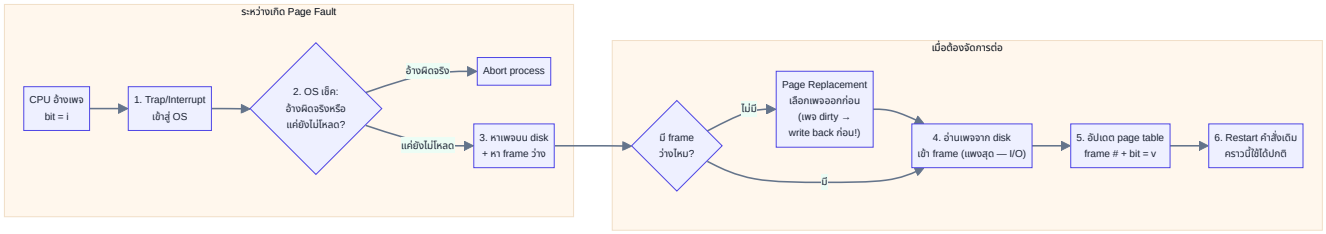


สไลด์ p.4 Virtual Memory ใหญ่กว่า Physical



สไลด์ p.6 Valid–Invalid Bit ใน page table

I2. Page Fault — ขั้นตอนจัดการ 6 ขั้น (เรียงได้ = ได้คะแนน)



1. Trap/Interrupt เข้าสู่ OSMMU เจอ bit = i (invalid) → ยิง trap ขึ้น kernel mode
2. OS ตรวจชนิดของ invalidอ้างเกิน address space จริง → abort · แค่ยังไม่โหลด → ไปขั้นถัดไป
3. หาเพจบน disk + หา frame ว่างถ้าไม่มี frame ว่าง → ใช้ Page Replacement ก่อน
4. อ่านเพจจาก disk เข้า frameขั้นที่แพงที่สุด (เป็น I/O ทั้งตัว)
5. อัปเดต page tableใส่ frame number + เปลี่ยน bit เป็น v (valid)
6. Restart คำสั่งเดิมCPU รัน instruction ที่พังอีกครั้ง — คราวนี้อ่านได้ปกติ

> ⚠️ **Dirty page**
> เพจที่ถูกแก้ไข (dirty (เพจที่ถูกแก้ไขข้อมูลแล้ว)/modified = 1) ต้อง**เขียนกลับ disk ก่อน** (write back (เขียนกลับลงดิสก์ก่อน)) ค่อยแทนที่ — เพจ clean (เพจที่ยังไม่ถูกแก้ไข (ดิสก์มีสำเนาแล้ว)) ทับได้เลยเพราะ disk มีสำเนา

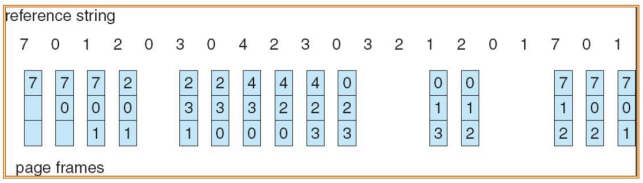
I3. Page Replacement — นับ fault 3 อัลกอริทึม 🔥🔥

โจทย์หลัก (สไลด์ใช้ string เดียวกัน, 3 frames (เฟรมหลายก้อน) เริ่มว่าง):

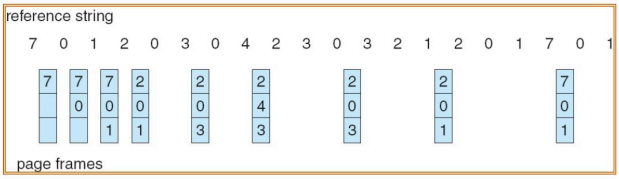
```
Reference String (ลำดับการอ้างเพจ): 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1
```

อัลกอริทึม	มองไหน	หลักเลือก victim	Faults (3 frames)	ใช้จริง?
FIFO	คิวเข้า	เข้ามาก่อน ออกก่อน (ไม่สนว่าใครฮิต)	**15**	ง่ายแต่โง่ · เจอ Belady
Optimal (OPT)	อนาคต 👁️	ตัวที่**จะไม่ถูกใช้อีกนานสุด**ในอนาคต	**9**	**ใช้จริงไม่ได้** (รู้อนาคตไม่ได้) — ไว้เทียบ benchmark
LRU	อดีต 🔙	ตัวที่**ไม่ได้ใช้มานานสุด**	**12**	ใช้จริงได้ (เก็บประวัติย้อนหลัง) ดีกว่า FIFO

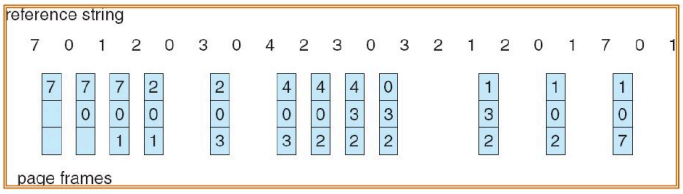
👕 ตู้เสื้อผ้า 3 ชั้น
FIFO = รีดตัวใส่นานสุดก่อน · Optimal = หมอดู รีดตัวที่จะไม่ใส่อีกนานสุด · LRU = จำได้ว่าใส่ตัวไหนล่าสุดเมื่อไร รีดตัวที่ทิ้งไว้นานสุด



สไลด์ p.13 FIFO Page Replacement



สไลด์ p.16 Optimal Page Replacement



สไลด์ p.17 LRU Page Replacement

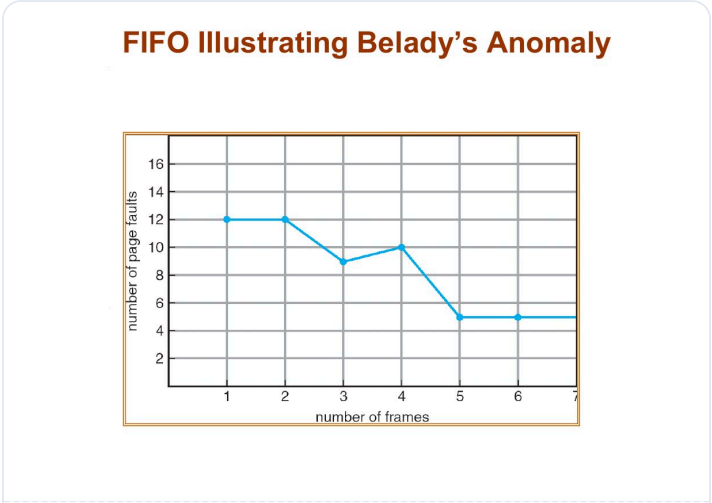
💡 เทคนิคไล่มือ: เขียน 3 ช่อง frame ใต้ string ทีละตัวอักษร · FIFO บอกตัวเลือก "เก่าสุด" ไว้ในหัว · LRU เขียนเลขกำกับ "ใช้ล่าสุดเมื่อไร" ใต้ช่อง · OPT กลับไปดูขวาของ string

I4. Belady's Anomaly — เพิ่ม frame แล้ว fault กลับโต! 🤯

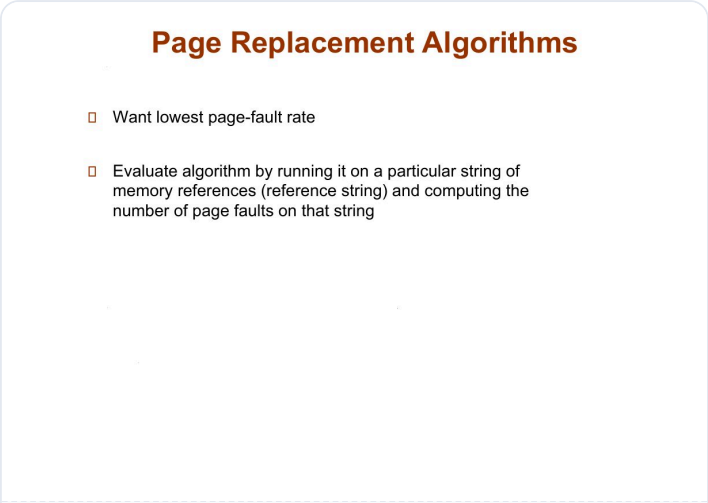
String: 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5 (FIFO):

จำนวน frame	3 frames	4 frames
Page faults (เหตุการณ์เพจไม่อยู่ใน RAM)	**9**	**10** 😱

เพิ่ม RAM fault กลับเพิ่ม — เกิดได้กับ **FIFO** (และ non-stack algorithm) แต่**ไม่เกิดกับ LRU / Optimal**



สไลด์ p.14 FIFO Illustrating Belady's Anomaly



สไลด์ p.11 Page Replacement Algorithms (กราฟ fault vs frames)

I5. EAT เมื่อมี Page Fault Rate (p)

$$EAT = (1 - p) \times \text{memory access time} + p \times \text{page fault service time}$$

ตัวเลขชวนคิด: mat = 200ns, fault service = 8ms → ที่ p = 0.001 (fault แค่ 0.1%) EAT = 0.999×200 + 0.001×8,000,000 ≈ 8,200ns = 8.2µs — ช้าขึ้น 40 เท่า! ห้ามให้ p โต

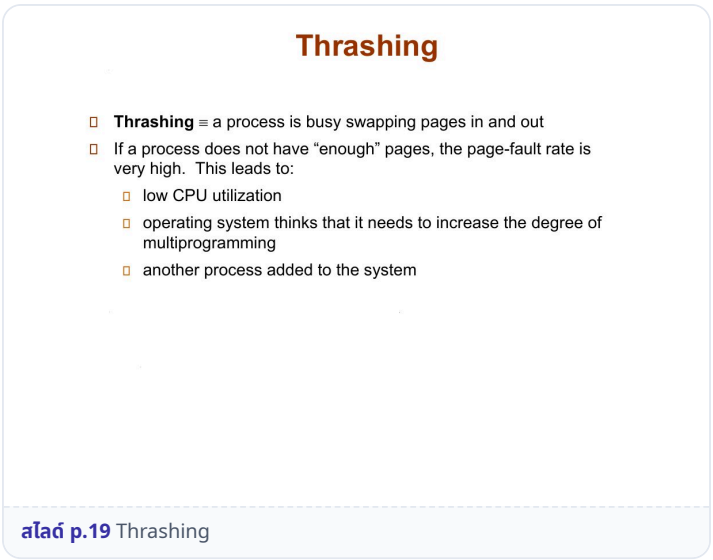
I6. Global vs Local Allocation

- **Global replacement**: แทนที่ได้จาก **frame ใดก็ได้ในระบบ** — แย่งข้าม process ได้
- **Local replacement**: เลือกจาก**เฉพาะ frame ของตัวเอง** (คนละบ้านกับคนละบ้าน)

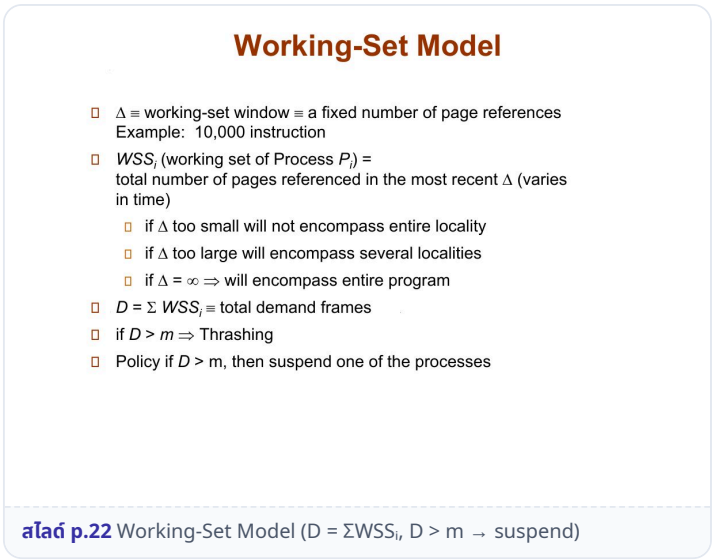
I7. Thrashing · Locality · Working Set (ชุดเพจที่ต้องมีใน RAM) 🔥

- **Thrashing**: process มีเพจไม่พอ → สลับเพจเข้า-ออก disk ตลอดเวลา → **CPU utilization ดิ่ง** · OS เข้าใจผิดว่า "งานน้อย" เลยเพิ่ม process → thrashing (สลับเพจหนักจนระบบแย่) ยิ่งหนัก (วงจรนรก) · เกิดทั้งข้ามงานและในงานเดียว (ลูปข้ามเพจ)
- **Locality of reference**: โปรแกรมมักอ้างกลุ่มเพจเดิมซ้ำ ๆ ในช่วงสั้น ๆ (อยู่ในฟังก์ชัน/ลูปเดิม)
- **Working set**: กลุ่มเพจที่ process ต้องมีใน RAM เพื่อไม่ให้ fault ในช่วง locality นั้น — มองย้อนหลัง Δ (working-set window เช่น 10,000 instruction) · **WSS_i** = จำนวนเพจที่ P_i อ้างใน Δ · Δ เล็กไปครอบ locality ไม่หมด / ใหญ่ไปกินหลาย locality / Δ = ∞ คือทั้งโปรแกรม
- **$D = \Sigma WSS_i$** = ความต้องการ frame รวม · ถ้า **D > m (frame จริง)** → **thrashing!** → OS ต้อง **suspend บาง process** คืน frame ให้คนที่เหลือ

> 🍳 **อุปมา**
> Working set = ของที่ต้องมีบนเคาน์เตอร์เพื่อทำเมนูให้จบ ถ้าวางไม่ครบ คุณวิ่งไปตู้เย็น (disk) ทุก 30 วินาที = ไม่มีอะไรเสร็จ → ทางแก้: ลดจำนวนเมนูที่ทำพร้อมกัน (suspend process)



สไลด์ p.19 Thrashing



สไลด์ p.22 Working-Set Model (D = ΣWSSᵢ, D > m → suspend)

I8. โจทย์สุดคลาสสิก: Array 128×128 🔥 (สไลด์ p.24)

`int data[128][128]` · **1 แถว = 1 เพจ** · เก็บแบบ **row-major** (เรียงตามแถว)

	ลูป	พฤติกรรม	Page faults
❌ **แย่**	`for (j...) for (i...) data[i][j]=0;` (column นอก, row ใน)	ทุกครั้งกระโดดข้ามแถว = เปลี่ยนเพจทุกครั้ง	**128 × 128 = 16,384**
✅ **ดี**	`for (i...) for (j...) data[i][j]=0;` (row นอก, column ใน)	อยู่เพจเดียวกันทั้งแถว fault เฉพาะตอนเปลี่ยนแถว	**128**

โค้ดเหมือนกัน สลับลูปเท่านั้น → `fault` ต่างกัน `128` เท่า `(16,384 : 128)`

บทเรียน: เขียนลูปให้เข้าถึงข้อมูลต่อเนื่องตาม row-major = locality ดี

ไล่ address จริงให้เห็นตา — ทำไมถึงต่างกันขนาดนั้น

สมมติเพจเริ่มที่ address `A` — สมาชิกติดกันในแถวห่างกัน 4 bytes แต่**คนละแถวห่างกัน 128 × 4 = 512 bytes = ครบ 1 เพจพอดี**

	ลำดับการอ้าง address	ผลต่อเพจ
✅ **row นอก** `data[0][0] → data[0][1] → …`	A, A+4, A+8, … , A+508 (128 ตัวต่อเนื่อง) → แล้วค่อยกระโดดไป A+512	เพจเดียวกันทั้งแถว → **fault 1 ครั้ง/แถว** × 128 แถว
❌ **column นอก** `data[0][0] → data[1][0] → …`	A, A+512, A+1024, … (ทุกครั้งข้าม 1 เพจเต็ม)	เปลี่ยนเพจ**ทุกครั้ง**ที่แตะสมาชิก → 16,384 ครั้ง

💡 หลักเดียวกันใช้ได้กับทุกโจทย์ลักษณะนี้: ดูว่าการอ้างสมาชิกตัวถัดไป "ห่างกันกี่ bytes" — ถ้าห่างน้อยกว่าขนาดเพจ = อยู่เพจเดียวกัน (ไม่ fault) · ถ้าห่างเกินขนาดเพจ = fault

J. File System (OSNo10)

📖 รู้จักส่วนนี้ก่อนอ่าน

ความหมาย: File System = ผู้จัดการไฟล์บนดิสก์ — ตั้งชื่อ เก็บ ดึง แก้ไข จัด metadata และกำหนดสิทธิ์การเข้าถึง โดยที่เราไม่ต้องรู้เลยว่าข้อมูลอยู่เซกเตอร์ไหนของฮาร์ดดิสก์

ความสำคัญกับข้อสอบ: เทียบ FAT32 (ระบบไฟล์เก่า ไฟล์เกิน 4GB ไม่ได้)/NTFS (ระบบไฟล์ของ Windows)/ext4/exFAT (FAT32 ตัวใหม่ ไฟล์ใหญ่ได้)/APFS (ระบบไฟล์ของ Apple), วิธีจัดสรรบล็อก 3 แบบ, คำนวณ inode (โครงสร้างเก็บข้อมูลไฟล์ของยูนิกซ์), hard link vs symbolic link, สิทธิ์ rwx → ส่วนใหญ่ออกถูก/ผิด + โจทย์คำนวณสั้น ๆ

ใช้จริงในคอมพิวเตอร์: เลือกฟอร์แมตแฟลช (FAT32 เก็บไฟล์เกิน 4GB ไม่ได้!), คำสั่ง chmod/ls -l ที่เจอทุกวันใน Linux, สร้าง shortcut ไฟล์ = symbolic link — ใช้ในชีวิตประจำวันจริง

J1. File System รับผิดชอบอะไร

จัดการเก็บ/ดึง/จัดระเบียบข้อมูลบน storage — หน้าที่: **ตั้งชื่อ+โครงสร้างไฟล์ · เก็บข้อมูล · ดึง/แก้ไข · Metadata · ความปลอดภัย/สิทธิ์เข้าถึง** · หัวใจคือ **abstraction**: interface เข้าถึงไฟล์เหมือนกันไม่ว่าข้อมูลอยู่บน HDD/SSD/USB (device independence — จาก Quizizz 4 ข้อ 2)

J2. 5 ระบบไฟล์ยอดนิยม

ระบบไฟล์	ไฟล์สูงสุด	Volume สูงสุด	Journaling (สมุดบันทึกเพื่อกู้คืนหลังพัง)	ใช้ที่ไหน
FAT32 (1996)	**4 GB** 😅	8 TB	ไม่มี	USB/การ์ด — เข้ากันได้*ทุก*เครื่อง
NTFS (1993)	16 TB	256 TB	✅	Windows — สิทธิ์/เข้ารหัส (EFS)/compression/quota
ext4 (2008)	16 TB	1 EB	✅	Linux หลัก (ต่อจาก ext3)
exFAT (2006)	16 EB	128 PB	ไม่มี	USB/SD ใหญ่ข้าม OS — แก้ขีดจำกัด 4GB
APFS (2017)	8 EB	8 EB	✅ + Snapshots	Apple ทั้งหมด — เข้ารหัสทั้งดิสก์

🎯 จาก Quizizz 4

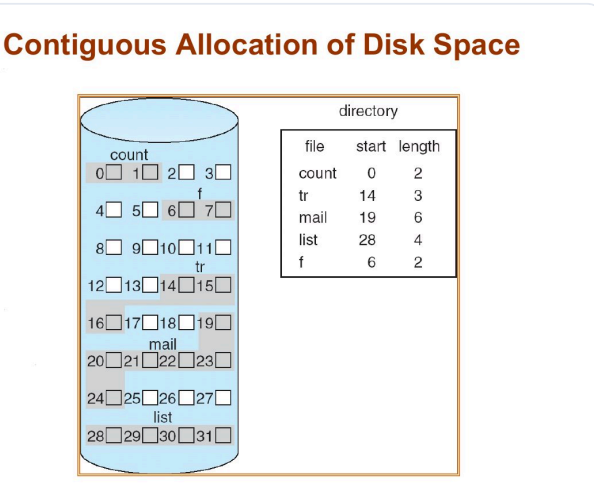
• USB/external drive = **ExFAT** · • **Journaling ≠ เข้ารหัส!** — journaling คือบันทึก log ก่อนเขียนจริงเพื่อ**กู้คืนหลัง crash** · เข้ารหัสของ NTFS คือ **EFS** แยกกัน · • FAT32 ไฟล์เดี่ยว**ต้น 4GB เด็ดขาด** (ฟิลด์ 32 บิต) ดิสก์ว่างเยอะก็ไม่ช่วย

J3. File (ไฟล์) Allocation Methods 🔥 (วิธีจัดสรรบล็อกให้ไฟล์)

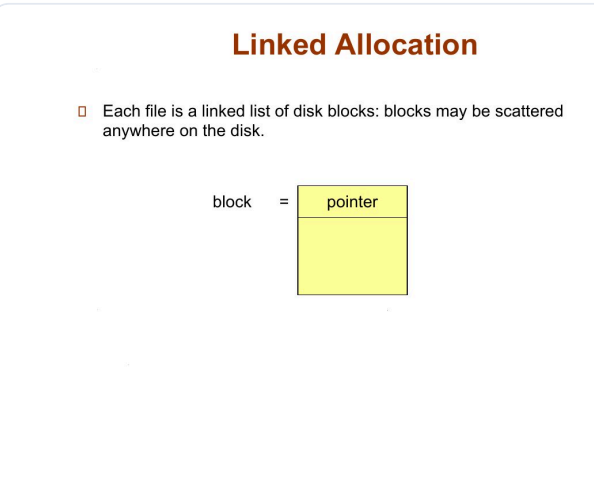
วิธี	หลักการ	ข้อดี	ข้อเสีย
Contiguous	บล็อก**ติดกันเป็นผืน** · directory เก็บ start + length	อ่านเร็วมาก (ไม่ต้อง seek)	**External fragmentation** + ขยายไฟล์ยาก (บล็อกถัดไปถูกจอง = ต้องย้ายทั้งไฟล์)
Linked	บล็อกกระจาย + pointer (ตัวชี้ตำแหน่ง) ชี้บล็อกถัดไปเป็นโซ่ · directory เก็บ start + end	ไม่มี external frag · ขยายง่าย	**Random access ช้ามาก** (ไล่โซ่) · pointer เปลืองพื้นที่ · pointer พัง = ท่อนหลังหาย
Indexed	รวม pointer ทุกตัวไว้ใน **Index Block** · directory ชี้มาจุดเดียว	**Random access เร็ว** · ไม่มี external frag	เปลืองบล็อก index (ไฟล์จิ๋วก็เสีย 1 บล็อก) · ไฟล์ใหญ่ต้อง multi-level

🎯 FAT (File Allocation Table (ตาราง))

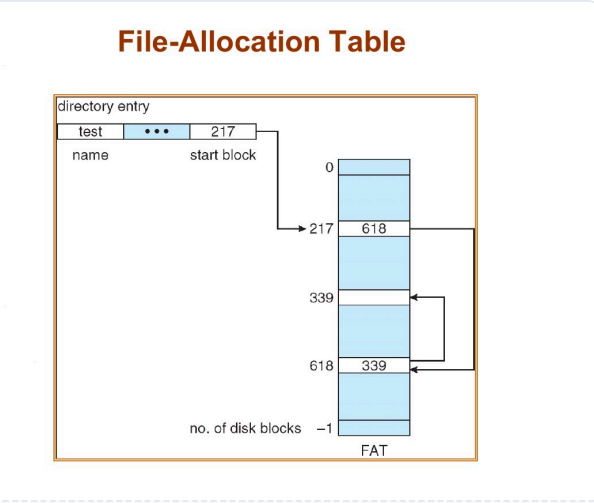
FAT = ดึง pointer ทั้งหมดออกจาก data block มารวมไว้ใน**ตารางเดียว (อยู่ใน RAM)** → ไล่โซ่ได้เร็วโดยไม่ต้องเปิดดิสก์ทีละบล็อก — นี่คือที่มาของชื่อ FAT filesystem



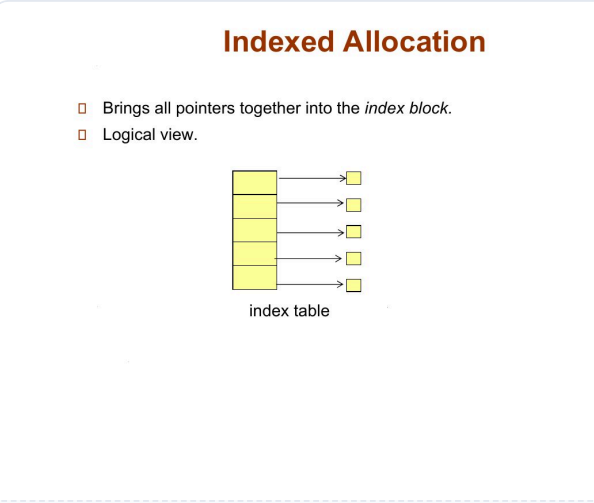
สไลด์ p.19 Contiguous Allocation (การจัดสรรบล็อกติดกัน)



สไลด์ p.20 Linked Allocation (การจัดสรรแบบโซ่ pointer)



สไลด์ p.24 File-Allocation Table



สไลด์ p.25 Indexed Allocation (การจัดสรรแบบสารบัญ)

J3.1 คำนวณ index / inode 🔥 (สูตรเดียวใช้ทุกโจทย์)

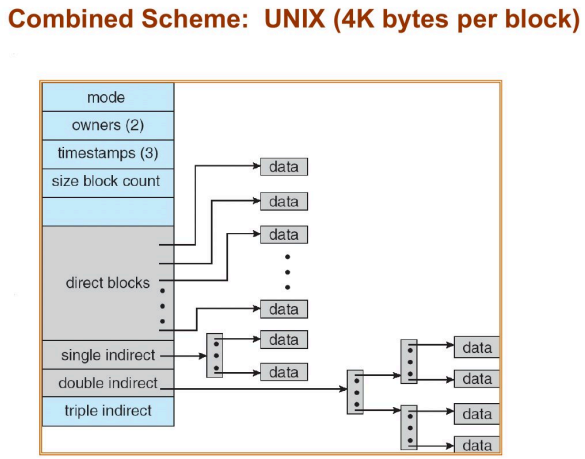
`Entries ต่อ 1 index block = Block Size ÷ Pointer Size`

จาก Quizizz 4: บล็อก 2KB ÷ pointer 4B = **512 entries** → ไฟล์ใหญ่สุด single-level = 512 × 2KB = 1MB

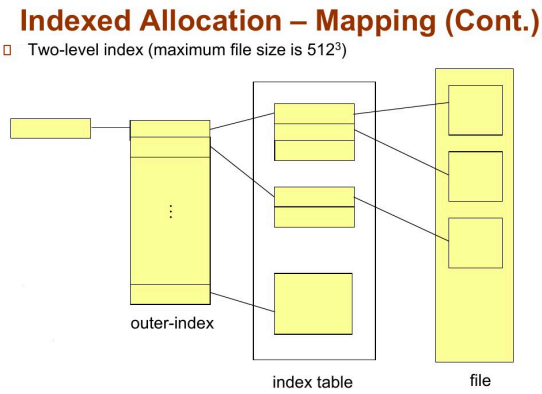
UNIX Combined Scheme — inode (สไลด์ p.29, block 4KB, pointer 4B): มี pointer 15 ตัว

Pointer	จำนวน	คำนวณ	ขนาด
Direct ชี้ data ตรง ๆ	12	12 × 4KB	**48 KB**
Single indirect	1	(4KB÷4B) × 4KB = 1024 × 4KB	**4 MB**
Double indirect	1	1024^2 × 4KB	**4 GB**
Triple indirect	1	1024^3 × 4KB	**4 TB**
ไฟล์ใหญ่สุด ≈ 48KB + 4MB + 4GB + 4TB			**≈ 4 TB**

> **🎯 จุดพลาด**
> ลืมหา "1024 ช่องต่อบล็อก" ก่อน / ลืมบวก direct 48KB / บวกเลขผิดหลัก (4MB ≠ 4×1024KB=4096KB ก็คือ 4MB ตรงกัน แต่ double = 1024×1024×4KB = 4GB)



สไลด์ p.29 Combined Scheme: UNIX (4K/block) — 15 pointers (ตัวชี้ตำแหน่ง)



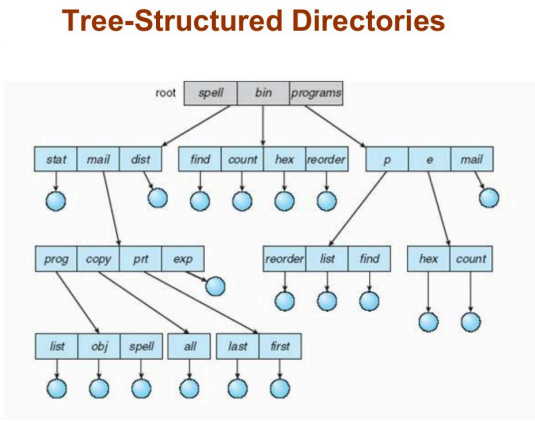
สไลด์ p.28 Two-level index (512^3 ในสไลด์ = block 512B)

J4. Directory (โฟลเดอร์/สารบัญไฟล์) · Tree vs Acyclic Graph

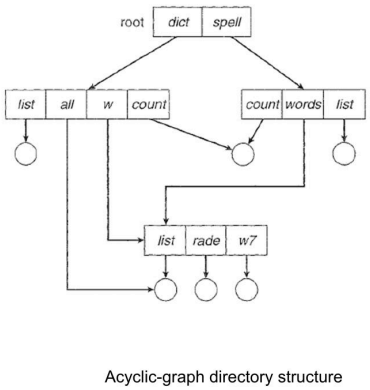
- **Directory** = symbol table แปลง "ชื่อไฟล์ → ข้อมูลไฟล์" · operations: search, create, delete, list, rename, traverse
- **Tree**: ไฟล์มี parent เดียว — ปลอดภัยแต่**แชร์ไม่ได้**
- **Acyclic graph** (ไม่มีวงวน): แชร์ได้ผ่าน **link** — ปัญหา: aliasing (ไฟล์เดียวหลายชื่อ), การลบไฟล์แชร์

การลบไฟล์แชร์ 2 แนวทาง

แนวทาง	ทำอย่างไร	ข้อควรระวัง
ลบเมื่อไรก็ได้	ลบไฟล์ทันทีที่ใครสั่ง	**Dangling pointer** — link ที่เหลือชี้หาอะไรที่ไม่มีแล้ว (แนวทาง symbolic link)
เก็บจน reference หมด	นับ **reference count**: เพิ่ม link → count+1, ลบ link → count−1, **count = 0 ค่อยปล่อยพื้นที่**	ต้องตรวจว่ากราฟไม่มีวงวน (cycle) — แนวทาง hard link ของ UNIX



สไลด์ p.32 Tree-Structured Directories



สไลด์ p.33 Acyclic-graph Directory

J5. Hard Link (ลิงก์ชี้ inode เดียวกัน) vs Symbolic (Soft) Link (ลิงก์ (ทางเชื่อม)) 🔥

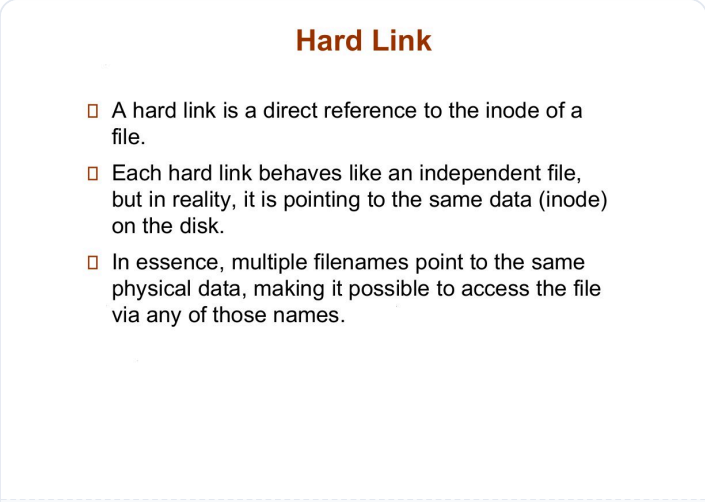
	Hard Link `ln original new`	**Symbolic Link (ลิงก์แบบเก็บ path (shortcut))** `ln -s original new`
ชี้ไปที่	directory entry ชี้**inode เดียวกัน**โดยตรง + เพิ่ม link count	**ไฟล์ใหม่แยกต่างหาก** เก็บ "path" ของต้นทาง (shortcut)
ลบต้นฉบับแล้ว	ข้อมูลยังอยู่ — ปล่อยจริงเมื่อ **link count = 0**	กลายเป็น **broken/dangling link**
ข้าม partition/FS	❌ ห้าม	✅ ได้
ชี้ directory	❌ ห้าม (กัน cycle)	✅ ได้

🏠 อุปมา

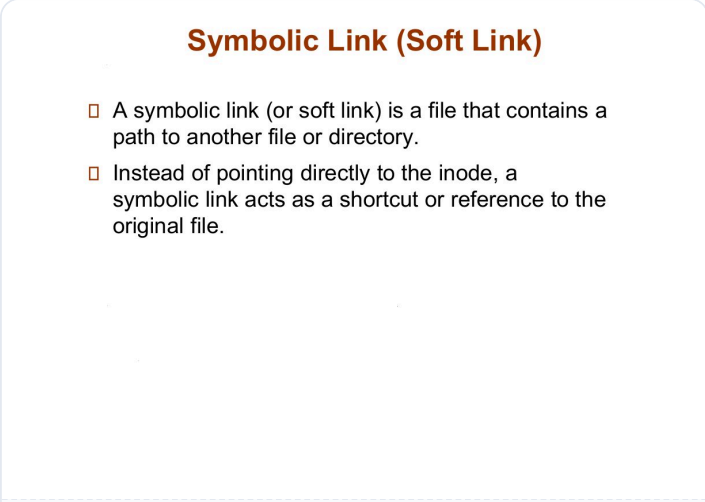
Hard link = ลงทะเบียนบ้านเพิ่มอีกชื่อ — บ้านหลังเดิม อยู่ต่อจนคนทะเบียนหมด · Soft link = กระดาษโน้ตเขียนว่า "บ้านอยู่ซอย 5" — โยนบ้านทิ้ง กระดาษก็ชี้อากาศ

🎯 จาก Quizizz 4

• Symbolic link อีกชื่อคือ **soft link** ไม่ใช่ hard link · • Hard link จัดการด้วย **Reference Counter** ✓ · • **ไม่มีคำสั่ง hardln!** — จริงคือ `ln` (hard) / `ln -s` (symbolic)



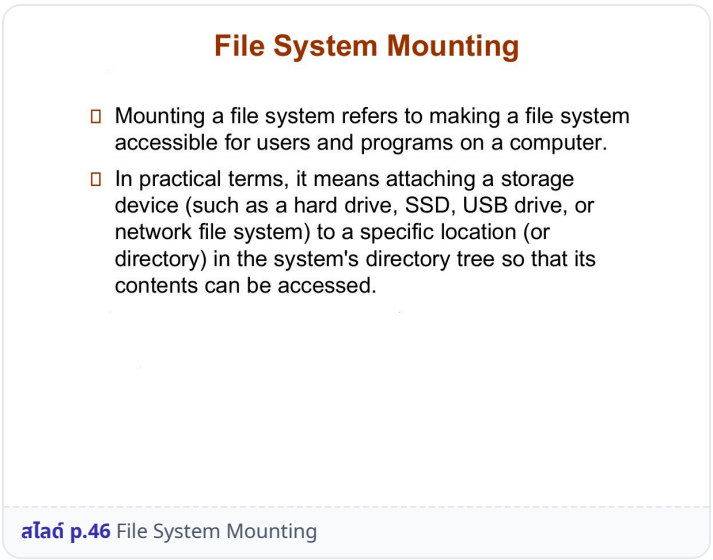
สไลด์ p.42 Hard Link



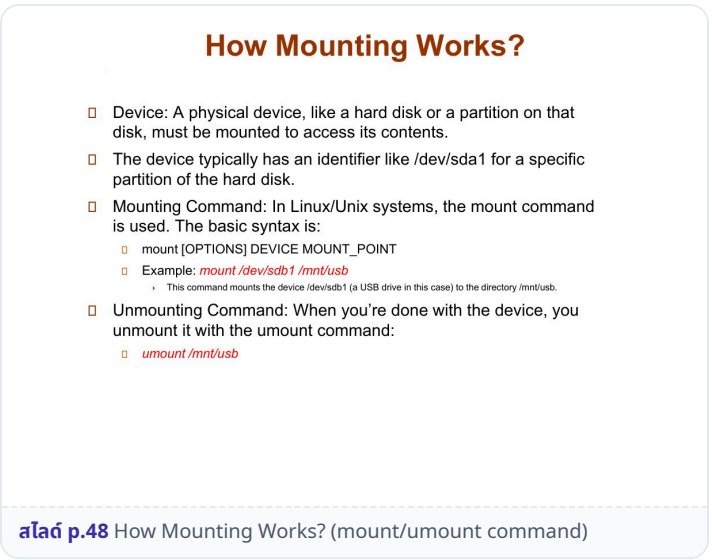
สไลด์ p.44 Symbolic Link (Soft Link)

J6. Mounting (การติดตั้งดิสก์เข้าระบบไฟล์)

- **Mounting** = ติด filesystem ของอุปกรณ์ (partition/USB/network) เข้า directory tree ที่ **mount (ติดตั้งเข้าระบบไฟล์) point** เช่น `mount /dev/sdb1 /mnt/usb`
- ก่อนถอดต้อง `umount` — ให้การเขียนค้าง ๆ เสร็จก่อน ไม่งั้นข้อมูลพัง
- จาก Quizizz 4: อุปกรณ์เดียว **mount ได้หลาย mount point** (ข้อความว่า "ไม่สามารถ" = เท็จ)



สไลด์ p.46 File System Mounting



สไลด์ p.48 How Mounting Works? (mount/umount command)

J7. Protection — สิทธิ์ RWX 🔥

เจ้าของไฟล์คุมได้ 2 อย่าง: **what can be done** (read/write/execute) กับ **by whom** (owner/group/others)

$$R = 2^2 = 4 \cdot W = 2^1 = 2 \cdot X = 2^0 = 1$$

3 กลุ่ม: owner, group, other (public) — ตัวอย่างสไลด์ p.50: owner=7 (rwx), group=6 (rw-), public=1 (--x) = 761

เลข	บวกได้จาก	แปลง
7	4+2+1	rwx — ทุกสิทธิ์
6	4+2	rw-
5	4+1	r-x (ไม่ใช่ 6! จาก Quizizz)
4	4	r--
0	—	--- ไม่มีสิทธิ์



สไลด์ p.50 Access Lists and Groups (RWX 7 6 1)

K. ความรู้จาก Quizizz ในห้องเรียน (4 ชุด) — จัดเรียงตามชุด

📖 รู้จักส่วนนี้ก่อนอ่าน

ความหมาย: ความรู้ที่สกัดจากแบบควิซในห้องเรียนจริง 4 ชุด (Signal/Pipe, Shared Memory (หน่วยความจำที่แชร์กัน), Semaphore, File Management) — ทุกข้อมี "ความรู้สำคัญ" ที่ตอบโจทย์แบบเดิมได้ทันที

ความสำคัญกับข้อสอบ: นี่คือธนาคารข้อสอบย่อที่ตรงสไตล์อาจารย์ที่สุด — หลายข้อมีกับดักที่อาจารย์ตั้งใจหลอก (เช่น ข้อ 8 ชุด 4 มีคีย์ขัดตำรา — ต้องตอบ False ตามคีย์!) อ่านก่อนสอบคุ้มที่สุด

ใช้จริงในคอมพิวเตอร์: ประเด็นในควิซคือข้อผิดพลาดจริงที่โปรแกรมเมอร์พลาดบ่อย เช่น เข้าใจว่า signal() ใช้ส่งสัญญาณ, ลืมปิดปลาย pipe (ท่อส่งข้อมูลทางเดียว) — รู้ไว้เขียนโค้ดจริงไม่พัง

ทุกข้อเป็นความรู้ที่ตอบโจทย์แบบเดิมได้ทันที — อ่านกล่อง "ความรู้สำคัญ" ต่อข้อ

K1. Quiz 1 — IPC & Unix Signal

ข้อ 2 False

"Signal ใช้ส่งข้อความระหว่าง process"

ความรู้: Signal = **event notification** เท่านั้น — ไม่มี payload แนบข้อมูลไม่ได้ ผู้รับรู้แค่ว่า "signal ชนิด X มา" · ส่งข้อมูลจริงใช้ Message Passing (การส่งข้อความผ่าน OS) / Shared Memory

ข้อ 3 Message Passing

"IPC ที่ OS จัดจังหวะการรับส่งให้"

ความรู้: Message passing ให้ OS จัดทั้ง send/receive และ**จังหวะ** (blocking/non-blocking) · Shared Memory จัดเตรียมแค่พื้นที่ — **ซิงก์จังหวะเป็นหน้าที่ของ process เอง** (เช่นใช้ semaphore)

ข้อ 4 SIGINT

"Ctrl-C ส่ง signal อะไร"

ความรู้: Ctrl-C → **SIGINT** (INT = INTerrupt) · คู่เทียบ: SIGFPE = หารศูนย์, SIGTERM = ขอจบสุภาพ (default ของ kill (ส่งสัญญาณไปยังกระบวนการอื่น)), SIGQUIT = Ctrl-\, SIGKILL = ฆ่าทันทีจับไม่ได้ (9)

ข้อ 5 False

"signal() ใช้ส่ง signal ระหว่าง process"

ความรู้: `signal()` = **ลงทะเบียน handler (ฟังก์ชันรับมือเหตุการณ์)** ("โดน X แล้วทำอะไร") · ตัวส่งคือ `kill(pid, sig)` (ส่งข้าม process) และ `raise(sig)` (ส่งให้ตัวเอง)

ข้อ 6 False

"1 process จัดการ signal ได้แค่ 1 ตัว"

ความรู้: จัดการได้**หลาย signal** — แต่ละชนิดมี handler ของตัวเอง · ข้อจำกัดจริง: 1 ชนิด มี 1 handler (ลงทะเบียนซ้ำ = ทับตัวเดิม)

ข้อ 7 False

"kill() ใช้ส่งได้แค่ SIGKILL"

ความรู้: System (ระบบ) call `kill(pid, sig)` ส่งได้**ทุกชนิด** — SIGKILL แค่ 1 ตัวเลือก (ชื่อมาจากประวัติศาสตร์ ทำให้เข้าใจผิด)

ข้อ 8 True

"คำสั่ง kill ใน UNIX ใช้ส่ง signal ให้ process"

ความรู้: หน้าที่จริงของคำสั่ง kill = **ส่ง signal** (default SIGTERM) · `kill -9 pid` = SIGKILL · ⚠️ แยกจากข้อ 7: system call ส่งได้ทุกชนิด (โจทย์ใส่ "เท่านั้น" มาหลอก)

K2. Quiz 2 — Shared Memory & Message Passing (Pipe)

ข้อ 2 False

"Message passing รองรับทิศทางเดียว"

ความรู้: ทิศเดียวจริง ๆ คือ **pipe ธรรมดา (anonymous)** — สองทิศต้อง 2 pipe · Message passing แต่ละ process มีทั้ง send/receive = **สองทิศได้**

ข้อ 3 อาเรย์ int 2 ตัว

"พารามิเตอร์ของ pipe()"

ความรู้: `int pfd[2]; pipe(pfd);` — OS ใส่ค่า 2 ช่อง: **pfd[0] = ปลายอ่าน, pfd[1] = ปลายเขียน** (file (ไฟล์) descriptor ไม่ใช่ชื่อไฟล์ — pipe ธรรมดาไม่มีชื่อ)

ข้อ 4 write(pfd[1], data, length)

"ประโยคส่งข้อมูลเข้า pipe"

ความรู้: pipe ใช้ system call ไฟล์มาตรฐาน: **write()** ส่ง / **read()** รับ · เขียนต้องลงปลายเขียน **pfd[1]** · จำ: "write to 1, read from 0"

ข้อ 5 dup2(pfd[1], 1)

"เปลี่ยนทิศ stdout ไปปลายเขียน pipe"

ความรู้: `dup2(oldfd, newfd)` = ทำให้ **newfd** ชี้ที่เดียวกับ oldfd · fd 0=stdin, 1=stdout, 2=stderr → stdout(1) ไหลเข้าปลายเขียน = `dup2(pfd[1],1)` · pattern มาตรฐาน: ฝั่งเขียน dup2(pfd[1],1) แล้ว exec / ฝั่งอ่าน dup2(pfd[0],0) · ปิดปลายที่ไม่ใช้ก่อน

ข้อ 6 shmget()

"System call ขอสร้าง shared memory"

ความรู้: shm**get** = get — ขอสร้าง/อ้างถึง segment ได้ **shmid** · (shmat=attach, shmdt=detach, shmctl=control/ลบ)

ข้อ 7 ftok()

"ฟังก์ชันสร้าง key ของ shared memory"

ความรู้: `key_t ftok(path, id)` — path + id เดียวกัน → key เดียวกัน → อ้าง shm ก้อนเดียวกันได้ (นิยม path = ".")

ข้อ 8 0764

"rwxrw-r-- เป็นเลขฐานแปดอะไร"

ความรู้: r=4 w=2 x=1 → rwx=7, rw-=6, r--=4 → **0764** (ใช้กับ shmget flag เช่น IPC_CREAT | 0666 ด้วย)

ข้อ 9 IPC_PRIVATE

"shmget ที่การันตีไม่ซ้ำกับใครแน่นอน"

ความรู้: IPC_PRIVATE = key พิเศษ OS สร้าง segment ใหม่ไม่ซ้ำใครเสมอ — นิยมใช้ parent→child (shmid ส่งต่อได้ในตระกูล) · IPC_EXCL แค่ flag กันชน ไม่การันตี

ข้อ 10 shared memory id

"shmget() คืนอะไร"

ความรู้: คืน **shmid** (เลขจำนวนเต็ม) — เอาไปใช้กับ shmat/shmctl/shmdt · flow: key → shmget → **shmid** → shmat → pointer

ข้อ 11 shmat()

"ฟังก์ชันให้ตัวแปรอ้างถึงตำแหน่ง shared memory"

ความรู้: shm**at** = attach — ผูก segment เข้า address space แล้ว **คืน pointer**: `shm_ptr = shmat(shmid, NULL, 0)`

ข้อ 12 False

"ตัวแปรเก็บตำแหน่ง shm ต้องเป็น pointer ไป int เท่านั้น"

ความรู้: shmat คืน **void*** — cast เป็นชนิดไหนก็ได้ (char*, struct data* ...) ข้อจำกัดเดียวคือต้องเป็น pointer

ข้อ 13 struct data *shm_ptr;

"shmget(..., sizeof(struct data), ...) ตัวแปรชี้ต้องประกาศยังไง"

ความรู้: จองขนาด sizeof(struct data) → pointer ต้องตรงชนิด: `struct data *p; p = (struct data *) shmat(shmid, NULL, 0);` ใช้ผ่าน `p->field`

🟢 Shared Memory ครบวงจร (จำ 6 ขั้น)

```
key_t key = ftok(".", 'a');                    // 1. สร้าง key
int shmid = shmget(key, sizeof(struct data), IPC_CREAT | 0666); // 2. สร้าง segment
struct data *p = (struct data *) shmat(shmid, NULL, 0);   // 3. attach
/* 4. อ่าน/เขียนผ่าน p */
shmdt(p);                                      // 5. detach
shmctl(shmid, IPC_RMID, NULL);                 // 6. ลบ (คนสุดท้าย)
```

K3. Quiz 3 — Semaphore

ข้อ 2 False

"Semaphore ที่ใช้กับ critical section คือ counting"

ความรู้: CS ใช้ **binary semaphore (mutex)** เริ่ม 1: wait→CS→signal · counting (เริ่ม = จำนวน resource) ใช้กับ resource หลายหน่วย เช่น bounded buffer

ข้อ 3 False

"Semaphore การันตีผ่านเงื่อนไข 3 ข้อของ CS"

ความรู้: 3 เงื่อนไข = Mutual Exclusion / Progress / Bounded Waiting — semaphore การันตีได้แค่ **ข้อ 1** · progress/bounded waiting ขึ้นกับ implementation (classic อาจ starvation)

ข้อ 4 2 ตัว

"bounded buffer ใช้ counting semaphore กี่ตัว"

ความรู้: ใช้รวม 3 ตัว: empty(n), full(0), mutex(1) — แต่**counting จริงมี 2** (empty, full) เพราะ mutex เป็น binary

ข้อ 5 empty

"producer ตรวจช่องว่างด้วยตัวไหน"

ความรู้: Producer: `wait(empty)` ขอช่องว่างก่อน · Consumer: `wait(full)` ก่อนอ่าน · mutex ใช้ตอนแตะ buffer จริง (ลำดับ: resource ก่อน mutex!)

ข้อ 6 mutex

"readers-writers คุมจำนวน reader ด้วย semaphore ไหน"

ความรู้: mutex ครอบช่วง readcount++/-- · **readcount เป็น int ธรรมดา** ไม่ใช่ semaphore · wrt คุม writer / reader แรกล็อก-ตัวสุดท้ายปลด

ข้อ 7 resource

"ตะเกียบใน Dining philosopher แทนอะไร"

ความรู้: ตะเกียบ = **shared resource** (นักปราชญ์ = process) · implement ใช้ semaphore 1 ตัวต่อ 1 ตะเกียบ แต่ตัวตนของมันคือ resource

ข้อ 8 False

"semaphore การันตีนักปราชญ์ทุกคนได้กิน"

ความรู้: หยิบซ้ายพร้อมกัน → รอขวาเป็นวง = **deadlock** + ไม่มี bounded waiting → starvation · แก้: จำกัด ≤ 4 คน / คนนึงหยิบขวาก่อน / ใช้ Monitor

ข้อ 9 sem_post()

"PThread semaphore ตัวไหน = signal()"

ความรู้: PThread: `sem_init(&s,0,val)` สร้าง · **sem_wait = P (ลด)** · **sem_post = V (เพิ่ม/ปลุก)** · sem_destroy เก็บกวาด · **ไม่มี sem_signal ใน POSIX!**

K4. Quiz 4 — Introduction to File Management

ข้อ 2 True

"เข้าถึงไฟล์เหมือนกันไม่ว่าอุปกรณ์อะไร"

ความรู้: ระบบไฟล์ = ชั้น abstraction → open/read/write ชุดเดียวกันทุกอุปกรณ์ (**device independence**)

ข้อ 3 False

"Journaling = การเข้ารหัสไฟล์"

ความรู้: Journaling = write-ahead log เพื่อ**กู้คืนหลัง crash** · การเข้ารหัสเป็นฟีเจอร์แยก (NTFS ใช้ **EFS**)

ข้อ 4 ExFAT

"ระบบไฟล์สำหรับ USB drive"

ความรู้: USB/external = **ExFAT** (ใหญ่กว่า 4GB ข้าม Windows-macOS) · NTFS=Windows, ext4=Linux, APFS=Apple

ข้อ 5 False

"contiguous ปัญหาคือ internal fragmentation"

ความรู้: contiguous → **external** fragmentation (ต้องกันบล็อกติดกัน พื้นที่แตกเป็นหลุม) · internal = เศษในบล็อกสุดท้ายที่ไม่เต็ม

ข้อ 6 False

"linked allocation เป็นการจัดสรรแบบต่อเนื่อง"

ความรู้: linked = บล็อกกระจาย + pointer โซ่ = **non-contiguous** โดยนิยาม (FAT อยู่ตระกูลนี้)

ข้อ 7 False

"บล็อกใหญ่ → external fragmentation"

ความรู้: บล็อกใหญ่ → เศษในบล็อกสุดท้ายเยอะ = **internal fragmentation** (ไฟล์ 1B ในบล็อก 4KB = เสีย ~4KB)

ข้อ 8 False (ตามคีย์!)

"single-level index: ขนาดไฟล์สูงสุดถูกจำกัดด้วยจำนวนช่องใน index block"

ความรู้: ⚠️**คีย์ผู้สอน = False** (แม้ตามตำราน่าจะ True) — ตอบ False ตามคีย์ · ความรู้จริง: ไฟล์ใหญ่สุด = entries × block size

ข้อ 9 512

"บล็อก 2KB, pointer 4B → 1 index block มีกี่ช่อง"

ความรู้: entries = 2048 ÷ 4 = **512** → single-level ใหญ่สุด 512 × 2KB = 1MB · สูตรเดียวกับ inode ของ J3.1

ข้อ 10 False

"Symbolic link เรียกอีกอย่างว่า hard link"

ความรู้: symbolic = **soft link** (ไฟล์แยกเก็บ path) · hard link = entry ชี้ inode เดียวกัน

ข้อ 11 True

"Hard link จัดการด้วย Reference Counter"

ความรู้: inode เก็บ **link count** — rm แค่ลด counter ปล่อยพื้นที่เมื่อ = 0

ข้อ 12 False

"คำสั่ง hardln ใช้สร้าง Hard Link"

ความรู้: ไม่มี hardln! จริงคือ `ln target link` (hard) / `ln -s target link` (symbolic)

ข้อ 13 False

"mount อุปกรณ์เดียวกันได้แค่ 1 mount point"

ความรู้: mount อุปกรณ์เดียวได้**หลาย mount point** (รองรับ bind mount) — "ไม่สามารถ" เป็นเท็จ

L. คลังข้อสอบซ้อม Socrative ในห้องเรียน (16 ชุด · 376 ข้อ) 🔥

📖 รู้จักส่วนนี้ก่อนอ่าน

ความหมาย: คลังคำถาม-เฉลยจากการซ้อม Socrative สด ๆ ในห้องเรียนครบทั้ง 16 ชุด เรียงตามลำดับบท — ทุกข้อมีคำตอบถูกต้อง และข้อคำนวณมีวิธีคิดกำกับในคอลัมน์เฉลย

ความสำคัญกับข้อสอบ: นี่คือ "สไตล์การถามจริง" ของอาจารย์ — แนวถูก/ผิด, ปรนัย A–E และเติมคำตอบ รวมถึงโจทย์โค้ด/กราฟ/ตารางที่ให้รวมก่อนตอบหลายข้อ (แถบข้อมูลสีฟ้า 📊/💻 ต้องอ่านก่อนตอบข้อในช่วงนั้น)

ใช้จริง: อ่านเร็ว ๆ ชุดละ 3 นาที — โฟกัสข้อคำนวณ (Scheduling · Paging · Segmentation · Page Fault · inode/Disk) ที่อาจารย์ออกซ้ำทุกปี และข้อ True/False ที่เป็นกับดักคำต่อคำ

L1. Introduction to OS — ชุดที่ 1 · 40 ข้อ

ข้อ	คำถาม	คำตอบ	เฉลย / ความรู้
1	ระบบปฏิบัติการคือฮาร์ดแวร์ตัวหนึ่งในระบบคอมพิวเตอร์	**ไม่จริง**	False (ระบบปฏิบัติการเป็นซอฟต์แวร์ระบบ ไม่ใช่ฮาร์ดแวร์)
2	ระบบปฏิบัติการจะควบคุมฮาร์ดแวร์เท่านั้น	**ไม่จริง**	False (OS ควบคุมและจัดการทั้งฮาร์ดแวร์และซอฟต์แวร์/โปรแกรมประยุกต์)
3	หน้าที่หนึ่งของระบบปฏิบัติการคือการทำให้โปรแกรมทำงานร่วมกันได้	**จริง**	True
4	หน้าที่หนึ่งของระบบปฏิบัติการคือการให้ผู้ใช้ใช้งานคอมพิวเตอร์ตามสิทธิที่ได้รับ	**จริง**	True
5	ในวิชานี้หน่วยประมวลผลจะรวมถึง GPU ด้วย	**ไม่จริง**	False (ในบริบทพื้นฐานของวิชานี้ หน่วยประมวลผลหลักหมายถึง CPU)
6	ในวิชานี้หน่วยความจำจะหมายถึง RAM เท่านั้น	**จริง**	True (ในเนื้อหาวิชานี้ หน่วยความจำหลักหรือ Main Memory อ้างถึง RAM เท่านั้น)
7	การติดตามการใช้งานทรัพยากรเป็นหน้าที่ที่ต้องมีในแต่ละองค์ประกอบของระบบปฏิบัติการ	**จริง**	True
8	องค์ประกอบบางตัวของระบบปฏิบัติการไม่จำเป็นต้องมีการควบคุมการเข้าถึงทรัพยากรที่มันดูแล	**ไม่จริง**	False (ทุกองค์ประกอบที่ดูแลทรัพยากรจำเป็นต้องมีการควบคุมการเข้าถึงเสมอ)
9	ส่วนของระบบปฏิบัติการที่ทำหน้าที่ที่สำคัญในการจัดการระบบคอมพิวเตอร์คือ	**D** kernel	D. kernel
10	โปรแกรมที่ใช้โหลดเคอร์เนลขึ้นมาทำงานมีชื่อเรียกรวม ๆ ว่า	**C** bootloader	C. bootloader
11	ชื่อของโปรแกรม boot loader ที่เป็นที่นิยมใช้บนระบบปฏิบัติการ Linux	**E** GRUB	E. GRUB
12	โปรแกรมที่ใช้โหลด boot loader ถ้าใช้ BIOS โปรแกรมดังกล่าวจะถูกเก็บอยู่ใน	**B** MBR	B. MBR (Master Boot Record)
13	โปรแกรมที่ใช้โหลด boot loader ถ้าใช้ UEFI โปรแกรมดังกล่าวจะถูกเก็บอยู่ใน	**E** ESP	E. ESP (EFI System Partition)
14	เมื่อเราเปิดเครื่องคอมพิวเตอร์โปรแกรมแรกที่จะถูกโหลดเข้ามารันคือ	**D** UEFI	D. UEFI (หรือ BIOS ในเครื่องรุ่นเดิม)
15	โปรแกรม BIOS มักจะถูกเก็บไว้ใน ของเครื่องคอมพิวเตอร์	**B** ROM	B. ROM
16	ถ้าใช้ UEFI การแสดงรายการของ OS ที่ติดตั้งอยู่ในเครื่องสามารถทำได้โดยไม่ต้องไปอ่านข้อมูลจาก MBR	**จริง**	True (UEFI บันทึก Boot Entries ไว้ใน NVRAM/ESP)
17	ข้อดีประการเดียวที่ BIOS เหนือกว่า UEFI คือมันทำงานเร็วกว่า	**ไม่จริง**	False (UEFI ทำงานเร็วกว่าและรองรับคุณสมบัติสมัยใหม่มากกว่า)
18	จะต้องมีการกันเนื้อที่หน่วยความจำส่วนหนึ่งไว้ให้ระบบปฏิบัติการใช้	**จริง**	True (สงวนไว้สำหรับ OS Kernel และตารางข้อมูลของระบบ)
19	ใน Dual Mode Operation คำสั่งที่จะถูกเรียกใช้เมื่อเครื่องอยู่ใน kernel mode เท่านั้นเรียกว่า	**D** Privileged Instruction	D. Privileged Instruction
20	คำสั่งอ่านข้อมูลวันที่และเวลาปัจจุบันของระบบไม่จำเป็นต้องเป็น Privileged Instruction	**จริง**	True (การอ่านค่าไม่อันตรายต่อความมั่นคงของระบบ จึงไม่ต้องเป็น privileged instruction)
21	การที่เราติดตั้งระบบปฏิบัติการมากกว่าหนึ่งตัวลงในคนละ partition ของฮาร์ดดิสก์เรียกว่าการทำ virtual machine	**ไม่จริง**	False (เรียกว่า Multi-boot หรือ Dual-boot)
22	คำสั่งกำหนดค่าให้กับตัวแปรควรเป็น privileged instruction เพราะมีการใช้งานหน่วยความจำซึ่งถ้าไม่ป้องกันอาจมีโปรแกรมไม่ประสงค์ดี ใช้งานหน่วยความจำได้อย่างอิสระ	**ไม่จริง**	False (การกำหนดค่าตัวแปรใน User Space เป็นคำสั่งพื้นฐานของโปรแกรมทั่วไป ไม่ใช่ privileged instruction)
23	ถ้าเรารันโปรแกรม Oracle Virtual Box ภายใต้ระบบปฏิบัติการ Mac OS และใช้โปรแกรม Oracle Virtual Box สร้าง Virtual Machine และติดตั้งระบบปฏิบัติการ Windows 10 ลงบน virtual machine ดังกล่าว โปรแกรม Oracle Virtual Box เป็นโปรแกรมที่เรียกว่า	**A** Virtualization Software	A. Virtualization Software (หรือ Hypervisor Type 2)
24	จากสถานการณ์ข้อ 23 Mac OS เป็นโปรแกรมที่เรียกว่า	**C** Host OS	C. Host OS
25	จากสถานการณ์ข้อ 23 Windows 10 เป็นโปรแกรมที่เรียกว่า	**B** Guest OS	B. Guest OS
26	ถ้าเรามี Windows 10 เป็น Host OS เราไม่สามารถสร้าง virtual Machine ที่ติดตั้ง Windows 10 ได้ เพราะมันจะทำงานขัดแย้งกัน	**ไม่จริง**	False (สามารถสร้างและรัน OS เดียวกันซ้อนกันได้เนื่องจากแยกสิ่งแวดล้อมกันอย่างสมบูรณ์)
27	แนวคิดของ Virtual Machine สามารถนำไปใช้กับระบบ Cloud ได้ด้วย	**จริง**	True
28	แนวคิดของ container technology คือการรวมเอา application และ guest OS เข้าไว้ด้วยกัน	**ไม่จริง**	False (Container แชร์ Host OS kernel ร่วมกัน ไม่ได้มี Guest OS เต็มตัว)
29	แนวคิดของ Virtual Machine สามารถนำไปใช้กับภาษาโปรแกรมได้ด้วย	**จริง**	True (เช่น Java Virtual Machine: JVM)

30	container แต่ละตัว จะทำงานบนสภาพแวดล้อมของตัวเอง ไม่ขึ้นกับสภาพแวดล้อมของ container ตัวอื่น	**จริง**	True
31	โปรแกรมหลักในการจัดการการทำงาน Container Technology ของ Docker เรียกว่า	**A** docker engine	A. docker engine
32	สิ่งที่เรา download จาก docker hub คือ	**B** docker image	B. docker image
33	สิ่งที่เรารันจากสิ่งที่เรา download จาก docker hub คือ	**C** docker container	C. docker container
34	หนึ่ง docker image สามารถรันได้แค่หนึ่ง docker container	**ไม่จริง**	False (Image เดียวสร้าง Container ได้ไม่จำกัด)
35	container ที่รันอยู่บนเครื่องเดียวกันจะใช้ Host OS ตัวเดียวกัน	**จริง**	True
36	คำสั่ง docker ที่ใช้รัน container จาก docker image คือ	**A** docker run	A. docker run
37	คำสั่ง docker ที่ใช้แสดง docker image บนเครื่องเราคือ	**B** docker images	B. docker images
38	คำสั่ง docker ที่ใช้เรียกให้ container ที่เคยรันอยู่แล้ว แต่เราสั่งหยุดไป ให้กลับมาทำงานคือ	**C** docker start	C. docker start
39	คำสั่ง docker ที่ใช้แสดง container ที่อยู่บนเครื่องเราคือ	**E** docker ps	E. docker ps
40	คำสั่ง docker rm คือการลบ docker image ออกจากเครื่องของเรา	**ไม่จริง**	False (`docker rm` ใช้ลบ container, ถ้าลบ image ต้องใช้ `docker rmi`)

L2. CompOr and OS & Categories of OS — ชุดที่ 2.1 · 16 ข้อ

ข้อ	คำถาม	คำตอบ	เฉลย / ความรู้
1	Device controller จะบอกว่ามันทำงานเสร็จแล้วโดยใช้	**B** Interrupt	B. Interrupt
2	โปรแกรมที่ใช้จัดการ interrupt เรียกว่า	**C** Interrupt Service Routine	C. Interrupt Service Routine (ISR)
3	การค้นหาโปรแกรมจัดการ interrupt จะทำผ่าน	**B** Interrupt vector	B. Interrupt vector (ตารางเก็บแอดเดรสของ ISR)
4	ระบบที่เมื่อทำงานแล้วจะทำงานไปจนจบโดยไม่มีการติดต่อกับผู้ใช้คือ	**A** Batch System	A. Batch System
5	คุณสมบัติที่สำคัญของระบบปฏิบัติการแบบโต้ตอบคือการแบ่งเวลาการใช้งาน CPU	**จริง**	True (Time-sharing)
6	ในระบบ hybrid งานแบบ batch จะทำงานหลังฉาก	**จริง**	True (Background process)
7	ความสามารถที่ OS ทั้งแบบที่เป็น Multi-Program Batch และ Interactive ต้องมีร่วมกันก็คือ การเลือกโปรแกรมที่อยู่ในหน่วยความจำเข้ามาใช้งาน CPU	**จริง**	True (CPU Scheduling)
8	ในวิชานี้ Windows 11 เป็นตัวอย่างของระบบปฏิบัติการแบบ	**B** Single-user multi-tasking	B. Single-user multi-tasking
9	ในวิชานี้ การที่ OS ยอมให้สร้างบัญชีผู้ใช้ได้หลาย ๆ บัญชี จะถือว่ามันเป็น multi-user OS	**ไม่จริง**	False (Multi-user ต้องยอมให้ผู้ใช้หลายคนล็อกอินใช้งานระบบและทรัพยากรพร้อมกันได้จริง เช่น UNIX/Linux)
10	ระบบซึ่งต้องตอบสนองในเวลาที่กำหนด	**C** Real Time System	C. Real Time System
11	ระบบ Real Time ต้องการ OS ที่พัฒนาขึ้นมาโดยเฉพาะเท่านั้น	**ไม่จริง**	False (สามารถดัดแปลง General OS หรือใช้ RTOS ก็ได้)
12	ระบบที่มี CPU หลายตัวในเครื่องเดียวเรียกว่า	**E** Parallel System	E. Parallel System (Tightly-coupled system)
13	ระบบปฏิบัติการที่เราใช้ในปัจจุบันส่วนใหญ่รองรับ parallel system	**จริง**	True (รองรับ Multi-core processor)
14	Loosely coupled system	**D** Distributed System	D. Distributed System
15	ถ้าใช้ระบบปฏิบัติการแบบกระจาย (distributed OS) ทรัพยากรจะถูกควบคุมโดย OS ของแต่ละโหนดในเครือข่าย	**จริง**	True
16	ในวิชานี้ Windows 11 จัดเป็น Network OS	**จริง**	True (Windows ยุคปัจจุบันมีคุณสมบัติรองรับการทำงานด้านเน็ตเวิร์ก แชร์ไฟล์ แชร์เครื่องพิมพ์ในตัว)

L3. OS Structure — ชุดที่ 2.2 · 16 ข้อ

ข้อ	คำถาม	คำตอบ	เฉลย / ความรู้
1	ส่วนของ OS ที่จัดเตรียมส่วนติดต่อให้โปรแกรมที่เขียนขึ้นเรียกใช้เพื่อใช้บริการของ OS	**B** System Calls	B. System Calls
2	บริการของ OS ที่เกี่ยวกับการวางกฎเกณฑ์ และควบคุมให้เป็นไปตามกฎเกณฑ์ในการใช้งานทรัพยากรของระบบ	**C** Protection	C. Protection
3	บริการของ OS ที่เกี่ยวกับการตรวจสอบสิทธิการเข้าใช้งานระบบ	**D** Security	D. Security (เช่น การยืนยันตัวตนและการเข้าถึงจากภายนอก)
4	บริการด้านการสื่อสารระหว่างโปรแกรมที่ OS เตรียมไว้ให้	**C** message passing	C. message passing (IPC แบบหนึ่ง)
5	Command Line Interface (CLI) ไม่สามารถอยู่ใน kernel ได้	**ไม่จริง**	False (ในบางสถาปัตยกรรม CLI สามารถฝังอยู่ในส่วนของ kernel ได้)
6	`ls` เป็นตัวอย่างคำสั่งแบบ command built-in ของ Linux	**ไม่จริง**	False (`ls` เป็น external program อยู่ที่ `/bin/ls`)
7	`cd` เป็นตัวอย่างคำสั่งแบบ command built-in ของ Linux	**จริง**	True (`cd` ถูกฝังอยู่ใน shell โดยตรง)
8	คำสั่งที่เป็น command built-in จะเริ่มต้นทำงานได้เร็วกว่าคำสั่งที่เป็นโปรแกรมภายนอก	**จริง**	True (ไม่ต้องโหลดไฟล์ binary จาก disk เข้า memory)
9	คำสั่งที่เป็น command built-in จะแก้ไขได้ง่ายกว่าคำสั่งที่เป็นโปรแกรมภายนอก	**ไม่จริง**	False (คำสั่ง built-in ต้องแก้ที่ตัวเชลล์แล้วคอมไพล์ใหม่)
10	OS จัดการการเรียกใช้ system call คล้ายกับการจัดการ hardware interrupt	**จริง**	True (ใช้ Trap/Software Interrupt ผ่าน Vector Table เหมือนกัน)
11	การใช้ system call เราจะต้องเขียนโปรแกรมเรียกใช้ฟังก์ชันของ System Call (ไม่ว่าจะเป็นระดับ OS หรือ high level API) โดยตรงเท่านั้น	**ไม่จริง**	False (สามารถเรียกผ่าน Library ทั่วไป เช่น Standard C Library `printf()`, `fopen()` ซึ่งจะไปเรียก system call ต่ออีกทอดหนึ่ง)
12	การใช้ high-level API มีข้อดีคือโปรแกรมจะทำงานเร็วกว่าเรียกใช้ System call ระดับ OS	**ไม่จริง**	False (API มี overhead มากกว่า จึงไม่เร็วกว่าการเรียก native system call โดยตรง)
13	การจัดการ System Call จะเหมือนกับการจัดการ Interrupt	**จริง**	True
14	การเรียกใช้ System Call ก็คือการ interrupt ทางซอฟต์แวร์	**จริง**	True (เรียกว่า Software Interrupt หรือ Trap)
15	สถาปัตยกรรมของ kernel ที่ให้ประสิทธิภาพสูงสุดคือ	**B** Monolithic	B. Monolithic (บริการทุกอย่างอยู่ใน Address Space เดียวกัน ทำให้เรียกใช้งานได้เร็ว ไม่ต้องสลับ context บ่อย)
16	สถาปัตยกรรมของ kernel ที่น่าจะทำให้การปรับปรุง OS ทำได้ง่ายที่สุดคือ	**A** Microkernel	A. Microkernel (บริการส่วนใหญ่ถูกแยกเป็น User Service ทำให้แก้ไขเพิ่มเติมได้โดยไม่ต้องรื้อ Kernel)

L4. Introduction to Process — ชุดที่ 3 · 24 ข้อ

ข้อ	คำถาม	คำตอบ	เฉลย / ความรู้
1	จากนิยามของ OS ไฟล์ `paint.exe` คือ process	**ไม่จริง**	False (ไฟล์บนดิสก์คือ Program ส่วน Process คือโปรแกรมที่ถูกโหลดขึ้นมาทำงานในหน่วยความจำแล้ว)
2	การสร้าง process จากโปรแกรมจะเป็นการจับคู่แบบหนึ่งต่อหนึ่ง คือ หนึ่งโปรแกรมจะสร้างได้หนึ่ง process	**ไม่จริง**	False (หนึ่งโปรแกรมสามารถถูกเปิดใช้งานพร้อมกันได้หลาย process)
3	แต่ละ process จะมี data section ของตัวเอง	**จริง**	True
4	โพรเซสที่รอให้ OS มาเลือกมันไปใช้งาน CPU จะอยู่ในสถานะ	**B** ready	B. ready
5	สมมติโพรเซสอยู่ในสถานะ running ถ้ามันต้องการทำ I/O มันจะเปลี่ยนสถานะเป็น	**D** waiting	D. waiting
6	การเปลี่ยนสถานะของโพรเซสในข้อใดต่อไปนี้ไม่สามารถเกิดขึ้นได้	**D** waiting → running	D. waiting → running (เมื่อทำ I/O เสร็จ ต้องกลับไปต่อคิวที่ ready ก่อนเสมอ จะกระโดดไป running ทันทีไม่ได้)
7	ข้อมูลของโพรเซสแต่ละตัวจะถูกเก็บไว้ใน	**D** PCB	D. PCB (Process Control Block)
8	โพรเซสที่รอการใช้งานเครื่องพิมพ์จะอยู่ใน	**C** Device queue	C. Device queue
9	กระบวนการสลับโพรเซสเข้าใช้งาน CPU เรียกว่า	**C** Context Switch	C. Context Switch
10	การทำ context switch จะเกิดจากส่วนของ OS ที่เรียกว่า	**D** Dispatcher	D. Dispatcher (Dispatcher เป็นตัวสลับ Context จริง ส่วน Scheduler เป็นตัวเลือกว่าจะเอาโปรเซสไหน)
11	ส่วนของ OS ที่ใช้เลือกโพรเซสที่อยู่ใน ready queue เพื่อให้เข้าไปใช้งาน CPU คือ	**E** Scheduler	E. Scheduler
12	แต่ละ process จะสร้าง process ย่อยได้ไม่เกินสาม process	**ไม่จริง**	False (สร้างได้ตามที่ทรัพยากรระบบจะอำนวย)
13	สมมติใช้คำสั่งต่อไปนี้ในการคอมไพล์ source code ภาษา C ชื่อ main.c `gcc main.c -o main.exe` สมมติในโปรแกรมไม่มี error ใด ๆ จะได้โปรแกรมชื่อ	**C** main.exe	C. main.exe (แฟล็ก `-o` ระบุชื่อไฟล์ output)
14	หลังจากการเรียกใช้ ฟังก์ชัน `fork()` child process จะเริ่มทำงานจากตำแหน่งเริ่มต้นของ program (เช่น เริ่มทำงานตามคำสั่งแรกที่เขียนไว้ใน main program)	**ไม่จริง**	False (Child process จะเริ่มทำงานต่อจากคำสั่ง `fork()` ทันที ไม่ได้กลับไปเริ่มที่ต้นฟังก์ชัน `main`)
15	หลังจากการเรียกใช้ ฟังก์ชัน `fork()` การเปลี่ยนค่าตัวแปรของ child process จะไม่ทำให้ตัวแปรชื่อเดียวกันใน parent process เปลี่ยนค่าตามไปด้วย	**จริง**	True (ทั้งสองมี Address Space แยกจากกัน)
16	ในโค้ดโปรแกรมเราสามารถแยกส่วนของโค้ดที่ต้องการให้ parent process ทำงาน กับส่วนที่ต้องการให้ child process ทำงาน โดยดูจากค่าที่ฟังก์ชัน `fork()` ส่งกลับมา	**จริง**	True (Child ได้รับค่า 0, Parent ได้รับค่า PID ของลูก)
17	OS จะให้สิทธิ parent process สูงกว่า child process คือหลังจาก fork OS จะให้ parent process ทำงานก่อน child process	**ไม่จริง**	False (ลำดับการทำงานขึ้นอยู่กับ CPU Scheduler ตัวใดอาจทำงานก่อนก็ได้)
18	ฟังก์ชัน `fork()` จะ return ค่าลบให้กับ child process ในกรณีที่ `fork()` ทำงานผิดพลาด (สร้าง process ไม่ได้)	**ไม่จริง**	False (ถ้าเกิดข้อผิดพลาด จะไม่มี child process ถูกสร้างขึ้น และจะ return ค่าลบให้กับ parent process เท่านั้น)
19	ผลลัพธ์ที่ได้จากการทำงานของโปรแกรมที่มีการสร้าง process ย่อย อาจมีได้มากกว่าหนึ่งแบบ	**จริง**	True (ขึ้นอยู่กับการสลับคิวของ Scheduler)

20	```
	/* fork_example1.c */
	#include <stdio.h>
	#include <stdlib.h>
	#include <unistd.h>
	#include <sys/types.h>
	#include <sys/wait.h>
	
	int main() {
	    pid_t pid;
	    int i;
	    for (i = 0; i < 2; i++) {
	        pid = fork();
	        if (pid == 0) {
	            printf("child %d\n", i + 1);
	            exit(0);
	        }
	    }
	    while(wait(NULL) != -1);
	    printf("goodbye\n");
	    return 0;
	}
	``
	จากโปรแกรมดังรูป จะมีโพรเซสถูกสร้างขึ้นกี่โพรเซส (รวมทั้ง parent process ด้วย)
		C 3	C. 3 รอบ i=0 แม่สร้างลูก 1 ซึ่งลูกปริ้นท์แล้ว `exit(0)` ทันที, รอบ i=1 แม่สร้างลูก 2 ซึ่งลูกปริ้นท์แล้ว `exit(0)` เช่นกัน รวม Parent (1) + Child1 (1) + Child2 (1) = 3 โพรเซส

21	จากโปรแกรมในข้อ 20 ผลลัพธ์ที่เป็นไปได้คือ	**A.** - B. - C. - D. ``` child 1 goodbye child 2 ```	A (เนื่องจากมีคำสั่ง `while(wait(NULL) != -1);` ทำให้ Parent ต้องรอลูกทุกตัวทำงานเสร็จก่อน คำว่า "goodbye" จึงต้องอยู่บรรทัดสุดท้ายเสมอ)

22	```
	#include <stdio.h>
	#include <stdlib.h>
	#include <unistd.h>
	#include <sys/types.h>
	#include <sys/wait.h>
	
	int main() {
	    pid_t pid;
	    int i;
	    for (i = 0; i < 2; i++) {
	        pid = fork();
	        if (pid == 0) {
	            printf("child %d\n", i + 1);
	        }
	    }
	    while(wait(NULL) != -1);
	    printf("goodbye\n");
	    return 0;
	}
	``
	จากโปรแกรมดังรูป (ไม่มีคำสั่ง `exit(0)` ใน child) จะมีโพรเซสถูกสร้างขึ้นกี่โพรเซส (รวมทั้ง parent process ด้วย)
		D 4	D. 4 ($2^2 = 4$ โพรเซส เพราะลูกตัวแรกไม่ได้ exit จึงวนลูปไป fork ลูกตัวที่สองต่อด้วย)

23	จากโปรแกรมในข้อ 22 คำว่า "goodbye" จะถูกพิมพ์ออกมากี่ครั้ง	**D** 4	D. 4 (ทั้ง 4 โพรเซสหลุดออกจากลูปแล้วรันมาถึงบรรทัด `printf("goodbye\n");`)
24	จากโปรแกรมในข้อ 22 คำว่า "child 2" จะถูกพิมพ์ออกมากี่ครั้ง	**B** 2	B. 2 (ในลูป i=1 จะมีโพรเซสแม่ และลูกตัวแรก ทำการ fork ทำให้เกิดลูกใหม่ 2 ตัวที่มี `pid == 0` ในรอบ i=1 จึงพิมพ์ "child 2" ทั้งหมด 2 ครั้ง)

L5. Fork and Exec — ชุดที่ 4.1 · 19 ข้อ

ข้อ	คำถาม	คำตอบ	เฉลย / ความรู้
1	System call ที่ใช้ในการเรียกโปรแกรมอื่นขึ้นมาทำงานคือ	**A** execl	A. execl (กลุ่มฟังก์ชัน exec)
2	System call ที่ใช้ในการเรียกโปรแกรมอื่นขึ้นมาทำงาน ที่เราต้องระบุพารามิเตอร์ในรูปของอาเรย์คือ	**B** execv	B. execv (`v` หมายถึง Vector/Array)
3	System call ที่ Parent Process ใช้เพื่อรอ Child Process จบการทำงาน	**E** wait	E. wait
4	System call ที่ Process ใช้จบการทำงานและส่งค่ากลับให้ Parent Process	**C** exit	C. exit
5	คำสั่งบนระบบปฏิบัติการ UNIX ที่ใช้แสดงสถานะของโพรเซส	**A** ps	A. ps
6	ถ้าในโค้ดมีการเขียนคำสั่งนี้ `execlp("/bin/ls", "-l", NULL);` ผลลัพธ์ที่ได้ก็เหมือนกับการป้อนคำสั่ง ls -l	**ไม่จริง**	False (เนื่องจากการเรียก `execlp` พารามิเตอร์ตัวแรกใน list ควรเป็นชื่อโปรแกรม หรือ `argv[0]` ในที่นี้ส่ง `"-l"` ไปเป็น `argv[0]` ทำให้คำสั่งทำงานผิดรูปแบบ ไม่เทียบเท่ากับการสั่ง `ls -l`)
7	ถ้าในโค้ดมีการเขียนคำสั่งนี้ `execlp("/bin/ls", "ly", "-l", NULL);` จะทำให้การเรียกใช้ฟังก์ชัน execlp ทำงานไม่สำเร็จ นั่นคือจะไม่มีการเรียกใช้คำสั่ง ls	**ไม่จริง**	False (คำสั่ง `ls` จะยังคงถูกโหลดมาทำงานสำเร็จ เพียงแต่ค่า `argv[0]` จะมีค่าเป็นคำว่า `"ly"`)
8	ถ้าในโค้ดมีการเขียนคำสั่งนี้ `execl("ls", "ls", "-l", NULL);` ถ้าคำสั่ง ls ไม่อยู่ในโฟลเดอร์เดียวกับโปรแกรมที่มีคำสั่งนี้ คำสั่ง execl จะทำงานไม่สำเร็จ นั่นคือจะไม่มีการเรียกใช้คำสั่ง ls	**จริง**	True (ฟังก์ชัน `execl` ธรรมดา ไม่ค้นหาจากตัวแปรแวดล้อม `PATH` เหมือน `execlp` ดังนั้นถ้าไม่ระบุ full path และไฟล์ไม่อยู่ใน directory ปัจจุบันจะรันไม่สำเร็จ)
9	ถ้าต้องการเขียนโปรแกรมเพื่อเรียกใช้คำสั่ง cd เพื่อเปลี่ยนไปยังโฟลเดอร์ test ที่อยู่ภายใต้โฟลเดอร์ปัจจุบันที่โปรแกรมทำงานอยู่ เราจะเขียนคำสั่งดังนี้ `execlp("cd", "cd", "./test", NULL);`	**ไม่จริง**	False (คำสั่ง `cd` เป็น Shell built-in ไม่มีไฟล์ executable แยกต่างหากใน disk ให้เรียกผ่าน `execlp` ได้)

10	สมมติเขียน shell script ดังนี้
	`mkdir test`
	`cd test`
	จากนั้นจัดเก็บไว้ในไฟล์ชื่อที่เพิ่งสร้างขึ้น ถ้าเรียกใช้ shell script นี้ โดยใช้คำสั่ง
	`bash test.sh`
	สมมติโฟลเดอร์ test ยังไม่มีอยู่ในโฟลเดอร์ปัจจุบันที่เราเรียกใช้งาน shell script นี้ เมื่อโปรแกรมทำงานจบ โฟลเดอร์ปัจจุบันที่เราจะใช้งานอยู่คือโฟลเดอร์ test ที่เพิ่งสร้างขึ้น
		ไม่จริง	False (การรันด้วย `bash test.sh` จะทำงานใน Subshell เมื่อสคริปต์จบ Subshell จะปิดตัวลง โฟลเดอร์ปัจจุบันของ Shell ตัวหลักจะไม่เปลี่ยน)

11	จากข้อ 10 ถ้าเรียกใช้ shell script นี้ โดยใช้คำสั่ง `source test.sh` เมื่อโปรแกรมทำงานจบ โฟลเดอร์ปัจจุบันที่เราจะใช้งานอยู่คือโฟลเดอร์ test ที่เพิ่งสร้างขึ้น	**จริง**	True (คำสั่ง `source` หรือ `.` จะรันคำสั่งทั้งหมดบน Shell ปัจจุบันโดยตรง)

12	`int main(int argc, char *argv[]) {`
	`  // ...`
	`}`
	เมื่อเรียกใช้โปรแกรมด้วยคำสั่ง:
	`./app Ungink 1234 5678`
	ค่าของ `argc` คือ
		D 4	D. 4 (นับ `./app` , `Ungink` , `1234` , `5678`)

13	จากโปรแกรมในข้อ 12 ค่าของ `argv[1]` คือ	**H. "Ungink"** - I. "1234" - J. "5678"	H. "Ungink" (`argv[0]` คือ `"./app"` , `argv[1]` คือ `"Ungink"`)

14	`public static void main(String[] args) {`
	`    // ...`
	`}`
	เมื่อเรียกใช้โปรแกรมด้วยคำสั่ง:
	`java app Ungink 1234 5678`
	ค่าของ `args[1]` คือ
		I. "1234" - J. "5678"	I. "1234" (ใน Java `args[0]` คือ `"Ungink"`, `args[1]` คือ `"1234"`)

15	`wait(NULL);` หมายถึง parent process จะไม่หยุดรอการทำงานของ child process	**ไม่จริง**	False (`wait(NULL)` สั่งให้ Parent หยุดรอจนกว่าลูกคนใดคนหนึ่งจะจบการทำงาน)

16	`#include <stdio.h>`
	`#include <stdlib.h>`
	`#include <unistd.h>`
	`#include <sys/wait.h>`
	`#include <sys/types.h>`
	
	`int main() {`
	`    pid_t pid;`
	`    int i;`
	`    for (i = 0; i < 3; i++) {`
	`        pid = fork();`
	`        if (pid == 0) {`
	`            if (i == 0) {`
	`                execlp("echo", "echo", "good morning", NULL);`
	`                printf("hi\n");`
	`            }`
	`            else if (i == 1) {`
	`                execlp("echo", "echo", "good afternoon", NULL);`
	`                printf("hi\n");`
	`            }`
	`            else {`
	`                execlp("echo", "echo", "good night", NULL);`
	`                printf("hi\n");`
	`            }`
	`        }`
	`    }`
	`    while(wait(NULL) != -1);`
	`    printf("goodbye\n");`
	`    return 0;`
	`}`
	จากโปรแกรมภาษา C ดังรูป จะมีผลลัพธ์กี่บรรทัด (หมายเหตุ: คำสั่ง echo มีเวอร์ชันที่เป็น external program)
		D 4	D. 4 `execlp` รัน echo 3 รอบ (3 บรรทัด) และไม่มีลูกตัวไหนมาทำ `printf("hi\n")` เพราะ `exec` แทนที่โปรเซสเดิมไปแล้ว จากนั้นแม่พิมพ์ `goodbye\n` อีก 1 บรรทัด รวมเป็น 4 บรรทัด

17	`#include <stdio.h>`
	`#include <stdlib.h>`
	`#include <unistd.h>`
	`#include <sys/wait.h>`
	`#include <sys/types.h>`
	
	`int main() {`
	`    pid_t pid;`
	`    int i;`
	`    for (i = 0; i < 3; i++) {`
	`        pid = fork();`
	`        if (pid == 0) {`
	`            if (i == 0) {`
	`                execlp("cd", "cd", "test", NULL);`
	`                printf("hi\n");`
	`            }`
	`            else if (i == 1) {`
	`                execlp("echo", "echo", "good afternoon", NULL);`
	`                printf("hi\n");`
	`            }`
	`            else {`
	`                execlp("echo", "echo", "good night", NULL);`
	`                printf("hi\n");`
	`            }`
	`        }`
	`    }`
	`    while(wait(NULL) != -1);`
	`    printf("goodbye\n");`
	`    return 0;`
	`}`
	จากโปรแกรมภาษา C ดังรูป จะมีผลลัพธ์กี่บรรทัด (หมายเหตุ: คำสั่ง cd เป็น shell built in คำสั่ง echo มีเวอร์ชันที่เป็น external program)
		D 7	D. 7 เมื่อ i=0 ลูกคนที่ 1 เรียก `execlp("cd", ...)` ซึ่งล้มเหลว (เพราะ cd เป็น built-in) ทำให้ทำงานต่อแล้วพิมพ์ `hi\n` (1) และลูกคนนี้ไม่ได้ exit จึงวนลูปไป fork ลูกต่ออีก 2 ตัว (พิมพ์ good afternoon, good night) แล้วตัวมันเองพิมพ์ `goodbye\n` ด...

18	จากโปรแกรมภาษา C ในข้อ 17 คำว่า "goodbye" จะถูกพิมพ์ออกมากี่บรรทัด	**B** 2	B. 2 (พิมพ์จากโพรเซสแม่หลัก 1 ครั้ง และพิมพ์จากลูกตัวแรกที่รอดมาจากการ exec ล้มเหลวอีก 1 ครั้ง)

19	จากโปรแกรมภาษา C ในข้อ 16 ผลลัพธ์ที่เป็นไปได้คือ	**A.** - B. - C. - D.
		`goodbye`
		`good morning`
		`good afternoon`
		`good night`
			A (เพราะ `while(wait(NULL) != -1);` บังคับให้แม่รอจนลูกๆ ทุกตัวทำงานเสร็จหมดก่อน "goodbye" จึงต้องอยู่ท้ายสุด)

L6. Thread — ชุดที่ 5.1 · 12 ข้อ

ข้อ	คำถาม	คำตอบ	เฉลย / ความรู้
1	จุดประสงค์ของการเขียนโปรแกรมแบบ multithread ก็คือต้องการให้มีส่วนของโปรแกรมหลาย ๆ ส่วนทำงานขนานกันไป	**จริง**	True
2	ข้อเสียของการสร้าง Thread คือมันใช้ทรัพยากรมากกว่าการสร้าง process ย่อย	**ไม่จริง**	False (Thread ใช้ทรัพยากรน้อยกว่าการสร้าง Process มาก จึงเรียกว่า Lightweight Process)
3	Thread แต่ละตัวจะใช้ program counter ร่วมกัน	**ไม่จริง**	False (Thread แต่ละตัวต้องมี Program Counter และ Register เซตของตัวเอง)
4	Thread แต่ละตัวจะใช้ stack ร่วมกัน	**ไม่จริง**	False (Thread แต่ละตัวต้องมี Stack ส่วนตัวสำหรับเก็บ Local variables และ Function call)
5	Thread แต่ละตัวจะใช้ data section จากโพรเซสที่สร้างพวกมันขึ้นมาร่วมกัน	**จริง**	True (แชร์ Code, Data และ Files ร่วมกัน)
6	Thread ที่ถูกสร้างจาก process เดียวกันไม่จำเป็นต้องมีสิทธิเท่ากัน	**ไม่จริง**	False (Thread ทุกตัวในโปรเซสเดียวกันถือครองสิทธิการเข้าถึงของโปรเซสนั้นเท่ากันทั้งหมด)
7	JVM เป็นระบบปฏิบัติการแบบใด	**B** Single-process multi-threaded	B. Single-process multi-threaded
8	Windows 10 เป็นระบบปฏิบัติการแบบใด	**D** multi-process multi-threaded	D. multi-process multi-threaded
9	การจับคู่ระหว่าง user-thread และ kernel-thread แบบ many-to-one จะมีปัญหาด้านการทำงานพร้อมกันของ user thread ที่ถูกสร้างขึ้น	**จริง**	True (ถ้าเธรดใดเธรดหนึ่งเรียกบล็อกกิ้งซิสเต็มคอลล์ เธรดอื่นทั้งหมดในโปรเซสนั้นจะถูกบล็อกไปด้วย)
10	Thread แต่ละ Thread ที่ถูกสร้างขึ้นไม่จำเป็นต้องมีที่เก็บข้อมูลของตัวเอง เพราะมันจะใช้ข้อมูลจาก PCB	**ไม่จริง**	False (แต่ละเธรดต้องมี Thread Control Block: TCB สำหรับเก็บค่า State, Registers, Stack pointer)
11	การจับคู่ user-level กับ kernel-level threads ที่ใช้ทรัพยากรมากที่สุดคือ	**C** One-to-One	C. One-to-One (สร้าง 1 user thread ต้องสร้าง 1 kernel thread มารองรับ ทำให้เกิด overhead สูงสุด)
12	การจับคู่ user-level กับ kernel-level threads ที่ user-thread จะไม่ถูกบล็อกจากการทำงานของ user-thread อื่นอย่างแน่นอนคือ	**C** One-to-One	C. One-to-One

L7. Thread Programming Concepts — ชุดที่ 5.2 · 22 ข้อ

ข้อ	คำถาม	คำตอบ	เฉลย / ความรู้
1	ในการเขียนโปรแกรมภาษา C เพื่อใช้ library Pthread ชื่อเมท็อดที่จะให้ thread ไปทำงานต้องชื่อว่า runner	ไม่จริง	False (ตั้งชื่อฟังก์ชันเป็นชื่ออะไรก็ได้ ขอเพียง signature ตรงตามที่กำหนดคือ `void func(void param)`)
2	ฟังก์ชันของ Pthread library ที่ทำหน้าที่คล้าย ๆ ฟังก์ชัน fork ของการสร้าง process คือ	**B** pthread_create()	B. pthread_create()
3	ฟังก์ชันของ Pthread library ที่ทำหน้าที่คล้าย ๆ ฟังก์ชัน wait ของการสร้าง process คือ	**C** pthread_join()	C. pthread_join()
4	การเขียนประโยคต่อไปนี้ `pthread_exit(5);` คือการที่ thread ลูก ส่งค่า 5 กลับไปให้ thread แม่	ไม่จริง	False (พารามิเตอร์ของ `pthread_exit` รับพอยน์เตอร์ `void *retval` การส่งค่า integer 5 ดิบๆ โดยไม่ cast pointer ทำให้ผิดไวยากรณ์)
5	ถ้าเขียนโปรแกรมภาษา C ชื่อ test.c ที่มีการเรียกใช้ Pthread library ในการคอมไพล์และลิงก์เพื่อให้ได้โปรแกรมชื่อ test จะใช้คำสั่งใดต่อไปนี้	**D** gcc -o test test.c -lpthread	D. gcc -o test test.c -lpthread
6	การที่จะทำให้คลาสที่สร้างขึ้นมีคุณสมบัติของ thread นักเขียนโปรแกรมภาษา Java จะสร้างคลาสโดยสืบทอดคุณสมบัติมาจากคลาส	**B** Thread	B. Thread (`extends Thread`)
7	การที่จะทำให้คลาสที่สร้างขึ้นมีคุณสมบัติของ thread นักเขียนโปรแกรมภาษา Java จะอิมพลีเมนต์ interface ชื่อ	**A** Runnable	A. Runnable (`implements Runnable`)
8	ส่วนของโปรแกรมที่เปรียบเสมือน main โปรแกรมสำหรับ thread ที่สร้างขึ้นในภาษา Java จะถูกเขียนไว้ใน method ชื่ออะไร	**A** run	A. run (`public void run()`)
9	เมท็อดสำหรับ Java thread ที่ทำหน้าที่คล้าย ๆ wait() ของการสร้าง process คือ	**D** join	D. join
10	ใน Java การสร้าง object ของคลาสที่สืบทอดคุณสมบัติของคลาส Thread ขึ้นมา เท่ากับว่า thread เริ่มต้นทำงานแล้ว	ไม่จริง	False (ต้องเรียก `.start()` ก่อน เธรดจึงจะเริ่มทำงาน)
11	สมมติ Foo เป็นคลาสที่อิมพลีเมนต์ runnable interface และเราสร้าง object ของคลาส Foo ชื่อ obj ถ้าต้องการให้ thread อยู่ในสถานะ runnable เราจะใช้คำสั่งต่อไปนี้ `obj.start();`	ไม่จริง	False (Interface `Runnable` ไม่มีเมท็อด `start()` ต้องส่งไปให้คลาส Thread เช่น `new Thread(obj).start();`)
12	เมื่อเรียกเมท็อด start() ของ object ที่เป็น object ของคลาส Thread ในภาษา Java, thread จะอยู่ในสถานะ running	ไม่จริง	False (จะเข้าสู่สถานะ Runnable เพื่อรอ Scheduler เลือกไปรัน)
13	การที่ thread ที่ได้รับคำสั่งให้จบการทำงาน แล้วจบการทำงานเลย เราจะเรียกการอิมพลีเมนต์ thread ในแบบนี้ว่า	**A** Asynchronous Cancellation	A. Asynchronous Cancellation
14	Java thread รองรับการสั่งจบการทำงานของ thread ในรูปแบบที่เรียกว่า (ให้ตอบรูปแบบที่ใกล้เคียงที่สุด)	**D** Cooperative Mechanism	D. Cooperative Mechanism (การใช้ Flag หรือ `Thread.interrupt()` ตรวจสอบสถานะร่วมกัน)
15	Pthread รองรับเฉพาะ deferred cancellation	ไม่จริง	False (Pthread รองรับทั้ง Asynchronous และ Deferred Cancellation)
16	คำสั่ง sleep() จัดเป็น cancellation point สำหรับ Pthread	จริง	True
17	คำสั่ง printf() จัดเป็น cancellation point สำหรับ Pthread	ไม่จริง	False
18	แนวคิดการสร้าง Thread ไว้ล่วงหน้า เพื่อที่เมื่อต้องการใช้จะสามารถนำมาใช้ได้เลย โดยไม่ต้องเสียเวลาไปสร้างขึ้นมาใหม่ แนวคิดนี้มีชื่อเรียกว่า	**B** Thread Pool	B. Thread Pool
19	ถ้าต้องการสร้าง Thread pool ใน Java แบบไม่จำกัดขนาดไว้ล่วงหน้า เราจะใช้เมท็อดใดจากคลาส Executors	**A** newCachedThreadPool()	A. newCachedThreadPool()
20	จำนวน thread ใน Java thread pool ที่สร้างได้สูงสุดคือ 10 thread	ไม่จริง	False (สามารถกำหนดขนาดได้ตามความต้องการ หรือยืดหยุ่นได้ไม่จำกัดใน Cached Thread Pool)

21	```
	#include <stdio.h>
	#include <stdlib.h>
	#include <pthread.h>
	
	int sum = 0;
	
	void *runner(void *param) {
	    int i, upper = atoi(param);
	    sum = 0;
	    for (i = 1; i <= upper; i++)
	        sum += i;
	    printf("I am child thread, I am
	calculating\n");
	    pthread_exit(0);
	}
	
	int main(int argc, char *argv[]) {
	    pthread_t tid;
	    pthread_attr_t attr;
	    pthread_attr_init(&attr);
	    pthread_create(&tid, &attr, runner, argv[1]);
	    printf("I am mother thread, I will wait for my
	child thread\n");
	    pthread_join(tid, NULL);
	    printf("from my child sum = %d\n", sum);
	    return 0;
	}
	``
	จากโปรแกรมภาษา C ต่อไปนี้ ถ้าผู้ใช้ run โปรแกรมนี้โดยใช้คำสั่ง `./pthread 5` ผลลัพธ์ที่เป็นไปได้คือ
		C
		``
		I am child thread, I
		am calculating
		I am mother thread,
		I will wait for my
		child thread
		from my child sum =
		15
		``
			C (ผลรวม 1+2+3+4+5 = 15 และลำดับข้อความของ Child หรือ Mother จะขึ้นก่อนก็ได้ แต่ `sum = 15` ต้องอยู่หลัง `pthread_join()`)

22	```
	class SharedData {
	    private int sum;
	    public int getSum() {
	        return sum;
	    }
	    public void setCalculatesum(int num) {
	        sum = num * (num + 1) / 2;
	    }
	}
	
	class Child extends Thread {
	    private SharedData data;
	    int num;
	    public Child(SharedData data, int num) {
	        this.data = data;
	        this.num = num;
	    }
	    public void run() {
	        System.out.print("I am calculating ");
	        data.setCalculatesum(num);
	    }
	}
	
	public class Main {
	    public static void main(String[] args) {
	        int num = Integer.parseInt(args[0]);
	        SharedData data = new SharedData();
	        Child child = new Child(data, num);
	        child.start();
	        try {
	            child.join();
	        } catch (InterruptedException ie) {}
	        System.out.print(data.getSum() + " ");
	        System.out.println("Goodbye");
	    }
	}
	``
	จากโปรแกรมภาษา Java ต่อไปนี้ ถ้าผู้ใช้รันโปรแกรมโดยใช้คำสั่ง `java Main 6` ผลลัพธ์ที่เป็นไปได้คือ
		C I am calculating 21 Goodbye	C. I am calculating 21 Goodbye ((6 × 7)/(2) = 21)

L8. CPU Scheduling — ชุดที่ 6 · 31 ข้อ

ข้อ	คำถาม	คำตอบ	เฉลย / ความรู้
1	ส่วนของ OS ที่เรียกว่า Scheduler มีหน้าที่ในการทำ context switch	**ไม่จริง**	False (Dispatcher เป็นตัวทำ Context switch ส่วน Scheduler ทำหน้าที่เลือกโปรเซส)
2	ถ้า OS ไม่สามารถขัดจังหวะการทำงานของโพรเซสได้หลังจากที่ให้โพรเซสได้ใช้งาน CPU Scheduling algorithm ที่ใช้จะเป็นแบบ nonpreemptive	**จริง**	True
3	การเลือกโพรเซสเข้าใช้งาน CPU แบบ round robin จะให้ response time ที่ดี	**จริง**	True
4	waiting time คือเวลาที่โพรเซสรออยู่ใน waiting state	**ไม่จริง**	False (Waiting time หมายถึงเวลาที่โปรเซสต้องรออยู่ใน **Ready Queue**)
5	สำหรับ multi-level feed back queue โพรเซสจะเลื่อนจากคิวที่มีสถานะสูงกว่าลงสู่คิวที่มีสถานะต่ำกว่าเสมอ	**ไม่จริง**	False (สามารถเลื่อนขึ้นไปยังคิวที่มีลำดับสูงกว่าได้ เช่น กลไก Aging)
6	aging ช่วยแก้ปัญหาการอดตาย (starvation) ของโพรเซส	**จริง**	True (เพิ่ม Priority ให้โปรเซสที่รอนานขึ้นเรื่อยๆ)
7	ถ้า waiting time มากจะทำให้ turnaround time มีค่ามากตามไปด้วย	**จริง**	True (Turnaround Time = Burst Time + Waiting Time)
8	FCFS เป็น algorithm แบบ preemptive	**ไม่จริง**	False (FCFS เป็น Non-preemptive)
9	ถ้าการเลือกโพรเซสเข้าใช้งาน CPU ทำให้ response time ดีแล้ว จะทำให้ทุกโพรเซสมี turnaround time ดีขึ้นด้วย	**ไม่จริง**	False (เช่น Round Robin ให้ response time ดีมาก แต่บ่อยครั้ง turnaround time เฉลี่ยกลับแย่กว่า SJF)
10	SRTF เป็น nonpreemptive algorithm	**ไม่จริง**	False (SRTF คือ Preemptive SJF)

📊 **ข้อมูลสำหรับข้อ 11 - 16**

Process	Arrival Time	Burst Time
P1	0	8
P2	2	4
P3	1	1

อัลกอริทึม: Non-preemptive SJF

Gantt Chart:

- t=0: P1 เริ่มรันยาวจนจบที่ t=8 (เพราะเป็น Non-preemptive)
- t=8: ในคิวมี P2 (burst 4) และ P3 (burst 1) → P3 สั้นกว่า จึงรัน t=8 ถึง t=9
- t=9: P2 รันต่อจนจบที่ t=13

สรุปค่าเวลา:

- P1: เสร็จ 8, Turnaround = 8-0 = 8, Waiting = 8-8 = 0
- P2: เสร็จ 13, Turnaround = 13-2 = 11, Waiting = 11-4 = 7
- P3: เสร็จ 9, Turnaround = 9-1 = 8, Waiting = 8-1 = 7

11	turnaround time ของ P1 คือ	**8**	
12	turnaround time ของ P2 คือ	**11**	
13	turnaround time ของ P3 คือ	**8**	
14	waiting time ของ P1 คือ	**0**	
15	waiting time ของ P2 คือ	**7**	
16	waiting time ของ P3 คือ	**7**	

📊 **ข้อมูลสำหรับข้อ 17 - 22**

(ใช้ตารางเดียวกับข้อ 11 - 16) **อัลกอริทึม:** Preemptive SJF (SRTF)

Gantt Chart:

- t=0: P1 รัน (เหลือ 8)
- t=1: P3 เข้ามา (burst 1), P1 เหลือ 7 → P3 แย่ง CPU รัน t=1 ถึง t=2 (P3 จบ)
- t=2: P2 เข้ามา (burst 4), P1 เหลือ 7 → P2 สั้นกว่า จึงรัน t=2 ถึง t=6 (P2 จบ)
- t=6: P1 รันต่อที่เหลือ 7 หน่วย จนจบที่ t=13

สรุปค่าเวลา:

- P1: เสร็จ 13, Turnaround = 13-0 = 13, Waiting = 13-8 = 5
- P2: เสร็จ 6, Turnaround = 6-2 = 4, Waiting = 4-4 = 0
- P3: เสร็จ 2, Turnaround = 2-1 = 1, Waiting = 1-1 = 0

17	turnaround time ของ P1 คือ	**13**	
18	turnaround time ของ P2 คือ	**4**	
19	turnaround time ของ P3 คือ	**1**	
20	waiting time ของ P1 คือ	**5**	
21	waiting time ของ P2 คือ	**0**	
22	waiting time ของ P3 คือ	**0**	

📊 **ข้อมูลสำหรับข้อ 23 - 25**

Process	Arrival Time	Burst Time
P1	0	8
P2	2	5
P3	1	3

อัลกอริทึม: Round Robin (RR), Time Quantum = 2

Gantt Chart (q=2):

- t=0..2: P1 รัน (เหลือ 6) → ณ t=2 คิวคือ [P3, P2, P1]
- t=2..4: P3 รัน (เหลือ 1) → คิวคือ [P2, P1, P3]
- t=4..6: P2 รัน (เหลือ 3) → คิวคือ [P1, P3, P2]
- t=6..8: P1 รัน (เหลือ 4) → คิวคือ [P3, P2, P1]
- t=8..9: P3 รัน (เหลือ 0) → **P3 จบที่ t=9**
- t=9..11: P2 รัน (เหลือ 1) → คิวคือ [P1, P2]
- t=11..13: P1 รัน (เหลือ 2) → คิวคือ [P2, P1]
- t=13..14: P2 รัน (เหลือ 0) → **P2 จบที่ t=14**
- t=14..16: P1 รัน (เหลือ 0) → **P1 จบที่ t=16**

23	turnaround time ของ P1 คือ	**16**	16 - 0 = 16
24	turnaround time ของ P2 คือ	**12**	14 - 2 = 12
25	turnaround time ของ P3 คือ	**8**	9 - 1 = 8
26	<table><tr><th>Process</th><th>Arrival Time</th><th>Burst Time</th></tr><tr><td>P1</td><td>0</td><td>8</td></tr><tr><td>P2</td><td>2</td><td>4</td></tr><tr><td>P3</td><td>1</td><td>1</td></tr></table> turnaround time ของ P2 จะเป็นเท่าไร ถ้า Scheduling Algorithm ที่ใช้คือ RR ที่มี time quantum เป็น 4	**H. 7**	7 (P1 รัน t=0..4, จากนั้น P3 รัน t=4..5, P2 รัน t=5..9 ทำเสร็จที่ t=9; Turnaround = 9 - 2 = 7)
27	สำหรับ multilevel queue โพรเซสที่อยู่ใน ready queue ใด ready queue หนึ่งจะอยู่ใน queue นั้นโดยไม่เปลี่ยนไปอยู่ที่ ready queue อื่น	**จริง**	True (Multilevel Queue ธรรมดาจะย้ายคิวไม่ได้ ต่างจาก Multilevel Feedback Queue ที่ย้ายได้)
28	ถ้ากำหนด time quantum น้อย ก็มีโอกาสที่จะเกิด context switch มากขึ้น	**จริง**	True
29	ถ้าใช้ multilevel feedback queue ที่มี 3 คิว และแต่ละคิวเป็นดังนี้ - q0 ใช้ Round Robin (RR) มี time quantum 8 - q1 ใช้ Round Robin (RR) มี time quantum 16 - q2 ใช้ FCFS โดยทุก process จะถูกส่งเข้า q0 ก่อน โดยมีเงื่อนไขว่าแต่ละ process จะได...	**B** q1	B. q1 (ใน q0 ทำได้ 2 รอบ รอบละ 8 หน่วย = 16 หน่วย งานยังเหลืออีก 4 หน่วย จึงตกมาที่ q1 ซึ่งมี quantum 16 หน่วย จึงทำงานเสร็จสิ้นที่ q1)
30	ถ้ามี thread ที่มีสิทธิสูงกว่าถูกส่งเข้ามาในระบบ JVM อาจหยุดการทำงานของ thread ที่ทำงานอยู่ และให้ thread ดังกล่าวเข้าทำงานแทน	**จริง**	True (Preemptive Priority-based Scheduling)
31	ถ้าต้องการให้ Java thread เปิดทางให้ thread อื่นได้ทำงาน เมท็อดที่ต้องเรียกใช้คือ	**D** yield()	D. yield()

L9. Process Synchronization — ชุดที่ 9.1 · 18 ข้อ

ข้อ	คำถาม	คำตอบ	เฉลย / ความรู้
1	การที่มีโพรเซสมากกว่าหนึ่งโพรเซสไปใช้แก้ข้อมูลที่ใช้งานร่วมกันโดยไม่มีการจัดจังหวะให้เหมาะสม ซึ่งอาจจะทำให้ข้อมูลผิดได้เรียกว่า	**C** Race Condition	C. Race Condition
2	ถ้าค่าที่อยู่ในตัวแปร banana มีค่าเป็น 7 และมีโพรเซสสองโพรเซสเข้ามาแก้ไขค่าตัวแปรตัวนี้ โดยโพรเซส A เพิ่มค่า banana ขึ้นไปอีก 2 และ process B ลดค่าของ banana ลง 1 ถ้าโพรเซส A และโพรเซส B ได้ทำงานโพรเซสละ 1 ครั้ง (ใครทำก่อนก็ได้) ถ้ามีการจัด...	**C** 8	C. 8 (7 + 2 - 1 = 8)
3	จากเงื่อนไขในข้อ 2 ถ้าไม่มีการจัดจังหวะอย่างเหมาะสม ค่าของตัวแปร banana จะมีค่าอะไรหลังจากโพรเซสทั้งสองจบการทำงาน	**F** 6 หรือ 8 หรือ 9	F. 6 หรือ 8 หรือ 9 (เกิด Race condition ขึ้นอยู่กับว่าตัวใดเขียนทับตัวใด หรือผลลัพธ์ปกติ)
4	แนวคิดหลักของการจัดจังหวะระหว่างโพรเซสในการเข้าใช้ทรัพยากรที่สามารถใช้งานร่วมกันได้คือ การล็อกทรัพยากรไม่ให้โพรเซสอื่นใช้ในขณะที่ตัวโพรเซสกำลังใช้งานทรัพยากรอยู่	**จริง**	True
5	ส่วนของโปรแกรมที่โพรเซสที่ถือครองทรัพยากรต้องทำงานให้เสร็จสิ้นก่อนที่จะปล่อยทรัพยากรให้โพรเซสอื่นใช้เรียกว่า	**D** Critical Section	D. Critical Section
6	ถ้าจะคิดขั้นตอนวิธีเพื่อแก้ไขเรื่อง Synchronization เพื่อให้สมบูรณ์จะต้องผ่านเงื่อนไขสามข้อ แต่ถ้าผ่านสามข้อไม่ได้ ข้อที่ต้องทำให้ผ่านให้ได้คือ	**C** Mutual Exclusion	C. Mutual Exclusion (เป็นกฎเหล็กสำคัญที่สุดว่าห้ามเข้าใช้งานพร้อมกันเด็ดขาด)
7	เงื่อนไขที่ต้องรับประกันว่า การเลือกโพรเซสจะเลือกจากโพรเซสที่ต้องการเข้าใช้งานเท่านั้นคือ	**B** Progress	B. Progress
8	เงื่อนไขที่ต้องรับประกันว่า ในขณะเวลาหนึ่งจะมีเพียง process เดียว ที่ได้ทำงานใน critical section คือ	**C** Mutual Exclusion	C. Mutual Exclusion
9	เงื่อนไขที่ต้องรับประกันว่า โพรเซสที่รอการใช้งานอยู่จะไม่รอไปเรื่อย ๆ โดยไม่มีจุดสิ้นสุดคือ	**A** Bounded Waiting	A. Bounded Waiting
10	ถ้าไม่ผ่าน progress แล้ว ก็จะไม่ผ่าน Bounded Waiting	**จริง**	True
11	กลไกการแก้ปัญหา Synchronization คือเมื่อ process ทำงานอยู่ในส่วนของ critical section แล้วมันจะไม่ถูกขัดจังหวะ	**ไม่จริง**	False (โพรเซสยังคงถูกสลับ Context หรือขัดจังหวะได้ เพียงแต่โพรเซสอื่นจะไม่สามารถเข้าสู่ Critical Section ของตนได้)
12	Bakery Algorithm ยอมให้มี process มากกว่าหนึ่ง process ที่มีหมายเลขคิวเดียวกันได้	**จริง**	True (กรณีหมายเลขคิวซ้ำกัน จะตัดสินด้วย Process ID ที่น้อยกว่า)
13	ลำดับหมายเลขคิวใน Bakery Algorithm รองรับ bounded waiting	**จริง**	True (เป็นระบบ First-Come, First-Served ตามเบอร์คิว)
14	จากขั้นตอนวิธีปัญหา bounded buffer ที่เป็นการพยายามจัดจังหวะของนักเขียนโปรแกรมเองตามที่เราเรียนกัน ตัวแปรที่เป็นตัวแปรที่ใช้ร่วมกันระหว่าง producer และ consumer คือ	**E** count และ buffer	E. count และ buffer (`in` เป็นของ Producer, `out` เป็นของ Consumer)
15	ตัวแปรที่ producer ใช้ในการอ้างถึงช่อง ใน buffer ที่มันจะเขียนข้อมูลคือ	**A** in	A. in
16	ในการแก้ปัญหา Bounded buffer แบบที่จัดจังหวะของนักเขียนโปรแกรมเองตามที่เราเรียนกัน เงื่อนไขหนึ่งในการทำงานให้สำเร็จคือ ฝั่ง consumer จะต้องอ่านข้อมูลให้เสร็จจนจบกระบวนการถึงการลดตัวนับจำนวนข้อมูลใน buffer แต่ producer สามารถถูกขัดจังหวะได้โ...	**ไม่จริง**	False (ทั้งคู่ต้องทำงานจนครบกระบวนการและไม่ถูกขัดจังหวะในจุดวิกฤต มิเช่นนั้นค่า `count` จะเพี้ยน)
17	ข้อจำกัดของ Peterson's algorithm คือมันรองรับแค่ Mutual Exclusion และ Progress แต่ไม่รองรับ Bounded Waiting	**ไม่จริง**	False (Peterson's algorithm ผ่านครบทั้ง 3 ข้อ รวมถึง Bounded Waiting)
18	ข้อจำกัดของ Bakery algorithm คือมันรองรับแค่ Mutual Exclusion และ Progress แต่ไม่รองรับ Bounded Waiting	**ไม่จริง**	False (Bakery algorithm ผ่านครบทั้ง 3 ข้อสำหรับกรณี N โพรเซส)

L10. Semaphore (Concept & Pthread) — ชุดที่ 9.2 · 25 ข้อ

ข้อ	คำถาม	คำตอบ	เฉลย / ความรู้
1	การจัดจังหวะที่ให้โพรเซสติดลูปรอจนกว่าเงื่อนไขเป็นเท็จจึงจะเข้าทำงานได้ เรียกว่าการจัดจังหวะที่มี busy waiting	**จริง**	True (หรือเรียกว่า Spinlock)
2	Semaphore เป็นกลไกในการจัดจังหวะที่ไม่มี busy waiting	**จริง**	True (ในการอิมพลีเมนต์แบบมาตรฐานคลาสสิกของ OS โพรเซสที่รอจะถูกจับเข้า Sleep/Wait queue โดยไม่เปลือง CPU time)
3	Semaphore ที่ใช้กับปัญหา critical section คือ counting semaphore	**ไม่จริง**	False (ใช้ Binary Semaphore หรือ Mutex Lock)
4	Binary semaphore มีชื่อเรียกอีกอย่างว่า	**B** Mutex lock	B. Mutex lock

5	`Semaphore S;`
	`wait(S)`
	`Critical Section`
	`signal(S)`
	เพื่อที่จะใช้ semaphore ในการแก้ปัญหา critical section ต้องกำหนดค่าเริ่มต้นให้ S เป็น
		B 1	B. 1

6	เมื่อใช้ semaphore แบบ ไม่มี busy waiting โพรเซสซึ่งรอการเข้าใช้ critical section จะรออยู่ใน wait state	**จริง**	True (Waiting Queue)
7	การใช้ Semaphore รับประกันว่าจะผ่านเงื่อนไขสามข้อของการแก้ปัญหา Critical Section	**ไม่จริง**	False (การใช้ Semaphore อย่างไม่ระมัดระวังอาจเกิด Deadlock หรือ Starvation ทำให้ไม่ผ่าน Bounded Waiting)
8	ฟังก์ชันของ PThread Semaphore ที่ทำหน้าที่เหมือนคำสั่ง wait() ของ Semaphore ที่แสดงใน slide คือ	**C** sem_wait()	C. sem_wait()
9	ฟังก์ชันของ PThread Semaphore ที่ทำหน้าที่เหมือนคำสั่ง signal() ของ Semaphore คือ	**B** sem_post()	B. sem_post()
10	ฟังก์ชันของ PThread Semaphore ที่ทำหน้าที่กำหนดค่าเริ่มต้นให้ semaphore คือ	**A** sem_init()	A. sem_init()

11	`#include <stdio.h>`
	`#include <pthread.h>`
	`#include <semaphore.h>`
	
	`int count = 0;`
	`sem_t mutex;`
	
	`void *thread_func(void *param) {`
	`  int i;`
	`  for (i = 0; i < 5; i++) {`
	`  sem_wait(&mutex);`
	`  count++;`
	`  sem_post(&mutex);`
	`  }`
	`  pthread_exit(0);`
	`}`
	
	`int main() {`
	`  pthread_t tid;`
	`  sem_init(&mutex, 0, 1);`
	`  pthread_create(&tid, NULL, thread_func, NULL);`
	`  pthread_join(tid, NULL);`
	`  printf("%d\n", count);`
	`  return 0;`
	`}`
	ผลลัพธ์ที่ได้จากโปรแกรมคือ
		C 5	C. 5 (วนลูปบวกค่า 5 ครั้ง มี semaphore ป้องกันอย่างถูกต้อง)

12	การแก้ปัญหา bounded buffer ในวิชานี้ใช้ตัวแปร semaphore กี่ตัว	**C** 3	C. 3 (ประกอบด้วย `mutex`, `empty`, `full`)
13	การแก้ปัญหา bounded buffer ในวิชานี้ใช้ counting semaphore กี่ตัว	**B** 2	B. 2 (คือ `empty` และ `full` ส่วน `mutex` เป็น binary semaphore)
14	การแก้ปัญหา bounded buffer ในวิชานี้ semaphore ที่ producer ใช้เพื่อตรวจสอบว่า buffer ยังมีช่องว่างหรือไม่คือ	**A** empty	A. empty
15	การแก้ปัญหา bounded buffer ในวิชานี้ semaphore ที่ consumer ใช้เพื่อตรวจสอบว่ามีข้อมูลใหม่ใน buffer หรือไม่คือ	**B** full	B. full
16	การแก้ปัญหา bounded buffer ในวิชานี้ semaphore ที่ใช้เพื่อให้แน่ใจว่า ณ.ขณะเวลาใดเวลาหนึ่งจะมีแค่ producer หรือ consumer เท่านั้นที่ได้อ่านหรือเขียน buffer คือ	**C** mutex	C. mutex
17	ปัญหา Dining philosopher ในวิชานี้ นักปราชญ์แต่ละคนจะเป็นตัวแทนของ	**A** process	A. process

18	ปัญหา Dining philosopher ในวิชานี้ ตะเกียบจะเป็นตัวแทนของ	**B** resource	B. resource (ทรัพยากรที่มีจำนวนจำกัด)
19	ปัญหา Dining philosopher ในวิชานี้ semaphore จะใช้เพื่อจัดสรรตะเกียบให้นักปราชญ์ เพื่อให้แน่ใจว่านักปราชญ์แต่ละคนจะได้กินข้าว	**ไม่จริง**	False (Semaphore ป้องกันแค่การแย่งตะเกียบอันเดียวกัน แต่ไม่ได้รับประกันว่าทุกคนจะได้กิน อาจเกิด Deadlock หรือ Starvation ได้)
20	การใช้ semaphore กับปัญหา dining philosopher โดยใช้ขั้นตอนวิธีในวิชานี้ อาจทำให้เกิดสิ่งที่เรียกว่า	**A** deadlock	A. deadlock (เมื่อนักปราชญ์ทุกคนหยิบตะเกียบข้างขวาพร้อมกัน)
21	การแก้ปัญหา readers-writers ในวิชานี้ใช้ semaphore กี่ตัว	**B** 2	B. 2 (คือ `mutex` สำหรับคุม `readcount` และ `wrt` สำหรับล็อกไฟล์ข้อมูล)
22	การแก้ปัญหา readers-writers ในวิชานี้ semaphore ที่ใช้จัดจังหวะระหว่าง readers และ writer คือ	**A** wrt	A. wrt
23	การแก้ปัญหา readers-writers ในวิชานี้ semaphore ที่ใช้เพื่อควบคุมการเพิ่มหรือลดจำนวน reader คือ	**B** mutex	B. mutex
24	การแก้ปัญหา reader-writers ในวิชานี้สามารถมี reader มากกว่าหนึ่งตัวอ่านข้อมูลได้พร้อม ๆ กัน	**จริง**	True (Shared Read)
25	การแก้ปัญหา reader-writers ในวิชานี้ reader คนแรกเป็นคนเริ่มการป้องกันไม่ให้ writer เข้ามาเขียนข้อมูลในขณะที่ยังมี readers อ่านข้อมูลอยู่ และเป็นคนที่ต้องปลดล็อกให้ writer เขียนข้อมูลได้ เมื่อไม่มี readers ที่ต้องการอ่านข้อมูลแล้ว	**ไม่จริง**	False (Reader คนแรกเป็นคนเริ่มล็อก `wrt` ก็จริง แต่คนที่เป็นคนปลดล็อกให้ Writer คือ **Reader คนสุดท้าย** ที่อ่านเสร็จ ไม่ใช่คนแรก) # รวมโจทย์ คำถาม และเฉลยวิชาระบบปฏิบัติการ (Operating Systems) ชุดที่ 10 - 14

L11. Monitor, Java Sync, Java Semaphore — ชุดที่ 10 · 34 ข้อ

ข้อ	คำถาม	คำตอบ	เฉลย / ความรู้
1	แนวคิดของ monitor คือ การรวมเอา shared data และ ฟังก์ชันที่ต้องการทำงานกับ shared data มาไว้ร่วมกัน และยอมให้มีทีละโพรเซสได้ทำงานใน monitor	**จริง**	True (Monitor รับประกัน Mutual Exclusion โดยอนุญาตให้มีเพียง 1 Thread/Process ทำงานอยู่ภายใน ณ ขณะใดขณะหนึ่ง)
2	Monitor เป็นกลไกที่มีระดับต่ำกว่า semaphore	**ไม่จริง**	False (Monitor เป็นกลไกระดับสูง High-level synchronization construct ที่คอมไพเลอร์ช่วยจัดการความปลอดภัย ต่างจาก Semaphore ที่เป็น Low-level)
3	ประเภทของตัวแปรที่ monitor เตรียมไว้ให้เรียกใช้ เมื่อ process ที่ทำงานอยู่ใน monitor ต้องการจะรอเงื่อนไขบางอย่างคือตัวแปรที่เรียกว่า	**A** condition	A. condition (ตัวแปรประเภท `condition` เช่น `condition x, y;`)
4	ฟังก์ชันที่ process ที่ทำงานอยู่ใน monitor จะเรียกใช้เพื่อรอเงื่อนไขบางอย่างคือ	**D** wait	D. wait (เช่น คำสั่ง `x.wait()`)
5	process ที่รอการเข้าใช้ monitor จะรออยู่ในคิวที่เรียกว่า	**B** entry	B. entry (Entry Queue หรือ Entry Set)
6	ฟังก์ชันที่ process ที่ทำงานอยู่ใน monitor จะปลุก process ที่รอเงื่อนไขอยู่คือ	**C** signal	C. signal (เช่น `x.signal()`)
7	Process ที่รอเงื่อนไขที่จะเกิดขึ้น จะรออยู่ในคิวแยกต่างหากจากคิวสำหรับ process ที่รอการเข้าใช้ monitor	**จริง**	True (Process ที่รอเงื่อนไขจะรออยู่ใน Condition Queue ของตัวแปรเงื่อนไขนั้นๆ ซึ่งแยกจาก Entry Queue ของตัว Monitor)
8	ถ้าจะเทียบกับ process state ตัว process ที่รอการเข้าใช้ monitor เปรียบได้กับ process ที่อยู่ใน ready state	**ไม่จริง**	False (โพรเซสที่รอเข้ามอนิเตอร์ถือว่าถูกบล็อก ไม่สามารถเข้าทำงานได้จนกว่ามอนิเตอร์จะว่าง)
9	ถ้าจะเทียบกับ process state ตัว process ที่รอเงื่อนไขที่จะเกิดขึ้นหลังจากได้เข้าทำใน monitor แล้ว เปรียบได้กับ process ที่อยู่ใน ready state	**ไม่จริง**	False (เปรียบได้กับ Waiting/Blocked state เนื่องจากต้องรอสัญญาณ signal จากโพรเซสอื่น)
10	ขั้นตอนวิธีในการแก้ปัญหา Dining Philosophers โดยใช้ monitor ในวิชานี้ ถ้านักปราชญ์หิวข้าว เขาจะเริ่มจากหยิบตะเกียบด้านซ้ายมือมาถือไว้	**ไม่จริง**	False (ในวิธี Monitor จะตรวจสอบว่าตะเกียบทั้งสองข้างว่างพร้อมกันหรือไม่ก่อน ถ้าว่างจึงจะหยิบตะเกียบทั้งสองข้างพร้อมกัน)
11	ขั้นตอนวิธีที่ใช้ monitor ในวิชานี้มาแก้ปัญหา dining philosopher อาจเกิดปัญหา	**C** starvation	C. starvation (แม้แก้ปัญหา Deadlock ได้ แต่นักปราชญ์บางคนอาจถูกเพื่อนข้างเคียงสลับกันหยิบตะเกียบตัดหน้าตลอดเวลาจนอดตาย)
12	กลไกที่คล้าย ๆ monitor ที่ Java เตรียมไว้ให้ใช้คือ	**C** synchronization	C. synchronization (คีย์เวิร์ด `synchronized`)
13	กลไกการทำ synchronization ของ Java คือจะมีเมท็อดชื่อ wait() และ notify() ให้เรียกใช้ในการล็อกออบเจกต์	**ไม่จริง**	False (การล็อกออบเจกต์เกิดขึ้นโดยอัตโนมัติจากการเข้าสู่ `synchronized method` หรือ `synchronized block` ส่วน `wait()` และ `notify()` ใช้สำหรับปล่อยล็อกและรอ/ปลุกสัญญาณ)
14	เมื่อ thread เรียกใช้ เมท็อดที่เป็น synchronized method, thread นั้นจะเรียกใช้ synchronized method นั้นได้ก็ต่อเมื่อไม่มี thread อื่นเรียก synchronized method ใด ๆ ของ object นั้นอยู่ก่อน	**จริง**	True (การล็อกระดับ Instance Object ตัวเดียวกัน จะครอบคลุมทุก Synchronized Method ของ Object นั้น)
15	thread ที่เรียกใช้ synchronized method ไม่ได้ จะรออยู่ในคิวที่เรียกว่า wait set	**ไม่จริง**	False (จะรออยู่ใน Entry Set ส่วน Wait Set จะไว้สำหรับเธรดที่เรียกคำสั่ง `wait()`)
16	การปลดล็อกทรัพยากรของ object ที่ถูกล็อกโดยการเรียก synchronized method จะเกิดขึ้นเมื่อ นักเขียนโปรแกรมเรียกเมท็อด yield()	**ไม่จริง**	False (`yield()` ไม่ได้ปล่อยการถือครองล็อก จะปลดล็อกเมื่อทำงานออกจากเมท็อดหรือเรียก `wait()`)
17	ถ้ามีการเรียกฟังก์ชัน wait() ใน synchronized method ของ Java thread จะไปรออยู่ใน wait set	**จริง**	True
18	ถ้ามีการเรียกฟังก์ชัน notify() ใน synchronized method ของ Java ตัว thread จะย้ายจาก wait set ไปอยู่ใน entry set	**จริง**	True (เธรดที่ถูกปลุกจะย้ายไป Entry Set เพื่อรอแย่ง Lock กลับมาทำงานต่อ)
19	เมท็อด notify() จะใช้เลือก process ที่นักเขียนโปรแกรมระบุออกจาก wait set	**ไม่จริง**	False (ระบบ JVM จะสุ่มเลือกมาหนึ่งเธรด ไม่สามารถเจาะจงระบุเธรดได้)

20	```
	class SharedData {
	    private int value = 5;
	    public synchronized void increment() {
	        value++;
	    }
	    public synchronized void decrement() {
	        value--;
	    }
	    public int getValue() {
	        return value;
	    }
	}
	
	class Worker1 extends Thread {
	    SharedData data;
	    public Worker1(SharedData data) {
	        this.data = data;
	    }
	    public void run() {
	        for (int i = 0; i < 3; i++) {
	            data.increment();
	        }
	    }
	}
	
	public class Main {
	    public static void main(String[] args) {
	        SharedData data = new SharedData();
	        Worker1 w1 = new Worker1(data);
	        w1.start();
	        try {
	            w1.join();
	        } catch (InterruptedException e) {}
	        System.out.println(data.getValue());
	    }
	}
	``
	จากโปรแกรมภาษา Java ดังรูป ผลลัพธ์ของโปรแกรมคือ
		A 8	A. 8 (ค่าเริ่มต้น 5 + 3 = 8)

21	```
	class SharedData {
	    private int value = 5;
	    private boolean isAvailable = false;
	
	    public synchronized void increment() {
	        while (isAvailable) {
	            try { wait(); } catch
	(InterruptedException e) {}
	        }
	        System.out.println("increment");
	        isAvailable = true;
	        notify();
	    }
	
	    public synchronized void decrement() {
	        while (!isAvailable) {
	            try { wait(); } catch
	(InterruptedException e) {}
	        }
	        System.out.println("decrement");
	        isAvailable = false;
	        notify();
	    }
	
	    public int getValue() {
	        return value;
	    }
	}
	``
	จากโปรแกรมภาษา Java ดังรูป (มีการสลับรอบการทำงานด้วย `wait()` และ `notify()`) ผลลัพธ์ที่เป็นไปได้จากโปรแกรมคือ
		C
		``
		increment
		decrement
		increment
		decrement
		5
		``
			C (เพราะเงื่อนไข `wait()` และ `notify()` บังคับให้ `increment` และ `decrement` ทำงานสลับกันทีละครั้ง)

22	Method ของ Semaphore class ของ Java ที่ทำหน้าที่เหมือน wait() ของ Semaphore คือ	**A** acquire()	A. acquire()
23	Method ของ Semaphore class ของ Java ที่ทำหน้าที่เหมือน signal() ของ Semaphore คือ	**C** release()	C. release()

💻 **ข้อ 24 - 34:** โครงสร้างโค้ด BoundedBuffer ด้วย Semaphore ใน Java

```
import java.util.concurrent.Semaphore;

public class BoundedBuffer {
    private static final int BUFFER_SIZE = 5;
    private Object[] buffer = new Object[BUFFER_SIZE];
    private int in = 0, out = 0;

    // A, B, C: ประกาศและกำหนดค่าเริ่มต้นให้กับ Semaphores
    private Semaphore mutex = //A
    private Semaphore empty = //B
    private Semaphore full =  //C

    public void insert(Object item) {
        try {
            //D
            //E
            buffer[in] = item;
            in = (in + 1) % BUFFER_SIZE;
            //F
            //G
        } catch (InterruptedException e) {}
    }

    public Object remove() {
        Object item = null;
        try {
            //H
            //I
            item = buffer[out];
            out = (out + 1) % BUFFER_SIZE;
            //J
            //K
        } catch (InterruptedException e) {}
        return item;
    }
}
```

24	ที่จุด `//A` ต้องเติม	**C** `new Semaphore(1);`	Mutex Lock
25	ที่จุด `//B` ต้องเติม	**E** `new Semaphore(BUFFER_SIZE);`	นับจำนวนช่องว่าง
26	ที่จุด `//C` ต้องเติม	**D** `new Semaphore(0);`	เริ่มต้นยังไม่มีข้อมูล
27	ที่จุด `//D` ต้องเติม	**F** `empty.acquire();`	รอลดจำนวนช่องว่าง
28	ที่จุด `//E` ต้องเติม	**A** `mutex.acquire();`	เข้า Critical Section ใน insert
29	ที่จุด `//F` ต้องเติม	**B** `mutex.release();`	ออกจาก Critical Section ใน insert
30	ที่จุด `//G` ต้องเติม	**I.** **`full.release();`**	ส่งสัญญาณว่ามีข้อมูลพร้อมอ่านเพิ่มขึ้น 1
31	ที่จุด `//H` ต้องเติม	**H.** **`full.acquire();`**	รอลดจำนวนข้อมูลที่มีให้อ่าน
32	ที่จุด `//I` ต้องเติม	**A** `mutex.acquire();`	เข้า Critical Section ใน remove
33	ที่จุด `//J` ต้องเติม	**B** `mutex.release();`	ออกจาก Critical Section ใน remove
34	ที่จุด `//K` ต้องเติม	**G.** **`empty.release();`**	ส่งสัญญาณว่ามีช่องว่างเพิ่มขึ้น 1

L12. Deadlock — ชุดที่ 11 · 30 ข้อ

ข้อ	คำถาม	คำตอบ	เฉลย / ความรู้
1	Deadlock จะเกิดขึ้นต้องมีตั้งแต่ 2 โพรเซสขึ้นไปเกี่ยวข้อง	จริง	True
2	เงื่อนไขที่จะทำให้เกิด deadlock ที่ระบุว่าทรัพยากรจะถูกใช้งานได้ทีละโพรเซสเท่านั้นคือ	**A** Mutual Exclusion	A. Mutual Exclusion
3	เงื่อนไขที่จะทำให้เกิด deadlock ที่ระบุว่าเมื่อทรัพยากรถูกจัดสรรให้โพรเซสแล้ว ทรัพยากรจะถูกนำมาให้โพรเซสอื่นใช้ได้ เมื่อโพรเซสที่ถือครองอยู่ยอมปล่อยทรัพยากรออกมาเท่านั้นคือ	**C** No Preemption	C. No Preemption
4	ถ้ามีเงื่อนไขข้อใดข้อหนึ่งใน 4 ข้อของ deadlock เป็นจริง จะทำให้เกิด deadlock	ไม่จริง	False (ต้องเป็นจริงครบทั้ง 4 ข้อพร้อมกัน จึงจะเกิด Deadlock)
5	สำหรับ Resource-Allocation Graph ที่ใช้ในการโมเดล deadlock จะเป็นกราฟแบบไม่มีทิศทาง	ไม่จริง	False (เป็น Directed Graph กราฟแบบมีทิศทาง)
6	Request edge ของ resource-allocation graph มีทิศทางจากทรัพยากรไปยังโพรเซส	ไม่จริง	False (Request Edge มีทิศทางจาก $P_i \rightarrow R_j$, ส่วน Assignment Edge มีทิศทางจาก $R_j \rightarrow P_i$)
7	ถ้ามี cycle ใน resource-allocation graph จะหมายความว่าเกิด deadlock ขึ้น	ไม่จริง	False (ถ้าทรัพยากรมีหลายชิ้น/multi-instance การมี cycle เป็นเพียงเงื่อนไขจำเป็น แต่อาจไม่เกิด deadlock ก็ได้)

📊 **ข้อมูลสำหรับข้อ 8 - 14** (กราฟหน้า 2 และ 3)

โครงสร้างกราฟ:

- Processes: P_1, P_2, P_3
- Resources: R_1 (1 instance), R_2 (2 instances), R_3 (1 instance), R_4 (3 instances)
- Edges:
- $P_1 \rightarrow R_1$, $R_1 \rightarrow P_2$
- $R_2 \rightarrow P_1$, $R_2 \rightarrow P_2$, $P_3 \rightarrow R_2$
- $P_2 \rightarrow R_3$, $R_3 \rightarrow P_3$

8	จากรูปมีทรัพยากรกี่ประเภท	**D** 4	R_1, R_2, R_3, R_4
9	จากรูปมีโพรเซส โพรเซส	**C** 3	P_1, P_2, P_3
10	จากรูปทรัพยากร R2 มีกี่ชิ้น	**B** 2	
11	จากรูปทรัพยากร R1 ถูกจัดสรรให้โพรเซสใด	**B** P2	
12	จากรูปโพรเซส P3 ถือครองทรัพยากรใด	**C** R3	มีลูกศรจาก $R_3 \rightarrow P_3$
13	จากรูปโพรเซส P3 ร้องขอทรัพยากรใด	**B** R2	ลูกศร $P_3 \rightarrow R_2$
14	จากรูประบบไม่เกิด deadlock	ไม่จริง	เกิด Deadlock เนื่องจากมี Cycle และทรัพยากรทุกตัวใน Cycle ถูกใช้งานจนเต็ม
15	OS ที่เราใช้กันทั่วไปไม่มีกลไกจัดการ deadlock ในระดับ user process	จริง	True (ส่วนใหญ่ใช้วิธี Ostrich Algorithm คือเพิกเฉยปล่อยให้เป็นหน้าที่ของผู้ใช้หรือโปรแกรมเมอร์)
16	กลไกการจัดการ deadlock ที่ไม่ยอมให้เกิด deadlock ขึ้นในระบบ	**A** Deadlock avoidance	A. Deadlock avoidance (รวมถึง Deadlock Prevention)
17	การกำหนดให้โพรเซสต้องขอใช้ทรัพยากรเรียงตามลำดับจากทรัพยากรที่มีหมายเลขน้อยไปหามาก จะใช้แก้ปัญหา no preemption	ไม่จริง	False (ใช้ป้องกันเงื่อนไข Circular Wait)
18	แนวคิดของ deadlock avoidance คือระบบต้องอยู่ใน safe state เสมอ	จริง	True
19	เงื่อนไขหนึ่งของ deadlock avoidance คือโพรเซสจะต้องระบุทรัพยากรที่ต้องการใช้ทั้งหมดก่อนที่มันจะเริ่มทำงาน	จริง	True (ต้องรู้ Max Claim ล่วงหน้า)
20	การใช้ resource-allocation graph ในการทำ deadlock avoidance จะใช้ได้กับกรณีที่ทรัพยากรหนึ่งประเภทมีเพียงหนึ่งชิ้นเท่านั้น	จริง	True (ถ้ามีหลายชิ้นต้องใช้อัลกอริทึม Banker's Algorithm)
21	ด้านที่เพิ่มเข้ามาใน resource allocation graph เพื่อใช้ในการทำ deadlock avoidance คือ	**B** Claim Edge	B. Claim Edge (เส้นประ $P_i \dashrightarrow R_j$)
22	ถ้าทดลองว่าให้ทรัพยากรแล้ว ระบบไม่อยู่ใน Safe State OS จะเปลี่ยน... เป็น...	**D** Request Edge, Claim Edge	D (หรือหากในขั้นตอนการทดสอบ ระบบจะปฏิเสธการขอและไม่เปลี่ยนเป็น Assignment edge)
23	ระบบจะอยู่ใน Safe State เมื่อ OS เปลี่ยน claim edge เป็น assignment edge แล้ว ไม่พบว่ามี cycle ใน graph	จริง	True

📊 **ข้อมูลสำหรับข้อ 24 - 25** (กราฟหน้า 5)

โครงสร้าง: P_1, P_2 และ R_1, R_2 โดยมี $R_1 \rightarrow P_1$, $P_2 \rightarrow R_1$, เส้นประ $P_1 \dashrightarrow R_2$, เส้นประ $P_2 \dashrightarrow R_2$

24	จากรูป claim edge คือ	**A** P1 R2	
25	จากรูปถ้าโพรเซส P1 ร้องขอทรัพยากร R2 ระบบจะยอมให้การร้องขอนี้เกิดขึ้น	**ไม่จริง**	เพราะถ้าให้ R_2 แก่ P_1 จะเกิด Cycle และนำไปสู่ Unsafe state
26	กลไกการทำ detection เพื่อแก้ปัญหา critical section คือเปลี่ยน resource-allocation graph เป็น wait-for graph และตรวจหา cycle ใน wait-for graph	**ไม่จริง**	False (Wait-for graph ใช้สำหรับตรวจจับ **Deadlock** ไม่ใช่ Critical Section)
27	จากรูปหน้า 6 เมื่อเปลี่ยน resource-allocation graph เป็น wait-for graph แล้วจะพบว่า p2 ไม่ได้รอ p4 โดยตรง	**จริง**	True (ลูกศรผ่านทรัพยากรหลายทอด)
28	จากรูปหน้า 6 ทุกโพรเซสในระบบติดอยู่ใน deadlock	**ไม่จริง**	False (มีบางโพรเซส เช่น P_5 ที่สามารถทำงานจนเสร็จแล้วคืนทรัพยากรได้)
29	หลังจากที่มีการตรวจสอบพบว่าเกิด deadlock ขึ้นในระบบ วิธีเดียวในการแก้ปัญหาคือจบการทำงานของทุกโพรเซสที่เกี่ยวข้องกับ deadlock	**ไม่จริง**	False (สามารถใช้วิธียกเลิกทีละโพรเซส หรือทำ Resource Preemption ได้)
30	การแก้ปัญหา deadlock อาจทำให้เกิด starvation ได้	**จริง**	True (กรณีโพรเซสเดิมถูกเลือกเป็น Victim ให้ Rollback ซ้ำๆ)

L13. Real Memory Management (Contiguous scheme) — ชุดที่ 12.1 · 15 ข้อ

ข้อ	คำถาม	คำตอบ	เฉลย / ความรู้
1	ตำแหน่งของหน่วยความจำที่อ้างถึงโดย CPU คือ	**E** A หรือ C	E. A หรือ C (Logical address หรือ Virtual Address)
2	ตำแหน่งของหน่วยความจำที่อ้างถึงโดยฮาร์ดแวร์หน่วยความจำ	**B** Physical Address	B. Physical Address
3	ฮาร์ดแวร์ที่ใช้แปลงจาก logical address เป็น physical address	**A** Memory Management Unit	A. Memory Management Unit (MMU)
4	ไฟล์ที่มีนามสกุล .dll ของ Windows เป็นตัวอย่างของ	**B** Dynamic Linking	B. Dynamic Linking (Dynamic Link Library)
5	สำหรับแนวคิดของ dynamic linking ถ้าไลบรารีที่โพรเซสต้องการใช้ถูกโหลดเข้าหน่วยความจำโดยโพรเซสอื่นแล้ว โพรเซสนั้นจะสามารถใช้ไลบรารีตัวนั้นได้โดยไม่ต้องโหลดไลบรารีนั้นใหม่	**จริง**	True (Shared Library)
6	การจัดสรรหน่วยความจำให้กับโพรเซสแบบที่มีการแบ่งหน่วยความจำเป็นส่วน (partition) โดยที่ขนาดของแต่ละส่วนไม่จำเป็นต้องเท่ากัน แต่จะไม่สามารถเปลี่ยนจำนวนส่วนหรือขนาดของส่วนได้หลังจากเริ่มการทำงานของระบบไปแล้วเรียกว่า	**B** Fixed Partitions	B. Fixed Partitions
7	สำหรับการจัดสรรหน่วยความจำแบบ Fixed Partitions ณ. ขณะเวลาใดเวลาหนึ่งจะมีจำนวนโพรเซสสูงสุดได้เท่ากับจำนวนส่วน (partition) ที่แบ่งไว้	**จริง**	True (Multiprogramming Degree ถูกจำกัดด้วยจำนวนพาร์ติชัน)
8	สำหรับการจัดสรรหน่วยความจำแบบ Fixed Partitions การควบคุมไม่ให้โพรเซสไปใช้หน่วยความจำในส่วนอื่นนอกจากส่วนที่มันได้รับการจัดสรร จะใช้แนวคิดของ base และ limit รีจิสเตอร์ โดยที่ limit รีจิสเตอร์จะเก็บตำแหน่งเริ่มต้นของส่วนที่โพรเซสได้รับการจัดส...	**ไม่จริง**	False (สลับหน้าที่กัน: Base register เก็บตำแหน่งเริ่มต้น และ Limit register เก็บขนาด)
9	สมมติโพรเซสได้รับการจัดสรรลงในส่วนที่มีค่าใน base รีจิสเตอร์ 1000 และ limit รีจิสเตอร์ 2000 ถ้ามีการอ้าง logical address 1800 ถือว่าเป็นการอ้างตำแหน่งหน่วยความจำที่ถูกต้อง	**จริง**	True (1800 < 2000 จึงถูกต้อง)
10	สมมติโพรเซสได้รับการจัดสรรลงในส่วนที่มีค่าใน base รีจิสเตอร์ 1000 และ limit รีจิสเตอร์ 2000 ถ้ามีการอ้าง logical address 2000 ถือว่าเป็นการอ้างตำแหน่งหน่วยความจำที่ถูกต้อง	**ไม่จริง**	False (Logical Address ต้องน้อยกว่า Limit: $0 \le$ address < limit ดังนั้น 2000 จะเกิด Trap Addressing Error)
11	ปัญหาที่เกิดขึ้นกับวิธีการ Fixed Partitions ที่ทำให้มีหน่วยความจำเหลือว่างอยู่แต่ไม่มีโพรเซสใดสามารถใช้ได้เรียกว่า	**A** internal fragmentation	A. internal fragmentation
12	ปัญหาที่เกิดขึ้นกับวิธีการ variable Partitions ที่ทำให้มีหน่วยความจำเหลือว่างอยู่แต่ไม่มีโพรเซสใดสามารถใช้ได้เรียกว่า	**B** external fragmentation	B. external fragmentation
13	ในวิธี variable Partitions ถ้ามีเนื้อที่ว่างเหลืออยู่ในหน่วยความจำ เรียงตามตำแหน่งหน่วยความจำจากน้อยไปหามาก ดังต่อไปนี้ 16, 10, 5, 20 และ 3 MB ตามลำดับ ถ้ามี process ใหม่ที่ต้องการเนื้อที่หน่วยความจำ 4 MB และ OS จัดสรรให้ลงในช่อง 16 MB อัลก...	**E** first-fit	E. first-fit (เลือกช่องแรกสุดที่ขนาดเพียงพอ)
14	จากข้อมูลในข้อ 13 ถ้า OS จัดสรรให้ลงในช่อง 20 MB อัลกอริทึมที่ OS ใช้คือ	**C** worst-fit	C. worst-fit (เลือกช่องที่ใหญ่ที่สุด)
15	ในวิธี variable Partitions การขยับให้ process มาใช้เนื้อที่หน่วยความจำที่อยู่ติดกัน เพื่อให้เหลือเนื้อที่หน่วยความจำที่ว่างเรียงต่อกันเป็นผืนเดียวเรียกว่า	**G. compaction**	G. compaction

L14. Real Memory Management (Noncontiguous) — ชุดที่ 12.2 · 22 ข้อ

ข้อ	คำถาม	คำตอบ	เฉลย / ความรู้
1	แนวคิดการจัดสรรหน่วยความจำแบบ paging คือหน่วยความจำ จะถูกแบ่งเป็นบล็อกขนาดเท่าๆ กัน แต่ละบล็อกจะเรียกว่า page	**ไม่จริง**	False (บล็อกขนาดเท่ากันใน Physical Memory จะเรียกว่า **Frame** ส่วน **Page** คือบล็อกของ Logical Memory)
2	การจัดสรรหน่วยความจำจริง (real memory) ในระบบ paging โปรแกรมจะสามารถถูกโหลดเข้าไปทำงานได้ ถ้าจำนวน frame ที่ว่างเท่ากับจำนวน page	**จริง**	True
3	ถ้า CPU อ้างถึงตำแหน่งหน่วยความจำโดยใช้จำนวนบิต 16 บิต ขนาดหน่วยความจำสูงสุดที่ระบบปฏิบัติการนี้จะใช้ได้คือ ไบต์	**E** 65536	E. 65536 ไบต์ (2^{16} = 65,536)
4	ถ้า CPU อ้างถึงตำแหน่งหน่วยความจำโดยใช้จำนวนบิต 16 บิต ถ้า 1 page มีขนาด 16 ไบต์ จะได้ว่าจาก 16 บิตดังกล่าวจะถูกแบ่งมาใช้ระบุ page offset จำนวนกี่บิต	**A** 4	A. 4 บิต (2^4 = 16)
5	จากข้อ 4 จะได้ว่าหนึ่งโปรแกรมจะมีจำนวน page ได้สูงสุดกี่ page	**D** 4096	D. 4096 (16 - 4 = 12 บิต สำหรับเลขหน้า, 2^{12} = 4096)
6	ในระบบ paging จะใช้เพื่อใช้แปลง logical address ให้เป็น physical address	**C** Page table	C. Page table
📊 **ข้อมูลสำหรับข้อ 7 - 10** (รูปหน้า 2 และ 3) *โครงสร้าง*: Logical memory ขนาด 16 bytes แบ่งเป็น 4 pages (เพจละ 4 bytes: 0-3, 4-7, 8-11, 12-15). Page table: Page 0 → Frame 5, Page 1 → Frame 6, Page 2 → Frame 1, Page 3 → Frame 2.			
7	จากรูปโปรแกรมถูกแบ่งออกเป็นกี่ page	**D** 4	
8	จากรูป page แรกของโปรแกรมถูกโหลดเข้า frame ใดในหน่วยความจำ	**E** 5	
9	จากรูถ้าต้องการอ้างถึงตำแหน่งหน่วยความจำของตัว "g" (อยู่ที่ logical address 6) logical address จะอ้างถึง page ที่ และ offset ที่	**C** 1, 2	6 / 4 = 1, 6 mod 4 = 2
10	จากรูปตัว "g" ถูกเก็บไว้ที่ frame ที่ และ offset ที่	**E** 6, 2	Frame 6, Offset 2
11	ถ้าการเข้าถึง TLB ใช้เวลา 0.25 ไมโครวินาที และ การเข้าถึงหน่วยความจำใช้เวลา 1 ไมโครวินาที สมมติ Effective Access Time (EAT) มีค่า 1.35 ไมโครวินาที จะได้ว่า Hit Ratio มีค่ากี่เปอร์เซนต์	**E** 90	E. 90% EAT = h(0.25 + 1.0) + (1-h)(0.25 + 2.0) = 1.35 ⟹ 1.25h + 2.25 - 2.25h = 1.35 ⟹ h = 0.90
12	ในระบบ paging การป้องกันไม่ให้โพรเซสไปอ้างถึง page ที่มันไม่มีสิทธิใช้จะทำได้โดยกำหนดสิ่งที่เรียกว่า ไว้ใน page table	**D** valid-invalid bit	D. valid-invalid bit
13	สมมติผู้ใช้คนหนึ่งสั่งรันโปรแกรม vi สมมติอีกว่าส่วนที่เป็นโค้ดของโปรแกรม vi แบ่งเป็น 5 page และมี page ที่เป็น data 2 page ถ้าใช้แนวคิดของการ shared page ถ้าผู้ใช้อีกคนหนึ่งสั่งรันโปรแกรม vi จะมีจำนวนเฟรมในหน่วยความจำที่ใช้ร่วมกันระหว่าง 2 ...	**E** 5	E. 5 เฟรม (แชร์เฉพาะหน้าส่วนที่เป็น Code 5 หน้า)
14	แนวคิดของการจัดสรรหน่วยความจำแบบ segmentation ก็คือการแบ่งโปรแกรมออกเป็นส่วน ๆ แต่ละส่วนไม่จำเป็นต้องมีขนาดเท่ากัน แต่เวลาโหลดแต่ละส่วนเข้าสู่หน่วยความจำ จะต้องโหลดให้อยู่ต่อเนื่องกันในหน่วยความจำ	**ไม่จริง**	False (แต่ละเซกเมนต์กระจายตัวอยู่คนละตำแหน่งในแรมได้ ไม่จำเป็นต้องต่อเนื่องกันทั้งโปรแกรม)
15	ในระบบ segmentation แต่ละโพรเซสจะมี segment table เพื่อใช้แปลง logical address เป็น physical address	**จริง**	True
16	ใน segment table ส่วนที่ใช้ระบุตำแหน่งเริ่มต้นของ segment ในหน่วยความจำคือ	**A** base	A. base
17	ใน segment table ส่วนที่ใช้ระบุขนาด segment ในหน่วยความจำคือ	**C** limit	C. limit
18	**ตาราง Segment Table:** • Segment 0: limit 1000, base 1400 • Segment 1: limit 400, base 6300 • Segment 2: limit 400, base 4300 • Segment 3: limit 1100, base 3200 • Segment 4: limit 1000, base 4700 ถ้า logical address อยู่ในรูป segment number:offset และระบุค่ามาดังนี้ 2:300 จะได้ physical address คือ	**C** 4600	C. 4600 (Segment 2: limit 400, offset 300 < 400 ถูกต้อง; Base 4300 + 300 = 4600)
19	จาก segment table ในข้อ 18 ถ้า logical address ระบุมาเป็น 1:400 จะได้ physical address คือ	**E** error	E. error (Segment 1 มี limit 400, offset 400 ไม่น้อยกว่า limit จึงเกิด Trap Addressing Error)
20	การจัดสรรหน่วยความจำแบบ segmentation ยังมีปัญหาของ external fragmentation อยู่	**จริง**	True (เนื่องจากแต่ละเซกเมนต์มีขนาดไม่เท่ากัน)
21	ใน OS 16 บิต ถ้า page size คือ 256 ไบต์ และ 1 ช่องของ page table ใช้ 2 ไบต์ ถ้าใช้ single level paging และขนาดของโปรแกรมคือ 512 ไบต์ ขนาดของ page table คือกี่ไบต์	**D** 512	D. 512 ไบต์ Virtual Address Space 16 บิต, Page size 256 B (2^8) ⟹ ต้องมี 2^{16-8} = 256 Entries. ตารางขนาด 256 × 2 B = 512 ไบต์

22	ใน OS 16 บิต ถ้า page size คือ 256 ไบต์ และ 1 ช่องของ page table ใช้ 2 ไบต์ ถ้าใช้ 2-level paging โดยจำนวนบิตของ outer page table และ inner page table คือ 4 บิต และขนาดของโปรแกรมคือ 512 ไบต์ ขนาดของ page table คือกี่ไบต์	**A** 64	A. 64 ไบต์ Outer table = $2^4 \times 2$ B = 32 B. ขนาดโปรแกรม 512 B ต้องการ Inner table เพียง 1 ตัว = $2^4 \times 2$ B = 32 B. รวม 32 + 32 = 64 ไบต์

L15. Virtual Memory — ชุดที่ 13 · 22 ข้อ

ข้อ	คำถาม	คำตอบ	เฉลย / ความรู้
1	แนวคิดของ virtual memory จะทำให้ขนาดของ logical address ใหญ่กว่า physical address ได้	**จริง**	True
2	แนวคิดของ virtual memory โปรแกรมไม่จำเป็นต้องถูกโหลดเข้าหน่วยความจำทั้งโปรแกรม	**จริง**	True (โหลดเฉพาะส่วนที่กำลังใช้งาน)
3	ในการใช้ virtual memory แบบ demand paging เรายังใช้ page table ในการแปลง logical address เป็น physical address	**จริง**	True
4	บิตใน page table ที่ใช้เพื่อระบุว่า page ถูกโหลดเข้าสู่หน่วยความจำหรือยังคือ	**D** valid-invalid bit	D. valid-invalid bit
5	ในกรณีที่มีการอ้างถึง page ที่ไม่อยู่ในหน่วยความจำจะเกิดสิ่งที่เรียกว่า	**C** page fault	C. page fault
6	page fault จะเกิดขึ้นเมื่อโพรเซสอ้างถึงหน่วยความจำที่ตัวเองไม่มีสิทธิเข้าถึงเท่านั้น	**ไม่จริง**	False (เกิดขึ้นเมื่อโปรเซสอ้างถึงตำแหน่งที่มีสิทธิเข้าถึง แต่หน้านั้นยังไม่ได้ถูกดึงเข้ามาในแรม)
7	เมื่อเกิด page fault เนื่องจากส่วนของโปรแกรมที่ต้องการยังไม่ได้ถูกโหลดเข้าสู่หน่วยความจำ ระบบปฏิบัติการจะทำงานต่อไปนี้ยกเว้น	**E** เปลี่ยนค่า valid-invalid bit เป็น invalid	E. เปลี่ยนค่า valid-invalid bit เป็น invalid (ต้องเปลี่ยนเป็น **valid** เพราะโหลดเข้ามาสำเร็จแล้ว)
8	ในกรณีที่เกิด page fault และ ในหน่วยความจำไม่มีเฟรมที่ว่างเหลือแล้ว ระบบปฏิบัติการจะแจ้งข้อความข้อผิดพลาดให้ผู้ใช้ทราบ	**ไม่จริง**	False (OS จะเรียกกระบวนการ Page Replacement เพื่อสลับเพจเก่าออก)
9	วิธีวัดประสิทธิภาพของอัลกอริทึมการแทนที่เพจ ก็คือดูจากการเกิด page fault ถ้าอัลกอริทึมไหนเกิด page fault มากกว่า อัลกอริทึมนั้นจะดีกว่า	**ไม่จริง**	False (อัลกอริทึมที่ดีกว่าต้องเกิด Page Fault **น้อยกว่า**)
	📊 **ข้อมูลสำหรับข้อ 10 - 16** • หน่วยความจำมี 3 เฟรม ปัจจุบันเก็บเพจ: `2, 0, 1` (ลำดับการเข้า: 2 มาก่อน ตามด้วย 0 และ 1) • Page Reference String ที่เหลือ: `7 0 1 2 0 3 0 4 2 3 0`		
10	ด้วย FIFO เมื่อมีการเรียกใช้ page 7 จะแทนที่เพจใด	**C** 2	
11	ด้วย FIFO จะเกิด page fault ทั้งหมดกี่ครั้ง	**E** 7	
12	ด้วย Optimal เมื่อมีการเรียกใช้ page 7 จะแทนที่เพจใด	**C** 2	เพจ 2 ถูกใช้ไกลสุดในอนาคต
13	ด้วย Optimal จะเกิด page fault ทั้งหมดกี่ครั้ง	**C** 5	
14	วิธีการ optimal เป็นวิธีการที่ระบบปฏิบัติการนิยมใช้ เนื่องจากให้ผลลัพธ์เป็นจำนวน page fault น้อยที่สุด	**ไม่จริง**	เป็นวิธีในอุดมคติ ไม่สามารถนำมาใช้จริงได้เพราะไม่สามารถล่วงรู้อนาคต
15	ด้วย LRU เมื่อมีการเรียกใช้ page 7 จะแทนที่เพจใด	**C** 2	
16	ด้วย LRU จะเกิด page fault ทั้งหมดกี่ครั้ง	**E** 7	
17	การเลือกเฟรมที่จะถูกแทนที่จะถูกเลือกจากเฟรมที่ระบบปฏิบัติการจัดสรรให้กับโพรเซสนั้นเท่านั้น	**ไม่จริง**	False (มีทั้ง Local Replacement และ Global Replacement ที่ดึงเฟรมของโพรเซสอื่นได้)
18	เหตุการณ์ที่ระบบต้องสลับเพจเข้าและออกอยู่เกือบตลอดเวลาเรียกว่า	**D** thrashing	D. thrashing
19	ถ้าเกิด thrashing แล้วระบบปฏิบัติการตรวจสอบไม่พบ อาจจะเป็นผลให้ระบบปฏิบัติการโหลดโพรเซสเข้ามามากขึ้น เนื่องจาก CPU ไม่ถูกใช้งานเต็มที่	**จริง**	True (CPU Utilization ตกต่ำ ทำให้ OS นึกว่าระบบว่าง จึงเพิ่ม Multiprogramming Degree จนยิ่งแย่ลง)
20	สมมติ working set window คือ 10 และที่เวลา 11 โพรเซส p1 มี page reference string ดังนี้ `2 6 1 5 7 7 7 7 5 1` และ p2 มี page reference string เป็น `1 2 0 3 4 5 6 6 6 7` คอมพิวเตอร์จะต้องมีเฟรมว่างอย่างน้อยกี่เฟรม จึงจะไม่เกิด thrashing	**C** 13	C. 13 เฟรม ($WSS(P_1) = \{1, 2, 5, 6, 7\} = 5$; $WSS(P_2) = \{0, 1, 2, 3, 4, 5, 6, 7\} = 8$; รวม 5 + 8 = 13)

21	```
	public class Main {
	    public static void main(String[] args) {
	        int[][] a = new int[10][256];
	        for(int j = 0; j < 256; j++) {
	            for(int i = 0; i < 10; i++) {
	                a[i][j] = 0;
	            }
	        }
	    }
	}
	``
	สมมติ 1 แถวของอะเรย์ใช้เนื้อที่ 1 เพจ และหน่วยความจำมีเนื้อที่ว่าง 1 เฟรม ในการกำหนดค่าเริ่มต้นให้อะเรย์ a จะเกิด page fault กี่ครั้ง
		D 2560	D. 2560 ครั้ง (วนลูปตามหลัก Column-major เข้าถึงข้ามแถวทุกรอบ เกิด Page Fault ทุกครั้งที่เข้าถึง 256 × 10 = 2560)

22	```
	public class Main {
	    public static void main(String[] args) {
	        int[][] a = new int[10][256];
	        for(int i = 0; i < 10; i++) {
	            for(int j = 0; j < 256; j++) {
	                a[i][j] = 0;
	            }
	        }
	    }
	}
	``
	สมมติ 1 แถวของอะเรย์ใช้เนื้อที่ 1 เพจ และหน่วยความจำมีเนื้อที่ว่าง 1 เฟรม ในการกำหนดค่าเริ่มต้นให้อะเรย์ a จะเกิด page fault กี่ครั้ง
		A 10	A. 10 ครั้ง (วนลูปตามแถว Row-major เพจแถวเดิมถูกโหลด 1 ครั้งแล้วใช้งานต่อเนื่องครบ 256 ช่อง จึงเกิด Page Fault เพียงแถวละ 1 ครั้ง รวม 10 ครั้ง)

L16. File System — ชุดที่ 14 · 30 ข้อ

ข้อ	คำถาม	คำตอบ	เฉลย / ความรู้
1	ระบบไฟล์ทำให้การเข้าถึงไฟล์ผ่านทางซอฟต์แวร์มีลักษณะเดียวกันไม่ว่าอุปกรณ์ที่จัดเก็บข้อมูลจะเป็นอะไร	จริง	True (File System Abstraction)
2	หน้าที่หนึ่งของระบบไฟล์คือควบคุมสิทธิในการเข้าใช้งานไฟล์	จริง	True
3	ระบบไฟล์ที่ใช้กับระบบปฏิบัติการ Windows เป็นหลัก (เข้ากันไม่ค่อยได้กับระบบปฏิบัติการอื่น ๆ)	**B** NTFS	B. NTFS
4	ระบบไฟล์ที่ใช้กับระบบปฏิบัติการ Linux เป็นหลัก (เข้ากันไม่ค่อยได้กับระบบปฏิบัติการอื่น ๆ)	**C** ext4	C. ext4
5	ระบบไฟล์ที่ใช้กับระบบปฏิบัติการ Apple เป็นหลัก (เข้ากันไม่ค่อยได้กับระบบปฏิบัติการอื่น ๆ)	**D** APFS	D. APFS (Apple File System)
6	ระบบไฟล์ที่เข้ากันได้กับระบบปฏิบัติการหลาย ๆ ตัว	**A** exFAT	A. exFAT
7	ระบบไฟล์ที่มักจะใช้กับ External Storage อย่าง USB drive คือ	**A** exFAT	A. exFAT
8	ประโยชน์ของคุณสมบัติ Journaling ของระบบไฟล์บางตัวอย่าง NTFS คือ การเข้ารหัสไฟล์	ไม่จริง	False (Journaling ใช้บันทึก Log การเปลี่ยนแปลงโครงสร้างไฟล์เพื่อกู้คืนเมื่อระบบล่ม ไม่ใช่การเข้ารหัส)
9	ปัญหาของการจัดสรรเนื้อที่ให้กับไฟล์แบบต่อเนื่อง (contiguous) คือ internal fragmentation	ไม่จริง	False (ปัญหาสำคัญที่สุดของการจัดสรรต่อเนื่องคือ **External Fragmentation**)
10	ปัญหาของการจัดสรรเนื้อที่ให้กับไฟล์แบบต่อเนื่อง (contiguous) คือ การเพิ่มขนาดของไฟล์อาจจะทำได้ยาก	จริง	True (หากบล็อกถัดไปไม่ว่าง จะต้องย้ายไฟล์ทั้งไฟล์ไปที่อื่น)
11	การจัดสรรเนื้อที่ให้ไฟล์แบบ linked allocation เป็นการจัดสรรเนื้อที่ให้ไฟล์แบบต่อเนื่อง (contiguous)	ไม่จริง	False (Linked Allocation เป็นแบบ Non-contiguous)
12	ข้อเสียของการจัดสรรเนื้อที่แบบ linked allocation คือจะต้องเสียเนื้อที่ในบล็อกเพื่อเก็บลิงก์ไปยังบล็อกถัดไป	จริง	True (Pointer Overhead)
13	ข้อเสียของการจัดสรรเนื้อที่แบบ linked allocation คือถ้ามีบล็อกใดบล็อกหนึ่งเสีย อาจทำให้อ่านข้อมูลที่เหลือของไฟล์ไม่ได้	จริง	True (ลิงก์ขาดตอน)
14	การจัดสรรเนื้อที่แบบ linked allocation มีข้อเสียคือการอ่านไฟล์ทำได้ช้า	จริง	True (ไม่รองรับ Random/Direct Access ต้องไล่อ่านตามลำดับ)
15	FAT จัดเป็น Linked Allocation	จริง	True (เป็น Linked Allocation ที่ดึงพอยน์เตอร์มาเก็บรวมในตาราง FAT)
16	ขนาดสูงสุดของไฟล์สำหรับระบบที่ใช้ระบบไฟล์แบบ FAT32 คือ	**C** 4 GB	C. 4 GB (2^{32} - 1 bytes)
17	สมมติหนึ่งช่องของ FAT Table มีขนาด 16 บิต และทั้ง 16 บิตถูกใช้อ้างถึงบล็อกข้อมูล โดยขนาดของบล็อกข้อมูลหนึ่งบล็อกคือ 16K ขนาดของวอลุม (volume) สูงสุดของดิสก์คือกี่ GB	**A** 1	A. 1 GB (2^{16} = 65,536 บล็อก × 16 KB = 1,048,576 KB = 1 GB)
18	ปัญหาของขนาดของบล็อกที่มีขนาดใหญ่คือ external fragmentation	ไม่จริง	False (บล็อกใหญ่ทำให้เกิดปัญหา **Internal Fragmentation**)
19	ข้อดีประการหนึ่งของการเข้าถึงไฟล์แบบ index allocation คือ การเข้าถึงข้อมูลในไฟล์ไม่จำเป็นต้องเข้าถึงตามลำดับ	จริง	True (รองรับ Direct/Random Access)
20	ในการใช้งาน index allocation แบบ single level ขนาดของไฟล์สูงสุดจะถูกจำกัดด้วยขนาดของจำนวนช่อง (entry) ใน index block	จริง	True (เพราะมี Index block ได้เพียงก้อนเดียว)
21	สมมติ 1 บล็อกมีขนาด 2 KB และ pointer มีขนาด 4 bytes จะได้ว่า 1 index block จะมีจำนวนช่อง (entry) กี่ช่อง	**C** 512	C. 512 ช่อง (2048 / 4 = 512)
22	สมมติ 1 บล็อกมีขนาด 2 KB และ pointer มีขนาด 4 bytes ขนาดสูงสุดของไฟล์ที่เก็บได้ใน Double indirect block อย่างเดียวคือกี่ MB	**C** 512	C. 512 MB 512 × 512 = 262,144 data blocks × 2 KB = 524,288 KB = 512 MB
23	Symbolic link มีชื่อเรียกอีกอย่างว่า hard link	ไม่จริง	False (เรียกว่า Soft Link)
24	Symbolic link เปรียบได้กับการสร้าง shortcut ใน Windows	จริง	True
25	Hard Link ในระบบปฏิบัติการ Unix จัดการโดยใช้แนวคิด Reference Counter	จริง	True (นับ Link count ใน inode)
26	เราสามารถสร้าง Hard Link ไปยังไฟล์หรือไดเรกทอรีก็ได้	ไม่จริง	False (ระบบ Unix ส่วนใหญ่ไม่อนุญาตให้สร้าง Hard link ไปยังไดเรกทอรีเพื่อป้องกัน Loop ในโครงสร้างระบบไฟล์)
27	เราสามารถสร้าง Soft Link ไปยังไฟล์หรือไดเรกทอรีก็ได้	จริง	True
28	คำสั่ง softln ใน Unix ใช้สร้าง Soft Link	ไม่จริง	False (ใช้คำสั่ง `ln -s`)

29	เราไม่สามารถสร้าง mount point มากกว่าหนึ่ง mount point ที่ mount อุปกรณ์เดียวกันได้	**ไม่จริง**	False (สามารถ Mount อุปกรณ์เดียวกันไว้ที่หลายโฟลเดอร์/Mount point ได้)
30	ถ้าต้องการกำหนดสิทธิในการเข้าถึงไฟล์ดังนี้ - เจ้าของ: อ่านได้ เขียนได้ execute ได้ (`rwx` = 4+2+1 = 7) - ผู้ใช้ในกลุ่มเดียวกัน: อ่านได้ เขียนได้ execute ไม่ได้ (`rw-` = 4+2+0 = 6) - คนอื่นในระบบ: อ่านได้อย่างเดียว (`r--` = 4+0+0 = 4) สิทธิการเข้า...	**C** 0764	C. 0764

M. แผ่นสูตรเร็วท้ายเล่ม (อ่าน 3 นาทีก่อนเข้าห้อง)

📖 รู้จักส่วนนี้ก่อนอ่าน

ความหมาย: สรุปสูตรคำนวณทุกบทมารวมใน 1 หน้า (TAT/WT, Paging, EAT, นับ fault, Banker's, inode, rwx)

ความสำคัญกับข้อสอบ: เป็น index ระหว่างทำข้อสอบ — ตันหัวข้อไหนกลับมาเปิดหน้านี้ก่อน แล้วค่อยไล่ไปส่วนเต็มที่อยู่หน้าก่อน ๆ

ใช้จริงในคอมพิวเตอร์: สูตรพวกนี้คือสิ่งที่ OS คำนวณจริงทุกมิลลิวินาที (แปลง address ทุกครั้งที่เข้าถึง RAM) — เข้าใจสูตร = เข้าใจการทำงานเครื่อง

⏱️ CPU Scheduling

TAT	Completion − Arrival
WT	TAT − Burst (หา TAT ก่อนเสมอ!)
RR	รอไม่เกิน (n−1)q · q ใหญ่=FCFS
เท่ากัน	เลือกมาถึงก่อน (FCFS tiebreak)

🧮 Paging

offset	$\log_2$(page size) บิต
page#	บิต address − บิต offset
เพจ	ceil(ขนาด ÷ page size) — เริ่มนับ 0
PA	(frame × pageSize) + offset
EAT	2 + ε − α (ตย. ε=0.1, α=0.8 → 1.3µs)
2-level	inner = ceil(เพจ ÷ 1024) × 4KB + 4KB

💾 Page Replacement (7,0,1,2,0,3,0,4,2,3,0,3,2,1,2,0,1,7,0,1 / 3 frames)

FIFO	15	เข้าก่อนออกก่อน · มี Belady (3f=9, 4f=10)
OPT	9	มองอนาคต ใช้จริงไม่ได้
LRU	12	มองอดีต ใช้จริงได้
EAT(p)	(1−p)×mat + p×service	
128×128	row นอก = 128 · col นอก = 16,384	

🏦 Banker's · 📁 File

Need	Max − Allocation
ให้ได้ไหม	Req ≤ Need และ ≤ Avail แล้ว safety ใหม่
seg	d < Limit → PA = Base + d (≥ = trap)
fit	First=หลุมแรก Best=เหลือน้อยสุด Worst=ใหญ่สุด
inode	entries = block÷ptr (4K/4B=1024) · 48KB+4MB+4GB+4TB
rwx	r=4 w=2 x=1 · 761 = rwx rw- --x

🚨 กลยุทธ์ห้องสอบ (จากรีวิวรุ่นพี่)

• ถูก/ผิด: ถูก +2 ผิด −1 ไม่ตอบ 0 — **ไม่แน่จริงอย่าตอบ** · • ทำโจทย์คำนวณ (Paging/Replacement (การแทนที่)/Scheduling) **ก่อนข้อทฤษฎี** คะแนน/นาทีสูงกว่า · • จับเวลาข้อคำนวณ ≤ 2 นาที ติดให้ข้าม · • เขียนขั้นตอนสั้น ๆ ข้างคำตอบ · • โค้ดทุกข้อ compile ผ่านหมด — ไม่มีตอบ Error!